Laporan Tugas Besar 1

IF3170 Inteligensi Artifisial 2026/2027

Kelompok 35:

Eliana Natalie Widjojo 13524116

Safira Berlianti 13524128

Varistha Devi 13524135

1. Deskripsi Persoalan

1.1 Latar Belakang

Keluarga **Mejiro** sedang berencana mengekspansi bisnisnya ke bidang logistik darat dan ingin melakukan uji coba berskala kecil terlebih dahulu untuk torasen. Sebagai teman yang baik, **Mejiro McQueen** menawarkan posisi driver truk ujicoba kepada **Special Week**, karena sudah sering menyetir truk di kampung halamannya di Hokkaido. Usaha baru ini bernama **Mejiro Express** dengan slogan “Lebih kilat dari Umamusume”.

Karena **Special Week** sibuk berlatih untuk memenangkan lomba-lombanya, dia tidak bisa membuang waktu berlama-lama untuk menentukan pengaturan pengiriman yang optimal. Oleh karena itu, dia meminta kalian, toreena-toreena yang pintar dalam keinformatikawan, untuk membuat suatu program yang cerdas untuk menolongnya.

1.2 Spesifikasi

Tugas ini bertujuan untuk membuat konfigurasi pengisian paket paling optimal untuk sebuah truk peti kemas. Optimal disini berarti **memaksimalkan** fungsi objektif.

Daftar Variabel Permasalahan

Entitas	Atribut	Contoh	Keterangan
Package	id	“RusdiPackage”	*Identifier*
	Dimensions	{w:3, l:4, h:1}	Ukuran sebuah paket, yaitu *width*, *length*, dan *height*
	Orientation	{w:x, l:y, h:z}	Sumbu yang sejajar dengan setiap sisi paket. Terdapat enam kemungkinan orientasi, misalnya {w:y, l:x, h:z} setelah rotasi 90° terhadap sumbu *z*.
	Value	60	Nilai harga sebuah paket (satuan unit)
	Weight	5	Berat paket (satuan unit berat)
	IsFragile	False	Benda pecah-belah
	ETA	3	Estimated Time of Arrival, perkiraan waktu kedatangan paket (dalam sisa hari)
Truck	Dimensions	{w:10, l:20, h:5}	Ukuran sebuah peti kemas pada truk, yaitu *width*, *length*, dan *height*
	MaxCapacity	500	Kapasitas angkut maksimum pada suatu peti kemas (satuan unit berat)

Ketentuan Representasi

Berikut prosedur bagaimana dalam merepresentasikan permasalahan ini:

- Paket hanya berbentuk kubus / balok. Paket dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam peti kemas pada truk sehingga suatu paket dapat berada di luar **(seluruh bagian paket di luar)**
- Koordinat berupa bilangan bulat. Cara representasi koordinat dibebaskan.
- Struktur data yang digunakan untuk representasi dibebaskan. Misalnya menggunakan *key-value* seperti JSON.
- Inisialisasi (***initial state***) random.
- Langkah neighbor (***move***) yang diperbolehkan tiap iterasi:
    1. Menukar dua paket, baik di dalam maupun di luar truk
    2. Memindahkan sebuah paket ke suatu koordinat yang berbeda
    3. Merotasi paket 90° terhadap sumbu *x*, *y*, atau *z*
- Representasi solusi (***final state***) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada truk. Secara detail yang harus dapat ditunjukkan oleh state dan visualisasi adalah:
    1. Bagaimana keadaan semua paket yang di dalam maupun yang di luar
    2. Bagaimana keadaan peti kemas pada truk
    3. *Value* pada *state* tersebut

1.3 Objective Function & Constraints

Agar permasalahan ini tidak menjadi kompleks, berikut arahan yang diberikan

- **Objective function**
  Permasalahan ini bertujuan untuk memaksimalkan *value* peti kemas pada truk.
- **Kardus tidak bisa melayang dan menembus dinding peti kemas**
  Melayang disini didefinisikan dengan misalkan paket i setelah orientasi tertentu memiliki posisi sudut kiri-bawah-belakang

  $$(x_i, y_i, z_i)$$

  dan dimensi

  $$(w_i, d_i, h_i)$$

  sehingga alas paket i adalah

  $$B_i = \{(x, y, z_i) \mid x_i \leq x < x_i + w_i,\ y_i \leq y < y_i + d_i\})$$

  Sebuah titik di alas memiliki *support* apabila:

  1. $Z_{i=0}$, artinya di alas truk, atau
  2. Terdapat paket lain j yang mempunyai bagian atas tepat di bawah titik tersebut

  Maka melayang adalah kondisi tidak ada satupun titik alas yang memiliki support.

- **Barang pecah-belah tidak boleh di bawah paket lain**

  Barang pecah-belah tidak boleh menjadi *support*
- **Truk tidak boleh mengangkut melebihi kapasitasnya**

2. Pembahasan

2.1. Representasi Data dan State

2.1.1. Data Input

Data input yang digunakan untuk program ini terdiri dari dua file JSON, yaitu packages.json dan truck.json.

File package.json berisi informasi mengenai paket-paket yang tersedia, meliputi id, dimensi (w, l, h), orientasi (x, y, z), value, berat, status apakah mudah pecah (isFragile), dan eta. Sebagai contoh, salah satu package dapat direpresentasikan sebagai berikut.

```
{
        "id": 0,
        "dimension": {
            "w": 10,
            "l": 3,
            "h": 10
        },
        "orientation": {
            "w": "x",
            "l": "z",
            "h": "y"
        },
        "value": 67,
        "weight": 8,
        "isFragile": false,
        "eta": 4
    }
```

File truck.json berisi informasi mengenai truk-truk yang tersedia (bisa lebih dari satu truk), meliputi id truk, dimensi (w, l, h), dan kapasitas maksimumnya. Sebagai contoh, salah satu truk dapat direpresentasikan sebagai berikut.

```
{
        "id": "truckManhattanCafe",
        "dimension" :
        {
            "w":10,
```

```
            "l":20,
            "h":5
        },
        "max_capacity" : 500
    }
```

2.1.2. Representasi Data

State pada program kami menggambarkan kondisi penempatan seluruh package pada suatu waktu. Pada suatu state terdapat dua komponen, yaitu in_warehouse dan in_trucks. in_warehouse menyimpan id package yang tidak ditempatkan di dalam truk. in_trucks menyimpan daftar truk beserta daftar package yang ada di dalam masing-masing truk. Setiap package yang berada di dalam truk memiliki id, posisi, dan orientasinya di dalam truk. State direpresentasikan menggunakan struktur data nested dictionary sebagai berikut.

```
state = {
    "in_warehouse": [package_id, ...],
    "in_trucks": [
        {
            "truck": "truck_index",
            "packages": [
                {
                    "id": package_id,
                    "position": {"x": int, "y": int, "z": int},
                    "orientation": {"w": "x|y|z", "l": "x|y|z", "h":
"x|y|z"}
                }
            ]
        }
    ]
}
```

2.1.3. Data Output

Hasil eksekusi algoritma disimpan dalam beberapa bentuk output, yaitu info hasil run dalam bentuk file txt, file output JSON, serta visualisasi initial state, final state, dan plot cost dengan iterasi. Struktur file JSON ini memiliki beberapa atribut dasar, antara lain, initState, finalState, finalCost, finalVal, costHistory, seed, dan duratiom. Selain atribut dasar, terdapat juga atribut tambahan spesifik untuk tiap algoritma, misalnya jumlah sideWaysMove pada output Sideways Move Hill Climbing.

2.2. Implementasi dan *Source Code*

Berikut adalah struktur repository kami.

```
IF3170_G35_Eli5
    │   README.md
    │
    ├───data
    │       packages.json
    │       trucks.json
    │
    ├───output
    │   ├───[algoritma]_run[x]
    │   │       final.png
    │   │       [algoritma].json
    │   │       [algoritma]_info.txt
    │   │       initial.png
    │   │       plot.png
    │   │
    └───src
        │   display.py
        │   generate_packages.py
        │   genetic_algorithm.py
        │   helper_function.py
        │   hill_climbing.py
        │   main.py
        │   objective_function.py
        │   read_write_file.py
        │   sim_anneal.py
        │   states.py
```

2.2.1. read_write_file.py

File ini berfungsi untuk memuat dataset dari file JSON ke dalam memori saat program dijalankan. Pada file ini terjadi inisialisasi variabel packages dan trucks dengan json.load(). Berikut ini adalah source codenya.

```python
import json
import os

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA_DIR = os.path.join(BASE_DIR, "data")

with open(os.path.join(DATA_DIR, "packages.json"), 'r') as file:
    packages = json.load(file)
```

```
with open(os.path.join(DATA_DIR, "trucks.json"), 'r') as file:
    trucks = json.load(file)
```

2.2.2. helper_function.py

File ini berisi beberapa helper function, yaitu seperti yang ditampilkan pada tabel berikut.

Fungsi	Deskripsi
getPackageById(id)	Mencari dan mengembalikan objek package berdasarkan id.
getDimensionXYZ(package)	Mengonversi dimensi paket (w, l, h) menjadi vektor ruang (x, y, z) berdasarkan orientasi yang sedang aktif.
getPackagePlacement(state, package_id)	Mencari dan mengembalikan lokasi suatu paket di dalam state (apakah di warehouse atau di truk) berdasarkan package_id.

Berikut adalah source code untuk modul ini.

```
from read_write_file import *
import copy

def getPackageById(id):
    for package in packages:
        if package["id"] == id:
            return package
    return None

def getDimensionXYZ(package):
    dimension = package["dimension"]
    orientation = package["orientation"]

    dimension_xyz = {
        "x": 0,
        "y": 0,
        "z": 0
    }

    dimension_xyz[orientation["w"]] = dimension["w"]
```

```
    dimension_xyz[orientation["l"]] = dimension["l"]
    dimension_xyz[orientation["h"]] = dimension["h"]

    return dimension_xyz

def getPackagePlacement(state, package_id):
    if package_id in state["in_warehouse"]:
        return {
            "location": "warehouse",
            "package": package_id
        }

    for truck in state["in_trucks"]:
        for package in truck["packages"]:
            if package["id"] == package_id:
                return {
                    "location": "truck",
                    "truck": truck["truck"],
                    "package": package
                }

    return None
```

2.2.3. generate_packages.py

File ini berfungsi untuk generate packages dalam format JSON secara acak, sehingga packages tidak perlu dibuat satu-satu secara manual. Terdapat satu function pada file ini, yaitu createPackages(k) yang akan menciptakan random packages sesuai dengan format representasi data kami sebanyak k. Packages tersebut nantinya akan disimpan di data/packages.json. Berikut adalah source codenya.

```
import json
import random
import os

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA_DIR = os.path.join(BASE_DIR, "data")

def createPackages(k):
    packages = []
```

```
    for i in range(k):
        axes = random.sample(["x", "y", "z"], 3)
        packages.append({
            "id": i,
            "dimension":
            {
                "w": random.randint(1, 10),
                "l": random.randint(1, 10),
                "h": random.randint(1, 10)
            },
            "orientation": {
                "w": axes[0],
                "l": axes[1],
                "h": axes[2]
            },
            "value": random.randint(20, 100),
            "weight": random.randint(5, 20),
            "isFragile": bool(random.getrandbits(1)),
            "eta": random.randint(1, 5)
        })

    with open(os.path.join(DATA_DIR, "packages.json"), 'w') as f:
        json.dump(packages, f, indent=4)

    print("Created Packages!")

k = int(input("Masukkan jumlah: "))
createPackages(k)
```

2.2.3. objective_function.py

File ini berisi objective function yanng digunakan untuk menilai seberapa baik suatu state. Fungsi utama pada file ini adalah calculateCost yang menghitung cost dari suatu state. Semakin kecil nilai cost yang dihasilkan, semakin baik state tersebut. Sebelum menghitung cost akhir, program mengecek beberapa jenis pelanggaran yang ada pada state. Pengecekan tersebut dilakukan menggunakan fungsi getClippingPairs, getFlyingPackages, getFragileViolations, dan getOutOfBoundsPackages. Untuk setiap pelanggaran clipping, flying, fragile, maupun out of bounds, diberikan penalti sebesar 10000 per pelanggaran. Selain itu, juga terdapat pemberian penalti untuk kapasitas truk yang melebihi kapasitasnya (kelebihan berat * 100) dan penalti paket yang masih berada di gudang (beratnya * 20). Perhitungan cost

akhir dilakukan dengan mengurangi total value paket yang berhasil dimasukkan ke truk, kemudian menambahkan seluruh poin penalti.

Tabel di bawah ini akan menjelaskan fungsi-fungsi yang ada di file ini.

Fungsi	**Deskripsi**
calculateTruckWeight(state, truck_index)	Menghitung total berat paket yang ada pada truck_index di state tertentu
calculateTruckValue(state, truck_index)	Menghitung total value paket yang ada pada truck_index di state tertentu
getFlyingPackages(state, truck_index)	Mengembalikan paket-paket yang terbang pada truck_index di state tertentu
isPackageFlying(state, truck_index)	Mengembalikan boolean apakah ada paket yang terbang pada truck_index di state tertentu
getClippingPairs(state, truck_index)	Mengembalikan pasangan paket yang clipping pada truck_index di state tertentu
isPackageClipping(state, truck_index)	Mengembalikan boolean apakah ada pasangan paket yang clipping pada truck_index di state tertentu
getFragileViolations(state, truck_index)	Mengembalikan paket-paket fragile yang di bawah tumpukan paket lain pada truck_index di state tertentu
isFragileBelow(state, truck_index)	Mengembalikan boolean apakah ada paket yang fragile di bawah tumpukan pada truck_index di state tertentu
getOutOfBoundsPackages(state, truck_index)	Mengembalikan paket-paket yang out of bounds pada truck_index di state tertentu
isPackageOutOfBounds(state, truck_index)	Mengembalikan boolean apakah ada paket yang out of bounds pada truck_index di state tertentu
calculateCapacityPenalty(state)	Menghitung penalty jika total berat paket lebih besar daripada max capacity truck pada sebuah state
calculateWarehousePenalty(state)	Menghitung penalty paket yang ditinggal di warehouse pada sebuah state
calculateTotVal(state)	Menghitung total value pada sebuah state

calculateCost(state)	Menghitung total cost pada sebuah state

Berikut adalah source code untuk modul ini.

```
from read_write_file import *
from helper_function import *

def calculateTruckWeight(state, truck_index):
    total_weight = 0

    for state_package in state["in_trucks"][truck_index]["packages"]:
        package = getPackageById(state_package["id"])
        total_weight += package["weight"]

    return total_weight

def calculateTruckValue(state, truck_index):
    total_weight = 0

    for state_package in state["in_trucks"][truck_index]["packages"]:
        package = getPackageById(state_package["id"])
        total_weight += package["value"]

    return total_weight

def getFlyingPackages(state, truck_index):
    truck_packages = state["in_trucks"][truck_index]["packages"]
    flying = []

    for sp in truck_packages:
        pkg = getPackageById(sp["id"])
        pos = sp["position"]
        dim = getDimensionXYZ({
            "dimension": pkg["dimension"],
            "orientation": sp["orientation"]
        })

        if pos["z"] == 0:
            continue
```

```
        is_supported = False

        for osp in truck_packages:
            if osp["id"] == sp["id"]:
                continue

            opkg = getPackageById(osp["id"])
            opos = osp["position"]
            odim = getDimensionXYZ({
                "dimension": opkg["dimension"],
                "orientation": osp["orientation"]
            })

            other_top = opos["z"] + odim["z"]
            if other_top != pos["z"]:
                continue

            x_overlap = (pos["x"] < opos["x"] + odim["x"] and pos["x"] +
dim["x"] > opos["x"])
            y_overlap = (pos["y"] < opos["y"] + odim["y"] and pos["y"] +
dim["y"] > opos["y"])

            if x_overlap and y_overlap:
                is_supported = True
                break

        if not is_supported:
            flying.append(sp["id"])

    return flying

def isPackageFlying(state, truck_index):
    return len(getFlyingPackages(state, truck_index)) > 0

def getClippingPairs(state, truck_index):
    truck_packages = state["in_trucks"][truck_index]["packages"]
    pairs = []
```

```
    for i in range(len(truck_packages)):
        p1s = truck_packages[i]
        p1 = getPackageById(p1s["id"])
        pos1 = p1s["position"]
        dim1 = getDimensionXYZ({
            "dimension": p1["dimension"],
            "orientation": p1s["orientation"]
        })

        for j in range(i + 1, len(truck_packages)):
            p2s = truck_packages[j]
            p2 = getPackageById(p2s["id"])
            pos2 = p2s["position"]
            dim2 = getDimensionXYZ({
                "dimension": p2["dimension"],
                "orientation": p2s["orientation"]
            })

            x_overlap = (pos1["x"] < pos2["x"] + dim2["x"] and pos1["x"]
+ dim1["x"] > pos2["x"])
            y_overlap = (pos1["y"] < pos2["y"] + dim2["y"] and pos1["y"]
+ dim1["y"] > pos2["y"])
            z_overlap = (pos1["z"] < pos2["z"] + dim2["z"] and pos1["z"]
+ dim1["z"] > pos2["z"])

            if x_overlap and y_overlap and z_overlap:
                pairs.append((p1s["id"], p2s["id"]))

    return pairs

def isPackageClipping(state, truck_index):
    return len(getClippingPairs(state, truck_index)) > 0

def getFragileViolations(state, truck_index):
    truck_packages = state["in_trucks"][truck_index]["packages"]
    violations = []

    for sp in truck_packages:
        pkg = getPackageById(sp["id"])
```

```
        pos = sp["position"]
        dim = getDimensionXYZ({
            "dimension": pkg["dimension"],
            "orientation": sp["orientation"]
        })

        for osp in truck_packages:
            if osp["id"] == sp["id"]:
                continue

            opkg = getPackageById(osp["id"])
            if not opkg["isFragile"]:
                continue

            opos = osp["position"]
            odim = getDimensionXYZ({
                "dimension": opkg["dimension"],
                "orientation": osp["orientation"]
            })

            other_top = opos["z"] + odim["z"]
            if other_top != pos["z"]:
                continue

            x_overlap = (pos["x"] < opos["x"] + odim["x"] and pos["x"] +
dim["x"] > opos["x"])
            y_overlap = (pos["y"] < opos["y"] + odim["y"] and pos["y"] +
dim["y"] > opos["y"])

            if x_overlap and y_overlap:
                violations.append((sp["id"], osp["id"]))

    return violations

def isFragileBelow(state, truck_index):
    return len(getFragileViolations(state, truck_index)) > 0

def getOutOfBoundsPackages(state, truck_index):
    truck_data = trucks[truck_index]
```

```
    truck_packages = state["in_trucks"][truck_index]["packages"]
    out = []

    for sp in truck_packages:
        pkg = getPackageById(sp["id"])
        dim = getDimensionXYZ({
            "dimension": pkg["dimension"],
            "orientation": sp["orientation"]
        })
        pos = sp["position"]

        if (pos["x"] < 0 or pos["y"] < 0 or pos["z"] < 0 or
            pos["x"] + dim["x"] > truck_data["dimension"]["w"] or
            pos["y"] + dim["y"] > truck_data["dimension"]["l"] or
            pos["z"] + dim["z"] > truck_data["dimension"]["h"]):
            out.append(sp["id"])

    return out

def isPackageOutOfBounds(state, truck_index):
    return len(getOutOfBoundsPackages(state, truck_index)) > 0

def calculateCapacityPenalty(state):
    penalty = 0

    for truck_index, truck in enumerate(state["in_trucks"]):
        truck_id = truck["truck"]

        weight = calculateTruckWeight(state, truck_index)

        truck_data = next(
            t for t in trucks
            if t["id"] == truck_id
        )

        if weight > truck_data["max_capacity"]:
            excess = weight - truck_data["max_capacity"]
            penalty += excess * 100
```

```
    return penalty

def calculateWarehousePenalty(state):
    weight = 0

    for package_id in state["in_warehouse"]:
        package = next(
            p for p in packages
            if p["id"] == package_id
        )

        weight += package["weight"]

    return weight * 20

def calculateTotVal(state):
    total_value = 0

    for truck_index in range(len(state["in_trucks"])):
        total_value += calculateTruckValue(state, truck_index)

    return total_value

def calculateCost(state):
    cost = 0

    cost -= calculateTotVal(state)
    cost += calculateCapacityPenalty(state)
    cost += calculateWarehousePenalty(state)

    for truck_index in range(len(state["in_trucks"])):
        cost += ((len(getClippingPairs(state, truck_index)) +
len(getFlyingPackages(state, truck_index)) +
                 len(getFragileViolations(state, truck_index)) +
len(getOutOfBoundsPackages(state, truck_index)))
                 * 10000)

    return cost
```

2.2.5. states.py

File ini menangani representasi state dan proses pembentukan neighbor state yang akan digunakan pada local search nantinya. State direpresentasikan menggunakan struktur nested dictionary, detailnya sudah dijelaskan pada poin 2.1.2.

State awal (inisialisasi) dibuat dengan kondisi awal seluruh truk masih kosong. Paket kemudian diacak menggunakan random.shuffle() agar urutan penempatannya berbeda-beda. Untuk setiap paket, program mencari truk yang masih memiliki kapasitas yang cukup, kemudian mencoba menempatkan paket menggunakan tryPlacePackage. Fungsi tersebut mencoba hingga 50 kali kombinasi orientasi dan posisi secara acak. Posisi yang dipilih tetap dibatasi agar seluruh dimensi paket berada di dalam batas truk. Jika setelah percobaan tersebut paket tidak berhasil ditempatkan, ID paket dimasukkan ke in_warehouse.

Transisi suatu state ke state berikutnya dilakukan melalui enam operator, yaitu movePackageBetweenTrucks, swapPackageBetweenTrucks, movePackageToWarehouse, movePackageFromWarehouse, changePosition, dan changeOrientation.

Untuk menghasilkan neighbor, program terlebih dahulu membuat salinan state menggunakan deepCopyState, sehingga perubahan yang dilakukan pada kandidat tidak langsung mengubah state asli. Selanjutnya, salah satu dari enam operator dipilih secara acak menggunakan random.choices(). Probabilitas setiap operator dibedakan berdasarkan algoritma yang digunakan. Jika operasi yang dipilih gagal dilakukan, fungsi mengembalikan state awal. Jika berhasil, state hasil operasi digunakan sebagai neighbor.

Tabel di bawah ini akan menjelaskan fungsi-fungsi yang ada di file ini.

Fungsi	**Deskripsi**
generateRandomState()	Meng-*generate* random state dengan menempatkan paket secara acak ke truk yang kapasitasnya cukup.
deepCopyState(state)	Membuat salinan state secara mendalam agar modifikasi pada neighbor tidak merusak state asal.
tryPlacePackage(state, truck_index, package)	Mencoba menempatkan satu paket ke truk tertentu dengan orientasi dan posisi acak.
removePackageFromTruck(state, truck_index, package_id)	Menghapus paket dari list paket truk berdasarkan truck_index dan package_id
chooseRandomSourceTruck(state)	Memilih satu truk acak yang setidaknya memiliki satu paket.
getTrucksWithCapacity(state, package, exclude_index=None)	Mengembalikan list truk yang kapasitasnya masih muat untuk package tersebut.

movePackageBetweenTrucks(state)	Memindahkan satu paket dari suatu truk ke truk yang lain.
swapPackageBetweenTrucks(state)	Menukar satu paket dari suatu truk dengan satu paket dari truk lainnya.
movePackageToWarehouse(state)	Mengeluarkan satu paket dari truk dan memindahkannya ke warehouse.
movePackageFromWarehouse(state)	Memindahkan satu paket dari warehouse ke truk.
changePosition(state)	Mengubah posisi koordinat x,y,z paket di truk yang sama tanpa mengubah orientasinya.
changeOrientation(state)	Memutar orientasi paket di truk yang sama tanpa mengubah posisinya.
generateNeighborState(state, algorithm)	Memilih satu operator secara acak berdasarkan probabilitas masing-masing algoritma untuk menghasilkan neighbor state.

Berikut adalah source code untuk modul ini.

```
from read_write_file import *
from objective_function import *
import random
from itertools import permutations

# bentuk state:
# state = {
#   "in_warehouse": [packages]
#   "in_trucks": [
#       "truck": id_truck
#       "packages": [
#           "id": id_package
#           "position": {x: ..., y: ..., z: ...}
#           "orientation": {w: ..., l: ..., h: ...}
#        ]
#    ]
#}

# generate random state
def generateRandomState():
```

```
    state = {
        "in_warehouse" : [],
        "in_trucks" : []
    }

    # create empty trucks
    for truck in trucks:
        state["in_trucks"].append({
            "truck" : truck["id"],
            "packages" : []
        })

    # load packages to truck
    shuffled_packages = packages.copy()
    random.shuffle(shuffled_packages)

    for package in shuffled_packages:
        possible_trucks = getTrucksWithCapacity(state, package)

        random.shuffle(possible_trucks)
        placed = False

        for truck_index in possible_trucks:
            placed = tryPlacePackage(state, truck_index, package)
            if placed:
                break

        if not placed:
            state["in_warehouse"].append(package["id"])

    return state

def deepCopyState(state):
    new_trucks = []

    for truck in state["in_trucks"]:
        new_packages = []

        for package in truck["packages"]:
```

```
            new_packages.append({
                "id": package["id"],
                "position": dict(package["position"]),
                "orientation": dict(package["orientation"])
            })

        new_trucks.append({
            "truck": truck["truck"],
            "packages": new_packages
        })

    return {
        "in_warehouse": state["in_warehouse"].copy(),
        "in_trucks": new_trucks
    }

# coba taruh 1 paket ke 1 truk tertentu
def tryPlacePackage(state, truck_index, package):
    truck_data = trucks[truck_index]
    for _ in range(50):
        # random orientasi
        w, l, h = random.sample(["x", "y", "z"], 3)
        orientation = {"w": w, "l": l, "h": h}

        orientation_package = {
            "id": package["id"],
            "dimension": package["dimension"],
            "orientation": orientation
        }

        dim = getDimensionXYZ(orientation_package)

        # skip kalo dimensi paket lbh dr truk
        if (dim["x"] > truck_data["dimension"]["w"] or dim["y"] >
truck_data["dimension"]["l"] or dim["z"] >
truck_data["dimension"]["h"]):
            continue
        else:
            # random posisi di dlm truk
```

```
            pos_x = random.randint(0, truck_data["dimension"]["w"] -
dim["x"])
            pos_y = random.randint(0, truck_data["dimension"]["l"] -
dim["y"])
            pos_z = random.randint(0, truck_data["dimension"]["h"] -
dim["z"])

            state["in_trucks"][truck_index]["packages"].append({
                "id": package["id"],
                "position": {"x": pos_x, "y": pos_y, "z": pos_z},
                "orientation": orientation
            })

            return True

    return False

# remove paket dari truk
def removePackageFromTruck(state, truck_index, package_id):
    truck = state["in_trucks"][truck_index]

    for i, package in enumerate(truck["packages"]):
        if package["id"] == package_id:
            del truck["packages"][i]
            return True

    return False

# milih truk random yang masih ada isinya
def chooseRandomSourceTruck(state):
    truck_candidates = []
    for i, truck in enumerate(state["in_trucks"]):
        if len(truck["packages"]) > 0: # cari truk yg ada isinya
            truck_candidates.append(i)

    if not truck_candidates:
        return None

    return random.choice(truck_candidates)
```

```python
# nyari truk yang masih muat utk paket tertentu
def getTrucksWithCapacity(state, package, exclude_index=None):
    possible_trucks = []

    for i, truck in enumerate(state["in_trucks"]):
        if i == exclude_index:
            continue

        if calculateTruckWeight(state, i) + package["weight"] <=
trucks[i]["max_capacity"]:
            possible_trucks.append(i)

    return possible_trucks

# pilih 1 paket dr truk a trs pindahin ke truk b
def movePackageBetweenTrucks(state):
    truck_a = chooseRandomSourceTruck(state)
    if truck_a is None:
        return False

    package = random.choice(state["in_trucks"][truck_a]["packages"])

    truck_b_candidates = getTrucksWithCapacity(state,
getPackageById(package["id"]), exclude_index=truck_a)
    if not truck_b_candidates:
        return False

    truck_b = random.choice(truck_b_candidates)

    # pindah paket kalo bisa
    removePackageFromTruck(state, truck_a, package["id"])

    if tryPlacePackage(state, truck_b, getPackageById(package["id"])):
        return True

    else: # rollback kalo gagal pindah
        state["in_trucks"][truck_a]["packages"].append(package)
        return False
```

```
# tukar masing" 1 paket dr truk a dan truk b
def swapPackageBetweenTrucks(state):
    truck_a = chooseRandomSourceTruck(state)
    if truck_a is None:
        return False

    package_a = random.choice(state["in_trucks"][truck_a]["packages"])
    w_a = calculateTruckWeight(state, truck_a)
    w_pkg_a = getPackageById(package_a["id"])["weight"]

    pkg_b_candidates = []

    for truck_b_index, truck_b in enumerate(state["in_trucks"]):
        if truck_b_index == truck_a or len(truck_b["packages"]) == 0:
            continue

        w_b = calculateTruckWeight(state, truck_b_index)
        valid_pkg_b = []

        for package_b in truck_b["packages"]:
            w_pkg_b = getPackageById(package_b["id"])["weight"]
            w_a_new = w_a - w_pkg_a + w_pkg_b
            w_b_new = w_b - w_pkg_b + w_pkg_a

            # cek apakah masih muat
            if (w_a_new <= trucks[truck_a]["max_capacity"] and w_b_new
<= trucks[truck_b_index]["max_capacity"]):
                valid_pkg_b.append(package_b)

        if valid_pkg_b:
            package_b = random.choice(valid_pkg_b)
            pkg_b_candidates.append((truck_b_index, package_b))

    if not pkg_b_candidates:
        return False

    truck_b_index, package_b = random.choice(pkg_b_candidates)
```

```
    # coba swap paket
    removePackageFromTruck(state, truck_a, package_a["id"])
    removePackageFromTruck(state, truck_b_index, package_b["id"])
    placed_b_in_a = tryPlacePackage(state, truck_a,
getPackageById(package_b["id"]))
    placed_a_in_b = tryPlacePackage(state, truck_b_index,
getPackageById(package_a["id"]))

    if placed_b_in_a and placed_a_in_b:
        return True

    # rollback kalo gagal swap
    if placed_b_in_a:
        removePackageFromTruck(state, truck_a, package_b["id"])
    if placed_a_in_b:
        removePackageFromTruck(state, truck_b_index, package_a["id"])

    state["in_trucks"][truck_a]["packages"].append(package_a)
    state["in_trucks"][truck_b_index]["packages"].append(package_b)
    return False

# pilih 1 paket dr truk trs keluarin
def movePackageToWarehouse(state):
    truck_index = chooseRandomSourceTruck(state)
    if truck_index is None:
        return False

    package = random.choice(state["in_trucks"][truck_index]["packages"])
    removePackageFromTruck(state, truck_index, package["id"])
    state["in_warehouse"].append(package["id"])
    return True

# pilih 1 paket dr warehouse trs masukin ke truk
def movePackageFromWarehouse(state):
    if not state["in_warehouse"]:
        return False

    package_id = random.choice(state["in_warehouse"])
    package = getPackageById(package_id)
```

```
    possible_trucks = getTrucksWithCapacity(state, package)
    if not possible_trucks:
        return False

    truck_index = random.choice(possible_trucks)

    if tryPlacePackage(state, truck_index, package):
        state["in_warehouse"].remove(package_id)
        return True
    else:
        return False

# ubah posisi 1 paket di truk, oriantasinya tetap
def changePosition(state):
    truck_index = chooseRandomSourceTruck(state)
    if truck_index is None:
        return False

    package = random.choice(state["in_trucks"][truck_index]["packages"])
    package_data = getPackageById(package["id"])
    old_pos = package["position"].copy()

    truck_data = trucks[truck_index]
    dim = getDimensionXYZ({
        "dimension": package_data["dimension"],
        "orientation": package["orientation"]
    })

    # cek apakah paket masih muat
    if (dim["x"] > truck_data["dimension"]["w"] or dim["y"] >
truck_data["dimension"]["l"] or dim["z"] >
truck_data["dimension"]["h"]):
        return False

    for _ in range(50):
        pos_x = random.randint(0, truck_data["dimension"]["w"] -
dim["x"])
        pos_y = random.randint(0, truck_data["dimension"]["l"] -
```

```
dim["y"])
        pos_z = random.randint(0, truck_data["dimension"]["h"] -
dim["z"])

        new_pos = {"x": pos_x, "y": pos_y, "z": pos_z}

        if new_pos == old_pos:
            continue

        package["position"] = new_pos
        return True

    # kalo gagal kembaliin ke posisi sebelumnya
    package["position"] = old_pos
    return False

# # ubah orientasi 1 paket di truk, posisinya tetap
def changeOrientation(state):
    truck_index = chooseRandomSourceTruck(state)
    if truck_index is None:
        return False

    package = random.choice(state["in_trucks"][truck_index]["packages"])
    package_data = getPackageById(package["id"])
    old_orientation = package["orientation"].copy()
    truck_data = trucks[truck_index]

    # enumerate semua kemungkinan orientasi
    orientations = [
        {"w": a, "l": b, "h": c}
        for a, b, c in permutations(["x", "y", "z"])
    ]
    random.shuffle(orientations)

    for new_orientation in orientations:
        # skip orientasi yang sama dgn sekarang
        if new_orientation == old_orientation:
            continue
```

```
        dim = getDimensionXYZ({
            "dimension": package_data["dimension"],
            "orientation": new_orientation
        })

        # cek apakah paket masih muat
        if (dim["x"] > truck_data["dimension"]["w"] or dim["y"] >
truck_data["dimension"]["l"] or dim["z"] >
truck_data["dimension"]["h"]):
            continue

        package["orientation"] = new_orientation
        return True

    # kalo gagal balikin ke orientasi sebelumnya
    package["orientation"] = old_orientation
    return False

# generate neighbor state, ini nanti ditune aja
def generateNeighborState(state, algorithm):
    neighbor_state = deepCopyState(state)

    if algorithm == "HC":
        operations = [movePackageBetweenTrucks,
swapPackageBetweenTrucks, movePackageToWarehouse,
movePackageFromWarehouse, changePosition, changeOrientation]
        probabilities = [0.2, 0.2, 0.2, 0.2, 0.1, 0.1]
    elif algorithm == "SA":
        operations = [movePackageBetweenTrucks,
swapPackageBetweenTrucks, movePackageToWarehouse,
movePackageFromWarehouse, changePosition, changeOrientation]
        probabilities = [0.3, 0.3, 0.1, 0.15, 0.075, 0.075]
    elif algorithm == "GA":
        operations = [movePackageBetweenTrucks,
swapPackageBetweenTrucks, movePackageToWarehouse,
movePackageFromWarehouse, changePosition, changeOrientation]
        probabilities = [0.30, 0.25, 0.10, 0.15, 0.10, 0.10]
    else:
        raise ValueError("Unknown algorithm: {}".format(algorithm))
```

```
    operation = random.choices(operations, weights=probabilities)[0]
    success = operation(neighbor_state)

    return neighbor_state if success else state
```

2.2.6. hill_climbing.py

File ini berisi beragam jenis algoritma Hill Climbing yang telah berhasil diimplementasikan.

Fungsi	**Deskripsi**
steepestAscentHillClimbing()	Menjalankan algoritma Hill Climbing versi Steepest Ascent
sideWaysHillClimbing(maxSideWaysMoves= 10)	Menjalankan algoritma Hill Climbing versi Sideways Move. Terdapat parameter tambahan, yakni maxSideWaysMoves yang bertindak sebagai batas eksekusi dari proses pencarian apabila jumlah sideway moves yang dilakukan mencapai maksimum yang sudah ditentukan
stochasticHillClimbing()	Menjalankan algoritma Hill Climbing versi Stochastic
randomRestartHillClimbing(maxRestarts=10)	Menjalankan algoritma Hill Climbing versi Random Restart. Terdapat parameter tambahan, yakni maxRestarts yang bertindak sebagai batas eksekusi dari proses pencarian apabila jumlah restarts yang dilakukan mencapai maksimum yang sudah ditentukan.

2.2.6.1. Steepest Ascent Hill Climbing

Pada Steepest Ascent Hill Climbing, pada setiap iterasi dipilih neighbor dengan cost paling rendah dari sejumlah kandidat neighbor yang dibandingkan. Program membangkitkan 50 kandidat neighbor secara acak menggunakan generateNeighborState(). Kami memilih metode sampling (bukan membangkitkan semua kandidat) agar beban komputasi tidak meledak, karena pada kasus ini kemungkinan perubahan state dapat menjadi sangat besar karena terdapat beberapa operator dan banyak packages. Dengan mengambil 50 kandidat secara acak, algoritma masih dapat mengeksplorasi beberapa alternatif tanpa harus mengevaluasi seluruh ruang pencarian.

Setiap kandidat kemudian dihitung costnya dan dibandingkan untuk mencari kandidat terbaik (yang costnya paling rendah). Dengan mengambil 50 kandidat secara acak, algoritma masih dapat mengeksplorasi beberapa alternatif tanpa harus mengevaluasi seluruh ruang pencarian.

Setelah 50 kandidat diperiksa, akan dipilih kandidat dengan cost terendah. Jika cost tersebut lebih kecil daripada cost state saat ini, algoritma berpindah ke state tersebut. Sebaliknya, jika tidak ditemukan kandidat yang lebih baik, proses dihentikan. Implementasi kami memiliki batas maksimal 100 iterasi untuk mencegah proses berjalan terlalu lama.

Berikut adalah source codenya.

```
def steepestAscentHillClimbing():
    curState = generateRandomState()
    initState = deepCopyState(curState)
    curCost = calculateCost(curState)
    costHistory = [curCost]
    iters = 0

    for i in range(100):
        iters += 1
        bestNeighbor = None
        bestCost = float('inf')

        for j in range(50):
            neighbor = generateNeighborState(curState, "HC")
            if neighbor is curState:
                continue

            cost = calculateCost(neighbor)
            if cost < bestCost:
                bestNeighbor = neighbor
                bestCost = cost

        if bestNeighbor is None:
            break

        if bestCost < curCost:
            curState = bestNeighbor
            curCost = bestCost
            costHistory.append(curCost)
        else:
            break
```

```
    return curState, curCost, costHistory, initState, iters
```

2.2.6.2. Sideways Move Hill Climbing

Implementasi Sideways Move Hill Climbing kami menggunakan mekanisme pembangkitan dan pemilihan neighbor yang sama dengan Steepest Ascent, yaitu membangkitkan hingga 50 kandidat acak pada setiap iterasi dan memilih kandidat dengan cost terendah. Perbedaannya terdapat pada kondisi penerimaan state.

Pada Steepest Ascent, perpindahan hanya dilakukan jika ditemukan neighbor dengan cost yang lebih kecil. Sedangkan, Sideways Move juga mengizinkan perpindahan ke neighbor dengan cost yang sama dengan state saat ini. Perpindahan ini memungkinkan algoritma bergerak melewati area lembah, yaitu kondisi ketika beberapa state memiliki nilai objektif yang sama, sehingga tidak langsung berhenti hanya karena tidak menemukan solusi yang lebih baik.

Jumlah perpindahan dengan cost yang sama dibatasi menggunakan parameter maxSideWaysMoves, yang secara default bernilai 10. Jika ditemukan neighbor yang lebih baik, penghitung sideways move akan direset menjadi 0. Namun, jika algoritma terus berpindah ke state dengan cost yang sama hingga mencapai batas tersebut, pencarian dihentikan. Sama seperti Steepest Ascent, algoritma juga memiliki batas maksimal 100 iterasi.

Berikut adalah source codenya.

```
def sideWaysHillClimbing(maxSideWaysMoves=10):
    curState = generateRandomState()
    initState = deepCopyState(curState)
    curCost = calculateCost(curState)
    costHistory = [curCost]
    iters = 0
    sideWaysMoves = 0

    for i in range(100):
        iters += 1
        bestNeighbor = None
        bestCost = float('inf')

        for j in range(50):
            neighbor = generateNeighborState(curState, "HC")
            if neighbor is curState:
                continue
```

```
            cost = calculateCost(neighbor)
            if cost < bestCost:
                bestNeighbor = neighbor
                bestCost = cost

        if bestNeighbor is None:
            break

        if bestCost < curCost:
            curState = bestNeighbor
            curCost = bestCost
            costHistory.append(curCost)
            sideWaysMoves = 0

        elif bestCost == curCost and sideWaysMoves < maxSideWaysMoves:
            curState = bestNeighbor
            curCost = bestCost
            costHistory.append(curCost)
            sideWaysMoves += 1

        else:
            break

    return curState, curCost, costHistory, initState, iters,
sideWaysMoves
```

2.2.6.3. Stochastic Hill Climbing

Stochastic Hill Climbing hanya membangkitkan satu neighbor secara acak pada setiap iterasi. Neighbor tersebut kemudian langsung dievaluasi. Jika costnya lebih kecil dari state saat ini, algoritma berpindah ke neighbor tersebut. Jika tidak, state saat ini dipertahankan dan algoritma melanjutkan ke iterasi berikutnya. Implementasi pada program kami memiliki batas iterasi sebanyak 5000 untuk memberikan kesempatan eksplorasi lebih banyak. Algoritma akan melanjutkan iterasi hingga mencapai batas tersebut.

Berikut adalah source codenya.

```
def stochasticHillClimbing():
```

```
    curState = generateRandomState()
    initState = deepCopyState(curState)
    curCost = calculateCost(curState)
    costHistory = [curCost]
    iters = 0

    for i in range(5000):
        iters += 1
        neighbor = generateNeighborState(curState, "HC")
        if neighbor is curState:
            costHistory.append(curCost)
            continue

        cost = calculateCost(neighbor)
        if cost < curCost:
            curState = neighbor
            curCost = cost

        costHistory.append(curCost)

    return curState, curCost, costHistory, initState, iters
```

2.2.6.4. Random Restart Hill Climbing

Random Restart Hill Climbing akan menjalankan Steepest Ascent Hill Climbing secara berulang dengan state awal yang berbeda. Pada setiap restart, fungsi steepestAscentHillClimbing() dipanggil kembali sehingga generateRandomState() akan menghasilkan konfigurasi awal baru. Hasil dari setiap restart, termasuk cost akhir dan jumlah iterasinya, kemudian dicatat. Setelah seluruh percobaan selesai, algoritma memilih state dengan cost paling rendah sebagai solusi akhir. Jumlah percobaan dibatasi oleh parameter maxRestarts, yang secara default bernilai 10.

Berikut adalah source codenya.

```
def randomRestartHillClimbing(maxRestarts=10):
    bestState = None
    bestCost = float('inf')
    bestInitState = None
    allCostHistory = []
    itersPerRestart = []
```

```
    restarts = 0

    for i in range(maxRestarts):
        restarts += 1
        curState, curCost, costHistory, initState, iters =
steepestAscentHillClimbing()
        allCostHistory.extend(costHistory)
        itersPerRestart.append(iters)

        if curCost < bestCost:
            bestState = curState
            bestCost = curCost
            bestInitState = initState

    return bestState, bestCost, allCostHistory, bestInitState,
itersPerRestart, restarts
```

2.2.7. sim_anneal.py

File ini merupakan modul untuk algoritma Simulated Annealing. Ia akan mencatat history perhitungan probabilitas dari tiap iterasi dan frekuensi algoritma stuck di optimum lokal, yang diasumsikan sebagai frekuensi ia menolak state yang lebih buruk. Berbeda dengan Hill-Climbing yang hanya menerima state yang lebih baik, SA juga dapat menerima state yang lebih buruk untuk mengurangi kemungkinan terjebak pada local optimum.

Pada awal proses, suhu T diatur sebesar 300 dengan cooldownRate = 0.999. Setiap iterasi menghasilkan satu neighbor menggunakan generateNeighborState() dengan mode. Perubahan cost dihitung dengan: deltaE = cost(nextState) - cost(curState). Jika deltaE <= 0, neighbor langsung diterima. Jika deltaE > 0, neighbor dapat diterima berdasarkan probabilitas: P = e^(-deltaE / T). Suhu kemudian diturunkan menggunakan T *= 0.999 hingga T <= 0.1.

Berikut adalah source codenya.

```
from states import *
from objective_function import *
import math
import random
import time

def simulatedAnnealing():
    T = 300
```

```
    cooldownRate = 0.999
    curState = generateRandomState()
    initState = deepCopyState(curState)
    costHistory = [calculateCost(curState)]
    probabilityHistory = []
    rejectedWorseMoves = 0
    iters = 0

    while (T > 0.1):
        nextState = generateNeighborState(curState, "SA")
        deltaE = calculateCost(nextState) - calculateCost(curState)

        if deltaE < 0:
            curState = nextState
        else:
            probability = math.exp(-deltaE / T)
            probabilityHistory.append((iters, probability))

            if probability > random.random():
                curState = nextState
            else:
                rejectedWorseMoves += 1

        iters += 1
        costHistory.append(calculateCost(curState))
        T *= cooldownRate

    return curState, calculateCost(curState), costHistory, initState,
probabilityHistory, rejectedWorseMoves, iters
```

2.2.8. genetic_algorithm.py

File ini merupakan modul untuk algoritma Genetic Algorithm. Ia akan meminta user untuk menginput ukuran populasi dan jumlah generasi yang diinginkan sebelum menjalankan program. Alur algoritma ini dimulai dengan membuat populasi awal secara acak. Di setiap generasi, ia membentuk populasi baru melalui tiga tahap:

1) Seleksi: Menggunakan Tournament Selection (mengambil 5 individu acak, memilih yang costnya terkecil).

2) Crossover: Membentuk child dengan menentukan penempatan setiap paket dari salah satu induk secara acak dengan peluang 50:50. Penempatan tersebut mencakup apakah paket berada di warehouse atau truck beserta posisi dan orientasinya.

3) Mutasi: Setiap child memiliki peluang 25% untuk dimutasi menggunakan generateNeighborState(). Mutasi digunakan untuk menghasilkan variasi baru pada populasi dan mengurangi kemungkinan populasi menjadi terlalu homogen.

Berikut adalah source codenya.

```
from states import *
from objective_function import *
import random
import json

def tournamentSelection(population):
    candidates = random.sample(population, 5)
    return min(candidates, key=calculateCost)

def crossover(parent1, parent2):
    # bikin truck anak
    child = {
        "in_warehouse": [],
        "in_trucks": []
    }

    for truck in parent1["in_trucks"]:
        child["in_trucks"].append({
            "truck": truck["truck"],
            "packages": []
        })

    package_ids = []

    for package_id in parent1["in_warehouse"]:
        package_ids.append(package_id)

    for truck in parent1["in_trucks"]:
        for package in truck["packages"]:
            package_ids.append(package["id"])
```

```
    # Crossover per package
    for package_id in package_ids:

        if random.random() < 0.5:
            source = parent1
        else:
            source = parent2

        placement = getPackagePlacement(source, package_id)

        if placement["location"] == "warehouse":
            child["in_warehouse"].append(placement["package"])

        else:
            for truck in child["in_trucks"]:
                if truck["truck"] == placement["truck"]:
                    truck["packages"].append(placement["package"])
                    break

    return child

def calculateDiversity(population):
    uniqueStates = {
        json.dumps(state, sort_keys=True)
        for state in population
    }

    return len(uniqueStates)

def geneticAlgorithm(populationSize = 100, generations= 100):
    population = []
    costHistory = []

    for i in range(populationSize):
        population.append(generateRandomState())

    best = min(population, key=lambda x: calculateCost(x))
    bestCost = calculateCost(best)
    initState = deepCopyState(best)
```

```
    costHistory = [calculateCost(best)]
    avgCostHistory = [
        sum(calculateCost(x) for x in population) / len(population)
    ]

    for generation in range(generations):
        newPopulation = []

        while len(newPopulation) < len(population):
            parent1 = tournamentSelection(population)
            parent2 = tournamentSelection(population)

            child = crossover(parent1, parent2)
            if random.random() < 0.25:
                child = generateNeighborState(child, "GA")

            newPopulation.append(child)

        population = newPopulation
        best = min(population, key=lambda x: calculateCost(x))
        bestCost = calculateCost(best)

        costHistory.append(bestCost)
        avgCostHistory.append(
            sum(calculateCost(x) for x in population) / len(population)
        )

    best = min(population, key = lambda x : calculateCost(x))
    bestCost = calculateCost(best)

    return best, bestCost, costHistory, initState, avgCostHistory,
generations
```

2.2.9. display.py

File ini berfungsi untuk menghasilkan segala kebutuhan visualisasi untuk program, dari membuat visualisasi initial state hingga membuat plotting hasil objective function terhadap iterasi algoritma. Berikut adalah source codenya.

```
import os
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
from read_write_file import trucks
from helper_function import *
from objective_function import *

FRAGILE_COLOR = "#e63946"
MAX_LABELS = 40
PALETTE = ["#8ecae6", "#a8dadc", "#b7e4c7", "#d8f3dc", "#ffe8a3",
"#ffd6a5", "#cdb4db", "#bde0fe", "#c7d3e0", "#e0e1a8", "#b8c0ff",
"#9fd8cb"]

#utils

#optional save terus tampilin hasil
def finish(fig, save_path, show):
    if save_path:
        folder = os.path.dirname(save_path)
        if folder:
            os.makedirs(folder, exist_ok=True)
        fig.savefig(save_path, dpi=150, bbox_inches="tight")
    if show:
        plt.show()
    else:
        plt.close(fig)

#muka-muka baloknya dengan sudut kiri-bawah-belakang di (x, y, z)
def boxFaces(x, y, z, dx, dy, dz):
    v = [(x, y, z), (x + dx, y, z), (x + dx, y + dy, z), (x, y + dy, z),
(x, y, z + dz), (x + dx, y, z + dz), (x + dx, y + dy, z + dz), (x, y +
dy, z + dz)]
    return [
        [v[0], v[1], v[2], v[3]],
        [v[4], v[5], v[6], v[7]],
        [v[0], v[1], v[5], v[4]],
        [v[2], v[3], v[7], v[6]],
        [v[1], v[2], v[6], v[5]],
```

```
        [v[0], v[3], v[7], v[4]],
    ]

#jenis-jenis violations yg ketemu
def violations(state, truck_index):
    found = []
    if isPackageClipping(state, truck_index):
        found.append("clipping")
    if isPackageFlying(state, truck_index):
        found.append("melayang")
    if isFragileBelow(state, truck_index):
        found.append("fragile di bawah")
    if isPackageOutOfBounds(state, truck_index):
        found.append("keluar dari truk")
    return found

def packageViolations(state, truck_index):
    info = {}

    for a, b in getClippingPairs(state, truck_index):
        info.setdefault(a, []).append(f"clipping dgn {b}")
        info.setdefault(b, []).append(f"clipping dgn {a}")

    for pid in getFlyingPackages(state, truck_index):
        info.setdefault(pid, []).append("melayang")

    for pid, fid in getFragileViolations(state, truck_index):
        info.setdefault(pid, []).append(f"di atas fragile {fid}")

    for pid in getOutOfBoundsPackages(state, truck_index):
        info.setdefault(pid, []).append("keluar dari truk")

    return info

#mapping warna per id paket biar init dan finalnya mudah dibandingin
def colorMap(state):
    ids = sorted([p["id"] for t in state["in_trucks"] for p in
t["packages"]] + list(state["in_warehouse"]))
    return {pid: PALETTE[i % len(PALETTE)] for i, pid in enumerate(ids)}
```

```
#garis putus2 ke sumbu xyz
def drawProjectionLines(ax, x, y, z, color="#888888"):
    ax.plot([0, x], [y, y], [z, z], color=color, linewidth=0.4,
linestyle="--", alpha=0.5)
    ax.plot([x, x], [0, y], [z, z], color=color, linewidth=0.4,
linestyle="--", alpha=0.5)
    ax.plot([x, x], [y, y], [0, z], color=color, linewidth=0.4,
linestyle="--", alpha=0.5)

#3d crates
def drawTruckkun(ax, truck_index, state, colors, labels=False):
    truck_data = trucks[truck_index]
    W, L, H = (truck_data["dimension"][k] for k in ("w", "l", "h"))
    placed = state["in_trucks"][truck_index]["packages"]
    #kerangka petinya
    ax.add_collection3d(Poly3DCollection(boxFaces(0, 0, 0, W, L, H),
facecolors=(0, 0, 0, 0), edgecolors="#555555", linewidths=1.2))
    for sp in placed:
        pkg = getPackageById(sp["id"])
        dim = getDimensionXYZ({
            "dimension": pkg["dimension"],
            "orientation": sp["orientation"]
        })
        pos = sp["position"]
        fragile = pkg["isFragile"]
        ax.add_collection3d(
            Poly3DCollection(
                boxFaces(pos["x"], pos["y"], pos["z"], dim["x"],
dim["y"], dim["z"]),
                facecolors=FRAGILE_COLOR if fragile else
colors[sp["id"]],
                edgecolors="#7a0011" if fragile else "#333333",
                linewidths=1.0 if fragile else 0.5, alpha=0.85,
            )
        )
        drawProjectionLines(ax, pos["x"], pos["y"], pos["z"])
        if labels and len(placed) <= MAX_LABELS:
            ax.text(pos["x"] + dim["x"] / 2, pos["y"] + dim["y"] / 2,
```

```
pos["z"] + dim["z"], sp["id"], fontsize=6, ha="center", va="bottom")

    # grid lantai
    for gx in range(0, W + 1, 2):
        ax.plot([gx, gx], [0, L], [0, 0], color="#dddddd",
linewidth=0.3, alpha=0.35)
    for gy in range(0, L + 1, 5):
        ax.plot([0, W], [gy, gy], [0, 0], color="#dddddd",
linewidth=0.3, alpha=0.35)

    # grid dinding belakang
    for gx in range(0, W + 1, 2):
        ax.plot([gx, gx], [L, L], [0, H], color="#dddddd",
linewidth=0.3, alpha=0.35)
    for gz in range(0, H + 1, 1):
        ax.plot([0, W], [L, L], [gz, gz], color="#dddddd",
linewidth=0.3, alpha=0.35)

    ax.set_xlim(0, W)
    ax.set_ylim(0, L)
    ax.set_zlim(0, H)
    ax.set_box_aspect((W, L, H))
    ax.set_xticks(range(0, W + 1, 2))
    ax.set_yticks(range(0, L + 1, 5))
    ax.set_zticks(range(0, H + 1))
    ax.tick_params(labelsize=7)
    ax.set_xlabel("x (w)", fontsize=8)
    ax.set_ylabel("y (l)", fontsize=8)
    ax.set_zlabel("z (h)", fontsize=8)
    ax.view_init(elev=26, azim=-28)
    ax.set_title(f"{state['in_trucks'][truck_index]['truck']}
({len(placed)} paket)", fontsize=10)

def infoState(state):
    lines = []
    tot_val = 0
    for i, t in enumerate(state["in_trucks"]):
        value = calculateTruckValue(state, i)
        weight = calculateTruckWeight(state, i)
```

```
        cap = trucks[i]["max_capacity"]
        bad = violations(state, i)
        tot_val += value
        lines.append(f"[{t['truck']}] value={value}
berat={weight}/{cap}")
        lines.append("   constraint: " + ("OK" if not bad else
"MELANGGAR -> " + ", ".join(bad)))
    lines.insert(0, f"TOTAL VALUE di truk : {tot_val}")
    lines.insert(1, "")

    lines.append("")
    lines.append("DI DALAM PETI KEMAS (id: x,y,z | w x l x h
[orientasi])")
    for ti, t in enumerate(state["in_trucks"]):
        bad = packageViolations(state, ti)
        lines.append(f"  --- {t['truck']} ({len(t['packages'])} paket)
---")
        for sp in t["packages"]:
            pkg = getPackageById(sp["id"])
            d = getDimensionXYZ({
                "dimension": pkg["dimension"],
                "orientation": sp["orientation"]
            })
            p = sp["position"]
            o = sp["orientation"]
            tag = " *fragile" if pkg["isFragile"] else ""
            mark = ""
            if sp["id"] in bad:
                mark = "  <-- " + ", ".join(bad[sp["id"]])
            lines.append(
                f"    {sp['id']}: {p['x']},{p['y']},{p['z']} |
{d['x']}x{d['y']}x{d['z']} "
                f"[{o['w']}{o['l']}{o['h']}]
v={pkg['value']}{tag}{mark}"
            )
    lines.append("")
    lines.append(f"DI LUAR / GUDANG ({len(state['in_warehouse'])}
paket)")
    outside = []
```

```
    for pid in state["in_warehouse"]:
        pkg = getPackageById(pid)
        d = pkg["dimension"]
        tag = " *fragile" if pkg["isFragile"] else ""
        outside.append(f"{pid}: {d['w']}x{d['l']}x{d['h']}
v={pkg['value']} berat={pkg['weight']}{tag}")
    lines += outside if outside else ["(kosong)"]
    return "\n".join(lines)

#tampilin kondisi suatu state -> tiap crate in 3d + panel teks
def showState(state, title="State", save_path=None, show=True,
labels=False):
    n = len(state["in_trucks"])
    colors = colorMap(state)
    info = infoState(state)
    height = max(7, 0.125 * (info.count("\n") + 1) + 1)
    fig = plt.figure(figsize=(6.5 * n + 6, height))
    gs = fig.add_gridspec(1, n + 1, width_ratios=[3] * n + [2.6])
    for i in range(n):
        ax = fig.add_subplot(gs[0, i], projection="3d")
        drawTruckkun(ax, i, state, colors, labels)
    ax_text = fig.add_subplot(gs[0, n])
    ax_text.axis("off")
    ax_text.text(0.1, 1, info, va="top", ha="left", family="monospace",
fontsize=7)
    fig.legend(
        handles=[Patch(facecolor=FRAGILE_COLOR, edgecolor="black",
label="fragile"),
                 Patch(facecolor=PALETTE[0], edgecolor="black",
label="regular")],
        loc="lower left", fontsize=8,
    )
    fig.suptitle(title, fontsize=13, fontweight="bold")
    fig.tight_layout(rect=(0, 0.03, 1, 0.95))
    finish(fig, save_path, show)

def plotHC(results, save_path=None, show=True, title="Algorithm"):
    fig = plt.figure(figsize=(12, 8))
    gs = fig.add_gridspec(2, 1, height_ratios=[3, 1])
```

```
    ax_cost = fig.add_subplot(gs[0, 0])
    ax_tab = fig.add_subplot(gs[1, 0])

    # plot cost vs iterasi
    for i, r in enumerate(results):
        history = r.get("costHistory", [])
        ax_cost.plot(
            range(len(history)),
            history,
            linewidth=1.5,
        )

    ax_cost.set_xlabel("Iterasi")
    ax_cost.set_ylabel("Cost")
    ax_cost.grid(alpha=0.3)

    # tabel ringkasan
    ax_tab.axis("off")
    r = results[0]

    headers = ["Value", "Cost", "Durasi", "Iterasi"]
    values = [
        str(r.get("finalVal", "-")),
        str(r.get("finalCost", "-")),
        f"{r.get('duration', 0):.1f}s",
        str(r.get("iterations", "-"))
    ]

    if "sideWaysMoves" in r:
        headers.append("Sideways Moves")
        values.append(str(r.get("sideWaysMoves", "-")))
        headers.append("Max Sideways Moves")
        values.append(str(r.get("maxSideWaysMoves", "-")))

    if "restarts" in r:
        headers.append("Restarts")
        values.append(str(r.get("restarts", "-")))
```

```
    if "iterationsPerRestart" in r:
        headers.append("Avg Iter/Restart")
        val = r.get("iterationsPerRestart", [])
        if isinstance(val, list) and len(val) > 0:
            avg = sum(val) / len(val)
            values.append(f"{avg:.1f}")
        else:
            values.append("-")

    table = ax_tab.table(
        cellText=[values],
        colLabels=headers,
        loc="center",
        cellLoc="center"
    )
    table.auto_set_font_size(False)
    table.set_fontsize(10)
    table.auto_set_column_width(list(range(len(headers))))
    table.scale(1, 1.8)

    fig.suptitle(title, fontsize=14, fontweight="bold")
    fig.tight_layout(rect=(0, 0, 1, 0.95))
    finish(fig, save_path, show)

#list result dict dari sim anneal -> panel cost vs iterasi, e^(dE/T) vs
iterasi, frekuensi stuck, tabel ringkasan
def plotSA(results, save_path=None, show=True, title="Algorithm"):
    fig, axes = plt.subplots(2, 2, figsize=(14, 8),
gridspec_kw={"height_ratios": [3, 1]})
    ax_cost, ax_prob = axes[0]
    fig.delaxes(axes[1, 0])
    fig.delaxes(axes[1, 1])
    ax_tab = fig.add_subplot(2, 1, 2)

    for i, r in enumerate(results):
        cost_history = r.get("costHistory", [])
        if cost_history:
            ax_cost.plot(range(len(cost_history)), cost_history,
linewidth=1.2, label=f"Run {i + 1}")
```

```
        prob_history = r.get("probabilityHistory", [])
        if prob_history:
            xs = [p[0] for p in prob_history]
            ys = [p[1] for p in prob_history]
            ax_prob.plot(xs, ys, linewidth=1, alpha=0.7, label=f"Run {i
+ 1}")

    ax_cost.set_title("Cost terhadap iterasi")
    ax_cost.set_xlabel("Iterasi")
    ax_cost.set_ylabel("Cost")
    ax_cost.grid(alpha=0.3)

    if results: ax_cost.legend()

    ax_prob.set_title("e^(ΔE/T) terhadap iterasi")
    ax_prob.set_xlabel("Iterasi")
    ax_prob.set_ylabel("e^(ΔE/T)")
    ax_prob.set_ylim(-0.02, 1.02)
    ax_prob.grid(alpha=0.3)

    ax_tab.axis("off")
    rows = []
    for i, r in enumerate(results):
        rows.append([
            r.get("finalVal", "-"),
            r.get("finalCost", "-"),
            f"{r.get('duration', 0):.1f}",
            r.get("iterations", len(r.get("costHistory", [])) - 1),
            r.get("rejectedWorseMoves", "-"),
        ])

    table = ax_tab.table(
        cellText=rows,
        colLabels=["Value", "Cost", "Durasi (s)", "Iterasi", "Stuck
(Frekuensi move lebih buruk ditolak)"],
        loc="center",
        cellLoc="center"
    )
```

```
    table.auto_set_font_size(False)
    table.set_fontsize(9)
    table.auto_set_column_width(list(range(5)))
    table.scale(1, 1.8)
    ax_tab.axis("off")

    fig.suptitle(title, fontsize=13, fontweight="bold")
    fig.subplots_adjust(top=0.9, bottom=0.08, hspace=0.35, wspace=0.25)
    finish(fig, save_path, show)

def plotGA(results, save_path=None, show=True, title="Algorithm"):
    fig, ax = plt.subplots(figsize=(10, 6))

    for i, res in enumerate(results, 1):
        best_history = res["costHistory"]
        avg_history = res["avgCostHistory"]

        iterations = range(len(best_history))

        # Biaya terbaik pada setiap generasi
        ax.plot(
            iterations,
            best_history,
            linestyle="-",
            alpha=0.85,
            label=f"Best Cost"
        )

        # Biaya rata-rata seluruh populasi
        ax.plot(
            range(len(avg_history)),
            avg_history,
            linestyle="--",
            alpha=0.55,
            label=f"Average Cost"
        )

    ax.set_title("Genetic Algorithm: Objective per Generation")
    ax.set_xlabel("Generation")
```

```
    ax.set_ylabel("Cost (lower is better)")
    ax.grid(True, alpha=0.3)
    ax.legend()
    fig.tight_layout()

    if save_path:
        os.makedirs(
            os.path.dirname(save_path) or ".",
            exist_ok=True
        )
        fig.savefig(save_path, dpi=150, bbox_inches="tight")

    if show:
        plt.show()

    plt.close(fig)
```

2.2.10. main.py

File main.py berfungsi sebagai pengatur utama untuk menjalankan algoritma dan mengelola hasil eksperimen. Program menyediakan pilihan Steepest Ascent, Sideways Move, Stochastic, Random Restart, Simulated Annealing, dan Genetic Algorithm. Setiap algoritma dijalankan melalui fungsi eksperimen masing-masing. Program membuat seed secara acak, mencatat durasi eksekusi, menyimpan state awal dan akhir, cost, value, serta data riwayat yang diperlukan untuk analisis. Hasil tersebut disimpan dalam format JSON dan informasi ringkas juga disimpan dalam file .txt. Untuk algoritma yang memiliki parameter tambahan, pengguna dapat melakukan input, misalnya maximum sideways moves, maximum restarts, serta ukuran populasi dan jumlah generasi untuk GA. Setelah algoritma selesai, program menampilkan state awal dan akhir serta membuat grafik hasil eksperimen. Berikut adalah source codenya.

```
import os, glob, json, random, time
from read_write_file import *
from sim_anneal import *
from hill_climbing import *
from genetic_algorithm import *
import display

OUTPUT_DIR = os.path.join(BASE_DIR, "output")
SHOW_WINDOWS = True #kalau false, gambarnya cuman disave jadi png, ga
```

```
ditampilin

ALGOS = { #file name, display name, func, ma func plot hasil
    "1": ("HC-SA", "Hill Climbing Steepest Ascent",
steepestAscentHillClimbing, display.plotHC),
    "2": ("HC-SM", "Hill Climbing with Sideways Move",
sideWaysHillClimbing, display.plotHC),
    "3": ("HC-S", "Stochastic Hill Climbing", stochasticHillClimbing,
display.plotHC),
    "4": ("HC-RR", "Random Restart Hill Climbing",
randomRestartHillClimbing, display.plotHC),
    "5": ("SA", "Simulated Annealing", simulatedAnnealing,
display.plotSA),
    "6": ("GA", "Genetic Algorithm", geneticAlgorithm, display.plotGA)
    #tambahin yang lain yaw
}

#simpan/baca hasil
def get_run_dir(name, run):
    path = os.path.join(OUTPUT_DIR, f"{name}_run{run}")
    os.makedirs(path, exist_ok=True)
    return path

def get_next_run(name):
    if not os.path.exists(OUTPUT_DIR):
        return 1
    runs = [0]
    for f in os.listdir(OUTPUT_DIR):
        if f.startswith(f"{name}_run"):
            try:
                num = int(f.split("_run")[1])
                runs.append(num)
            except:
                pass
    return max(runs) + 1

def saveRes(name, run, res):
    folder = get_run_dir(name, run)
    path = os.path.join(folder, f"{name}.json")
```

```
    with open(path, "w") as f:
        json.dump(res, f, indent=2)

def loadRes(name, run):
    path = os.path.join(OUTPUT_DIR, f"{name}_run{run}", f"{name}.json")
    if os.path.exists(path):
        with open(path, "r") as f:
            return json.load(f)
    return None

#jalankan dan tampilkan
def runHCSteepestExperiment(name, label, algo, run):
    seed = random.randrange(1, 1000000)
    random.seed(seed)

    start_time = time.perf_counter()
    finalState, finalCost, costHistory, initState, iters = algo()
    duration = time.perf_counter() - start_time
    res = {
        "initState": initState,
        "finalState": finalState,
        "finalCost": finalCost,
        "finalVal": calculateTotVal(finalState),
        "costHistory": costHistory,
        "seed": seed,
        "duration": duration,
        "iterations": iters
    }
    saveRes(name, run, res)

    info_init = display.infoState(initState)
    info_final = display.infoState(finalState)
    folder = get_run_dir(name, run)
    info_path = os.path.join(folder, f"{name}_info.txt")
    with open(info_path, "w", encoding="utf-8") as f:
        f.write(f"(seed={seed}) ===\n")
        f.write(f"Jumlah iterasi = {res['iterations']}\n")
        f.write(f"value akhir = {res['finalVal']} | cost =
{res['finalCost']} | durasi = {res['duration']:.1f}s\n\n")
```

```
        f.write("### INITIAL STATE ###\n")
        f.write(info_init + "\n\n")
        f.write("### FINAL STATE ###\n")
        f.write(info_final + "\n")

    print(f"    value di truk = {res['finalVal']}"
          f" | cost = {res['finalCost']}"
          f" | durasi = {res['duration']:.1f}s"
          f" | iterasi = {res['iterations']}")
    print(f"    info disimpan di: {info_path}")

    return res

def runHCSidewaysExperiment(name, label, algo, run, max_sm=10):
    seed = random.randrange(1, 1000000)
    random.seed(seed)

    start_time = time.perf_counter()
    finalState, finalCost, costHistory, initState, iters, sideWaysMoves
= algo(maxSideWaysMoves=max_sm)
    duration = time.perf_counter() - start_time
    res = {
        "initState": initState,
        "finalState": finalState,
        "finalCost": finalCost,
        "finalVal": calculateTotVal(finalState),
        "costHistory": costHistory,
        "seed": seed,
        "duration": duration,
        "iterations": iters,
        "sideWaysMoves": sideWaysMoves,
        "maxSideWaysMoves": max_sm
    }
    saveRes(name, run, res)

    info_init = display.infoState(initState)
    info_final = display.infoState(finalState)
    folder = get_run_dir(name, run)
    info_path = os.path.join(folder, f"{name}_info.txt")
```

```
    with open(info_path, "w", encoding="utf-8") as f:
        f.write(f"(seed={seed}) ===\n")
        f.write(f"Jumlah iterasi = {res['iterations']}\n")
        f.write(f"Jumlah side ways moves = {res['sideWaysMoves']} | max
side ways moves = {res['maxSideWaysMoves']}\n")
        f.write(f"value akhir = {res['finalVal']} | cost =
{res['finalCost']} | durasi = {res['duration']:.1f}s\n\n")
        f.write("### INITIAL STATE ###\n")
        f.write(info_init + "\n\n")
        f.write("### FINAL STATE ###\n")
        f.write(info_final + "\n")

    print(f"    value di truk = {res['finalVal']}"
          f" | cost = {res['finalCost']}"
          f" | durasi = {res['duration']:.1f}s"
          f" | iterasi = {res['iterations']}"
          f" | sideways moves dilakukan = {res['sideWaysMoves']}"
          f" | max side ways moves = {res['maxSideWaysMoves']}")
    print(f"    info disimpan di: {info_path}")

    return res

def runHCStochasticExperiment(name, label, algo, run):
    seed = random.randrange(1, 1000000)
    random.seed(seed)

    start_time = time.perf_counter()
    finalState, finalCost, costHistory, initState, iters = algo()
    duration = time.perf_counter() - start_time
    res = {
        "initState": initState,
        "finalState": finalState,
        "finalCost": finalCost,
        "finalVal": calculateTotVal(finalState),
        "costHistory": costHistory,
        "seed": seed,
        "duration": duration,
        "iterations": iters
    }
```

```
    saveRes(name, run, res)

    info_init = display.infoState(initState)
    info_final = display.infoState(finalState)
    folder = get_run_dir(name, run)
    info_path = os.path.join(folder, f"{name}_info.txt")
    with open(info_path, "w", encoding="utf-8") as f:
        f.write(f"(seed={seed}) ===\n")
        f.write(f"Jumlah iterasi = {res['iterations']}\n")
        f.write(f"value akhir = {res['finalVal']} | cost =
{res['finalCost']} | durasi = {res['duration']:.1f}s\n\n")
        f.write("### INITIAL STATE ###\n")
        f.write(info_init + "\n\n")
        f.write("### FINAL STATE ###\n")
        f.write(info_final + "\n")

    print(f"    value di truk = {res['finalVal']}"
          f" | cost = {res['finalCost']}"
          f" | durasi = {res['duration']:.1f}s"
          f" | iterasi = {res['iterations']}")
    print(f"    info disimpan di: {info_path}")

    return res

def runHCRandomRestartExperiment(name, label, algo, run, max_rr=10):
    seed = random.randrange(1, 1000000)
    random.seed(seed)

    start_time = time.perf_counter()
    finalState, finalCost, costHistory, initState, itersPerRestart,
restarts, totalIters = algo(maxRestarts=max_rr)
    duration = time.perf_counter() - start_time
    res = {
        "initState": initState,
        "finalState": finalState,
        "finalCost": finalCost,
        "finalVal": calculateTotVal(finalState),
        "costHistory": costHistory,
        "seed": seed,
```

```
        "duration": duration,
        "iterations": totalIters,
        "iterationsPerRestart": itersPerRestart,
        "restarts": restarts,
        "maxRestarts": max_rr
    }
    saveRes(name, run, res)

    info_init = display.infoState(initState)
    info_final = display.infoState(finalState)
    folder = get_run_dir(name, run)
    info_path = os.path.join(folder, f"{name}_info.txt")
    with open(info_path, "w", encoding="utf-8") as f:
        f.write(f"(seed={seed}) ===\n")
        f.write(f"Iterasi per restart =
{res['iterationsPerRestart']}\n")
        f.write(f"Jumlah restart = {res['restarts']}\n")
        f.write(f"Total iterasi = {res['iterations']}\n")
        f.write(f"value akhir = {res['finalVal']} | cost =
{res['finalCost']} | durasi = {res['duration']:.1f}s\n\n")
        f.write("### INITIAL STATE ###\n")
        f.write(info_init + "\n\n")
        f.write("### FINAL STATE ###\n")
        f.write(info_final + "\n")

    print(f"    value di truk = {res['finalVal']}"
          f" | cost = {res['finalCost']}"
          f" | durasi = {res['duration']:.1f}s"
          f" | iterasi per restart = {res['iterationsPerRestart']}"
          f" | total iterasi = {res['iterations']}"
          f" | jumlah restart = {res['restarts']}")
    print(f"    info disimpan di: {info_path}")

    return res

def runSAExperiment(name, label, algo, run):
    seed = random.randrange(1, 1000000)
    random.seed(seed)
```

```
    start_time = time.perf_counter()
    finalState, finalCost, costHistory, initState, probabilityHistory,
rejectedWorseMoves, iters = algo()
    duration = time.perf_counter() - start_time
    res = {
        "initState": initState,
        "finalState": finalState,
        "finalCost": finalCost,
        "finalVal": calculateTotVal(finalState),
        "costHistory": costHistory,
        "seed": seed,
        "duration": duration,
        "probabilityHistory": probabilityHistory,
        "rejectedWorseMoves": rejectedWorseMoves,
        "iterations": iters
    }
    saveRes(name, run, res)

    info_init = display.infoState(initState)
    info_final = display.infoState(finalState)
    folder = get_run_dir(name, run)
    info_path = os.path.join(folder, f"{name}_info.txt")
    with open(info_path, "w", encoding="utf-8") as f:
        f.write(f"(seed={seed}) ===\n")
        f.write(f"Frekuensi stuck (jumlah move lebih buruk ditolak) =
{res['rejectedWorseMoves']}\n")
        f.write(f"value akhir = {res['finalVal']} | cost =
{res['finalCost']} | durasi = {res['duration']:.1f}s\n\n")
        f.write("### INITIAL STATE ###\n")
        f.write(info_init + "\n\n")
        f.write("### FINAL STATE ###\n")
        f.write(info_final + "\n")

    print(f"    value di truk = {res['finalVal']}"
          f" | cost = {res['finalCost']}"
          f" | durasi = {res['duration']:.1f}s"
          f" | frekuensi stuck (jumlah move lebih buruk ditolak) =
{res['rejectedWorseMoves']}")
    print(f"    info disimpan di: {info_path}")
```

```
    return res


def runGAExperiment(name, label, algo, run, populationSize,
generations):
    seed = random.randrange(1, 1000000)
    random.seed(seed)

    start_time = time.perf_counter()

    (
        finalState,
        finalCost,
        costHistory,
        initState,
        avgCostHistory,
        iters
    ) = algo(
        populationSize=populationSize,
        generations=generations
    )

    duration = time.perf_counter() - start_time

    res = {
        "initState": initState,
        "finalState": finalState,
        "finalCost": finalCost,
        "finalVal": calculateTotVal(finalState),
        "costHistory": costHistory,
        "avgCostHistory": avgCostHistory,
        "populationSize": populationSize,
        "generations": generations,
        "iterations": iters,
        "seed": seed,
        "duration": duration
    }
```

```
    saveRes(name, run, res)

    info_init = display.infoState(initState)
    info_final = display.infoState(finalState)

    folder = get_run_dir(name, run)
    info_path = os.path.join(folder, f"{name}_info.txt")

    with open(info_path, "w", encoding="utf-8") as f:
        f.write(f"(seed={seed}) ===\n")
        f.write(f"Populasi = {populationSize}\n")
        f.write(f"Jumlah generasi = {iters}\n")
        f.write(f"value akhir = {res['finalVal']} | cost =
{res['finalCost']} | durasi = {res['duration']:.1f}s\n\n")

        f.write("### INITIAL STATE ###\n")
        f.write(info_init + "\n\n")

        f.write("### FINAL STATE ###\n")
        f.write(info_final + "\n")


    print(
        f"    Value di truk = {res['finalVal']} "
        f"| Cost = {finalCost} "
        f"| Durasi = {duration:.1f}s"
    )
    print(f"    Info disimpan di: {info_path}")

    return res

def showResults(name, label, result, run, plotFunc):
    folder = get_run_dir(name, run)

    display.showState(
        result["initState"], f"{label} - Run {run} - Initial",
        save_path=os.path.join(folder, "initial.png"), show=SHOW_WINDOWS
    )
    display.showState(
```

```
        result["finalState"], f"{label} - Run {run} - Final",
        save_path=os.path.join(folder, "final.png"), show=SHOW_WINDOWS
    )

    plotFunc(
        [result],
        save_path=os.path.join(folder, "plot.png"),
        show=SHOW_WINDOWS, title=f"{label} - Run {run}"
    )

def main():
    while True:
        print("\n=== Mejiro Express - Local Search ===")
        for code, (_, label, _, _) in ALGOS.items():
            print(f"{code}. {label}")
        print("0. Keluar")

        choice = input("Pilih algoritma: ").strip()
        if choice == "0":
            break

        if choice not in ALGOS:
            print("Pilihan tidak dikenal.")
            continue
        name, label, algo, plotFunc = ALGOS[choice]


        mode = input("(r)un baru atau (l)oad hasil dari output? [r/l]:
").strip().lower()
        if mode == "l":
            try:
                run_num = int(input("Mau load Run ke berapa? "))
                result = loadRes(name, run_num)
                if not result:
                    print(f"Hasil Run {run_num} untuk {label} tidak
ditemukan.")
                    continue
                showResults(name, label, result, run_num, plotFunc)
            except ValueError:
```

```
                print("Masukkan angka run yang valid.")
            continue

        run_num = get_next_run(name)

        params = {}
        if choice == "2":
            while True:
                try:
                    val = input("Max Sideways Moves [default: 10]:
").strip()
                    params["max_sm"] = int(val) if val else 10
                    if params["max_sm"] <= 0:
                        print("Angka harus lebih dari 0")
                        continue
                    break
                except ValueError:
                    print("Input tidak valid! Masukkan angka.")
        elif choice == "4":
            while True:
                try:
                    val = input("Max Restarts [default: 10]: ").strip()
                    params["max_rr"] = int(val) if val else 10
                    if params["max_rr"] <= 0:
                        print("Angka harus lebih dari 0")
                        continue
                    break
                except ValueError:
                    print("Input tidak valid! Masukkan angka.")
        elif choice == "6":
            while True:
                try:
                    p = input("Ukuran populasi [default: 100]:
").strip()
                    g = input("Jumlah generasi [default: 100]:
").strip()
                    params["populationSize"] = int(p) if p else 100
                    params["generations"] = int(g) if g else 100
                    if params["populationSize"] < 5 or
```

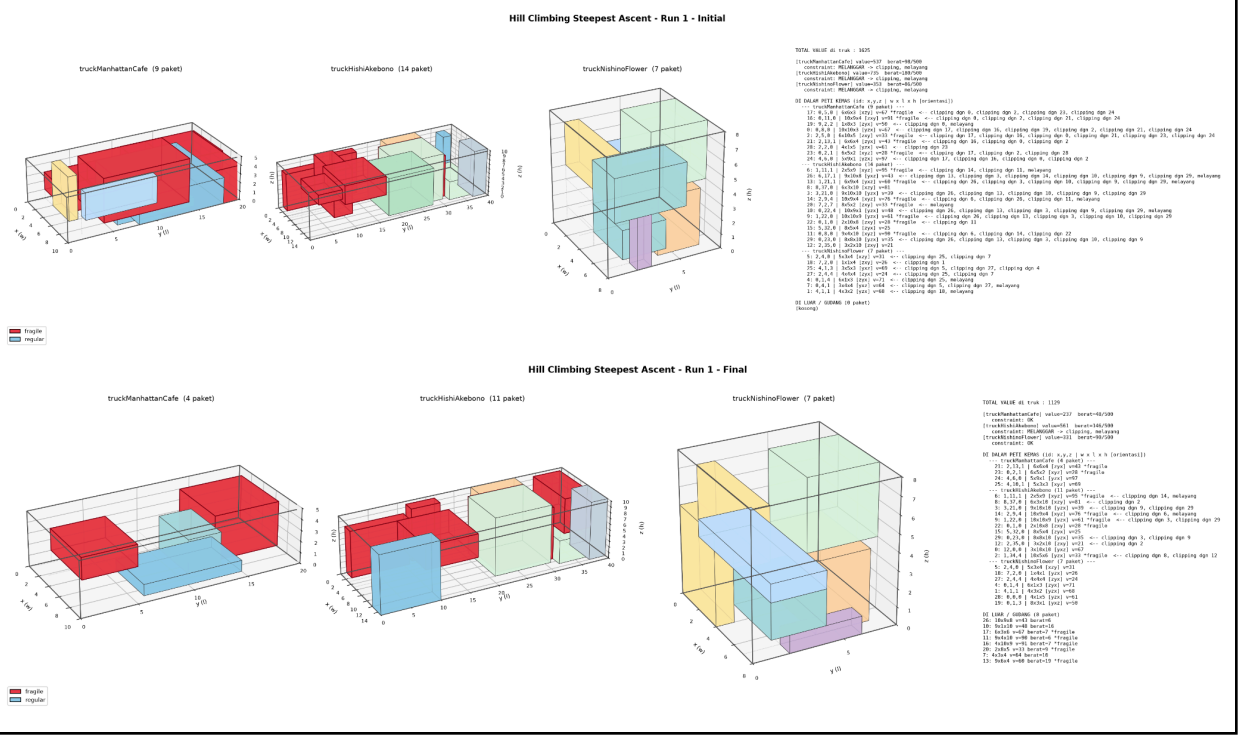
```
params["generations"] < 1:
                        print("Populasi minimal 5 dan generasi minimal
1. Coba lagi.")
                        continue
                    break
                except ValueError:
                    print("Input tidak valid! Masukkan angka.")

        if choice == "1": result = runHCSteepestExperiment(name, label,
algo, run_num)
        elif choice == "2": result = runHCSidewaysExperiment(name,
label, algo, run_num, **params)
        elif choice == "3": result = runHCStochasticExperiment(name,
label, algo, run_num)
        elif choice == "4": result = runHCRandomRestartExperiment(name,
label, algo, run_num, **params)
        elif choice == "5": result = runSAExperiment(name, label, algo,
run_num)
        elif choice == "6": result = runGAExperiment(name, label, algo,
run_num, **params)

        showResults(name, label, result, run_num, plotFunc)

if __name__ == "__main__":
    main()
```

2.3. Hasil dan Analisis Eksperimen

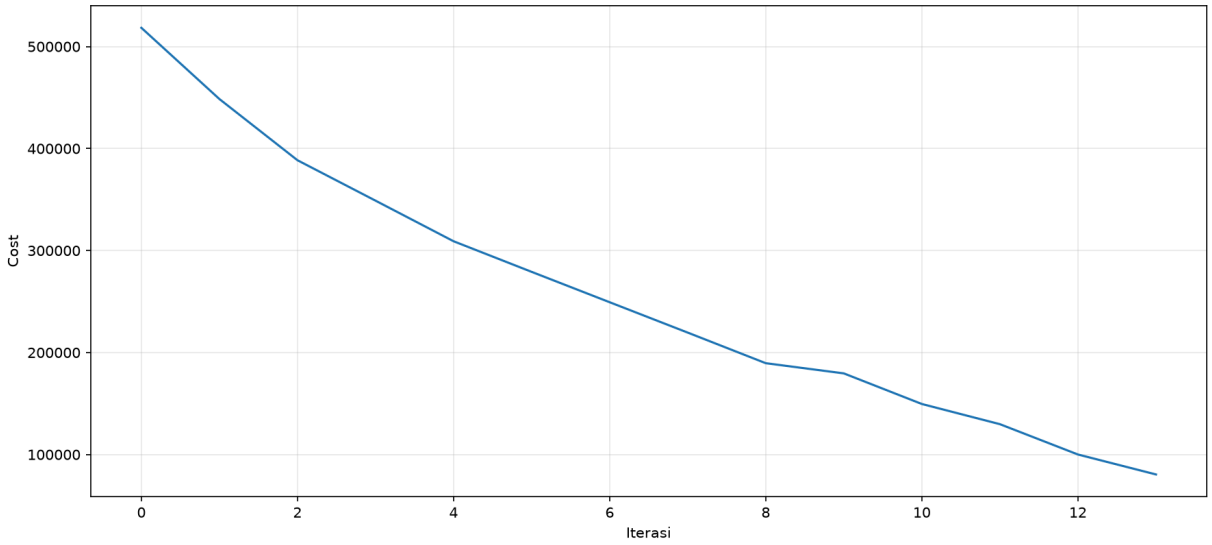
2.3.1. Steepest Ascent Hill Climbing

2.3.1.1 Hasil

Run 1



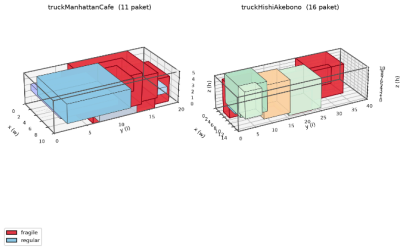
Hill Climbing Steepest Ascent - Run 1



Value	Cost	Durasi	Iterasi
1129	80471	0.7s	14

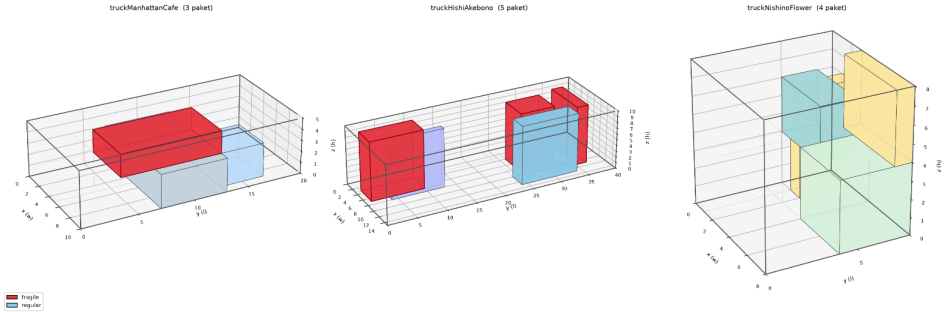
Run 2

Hill Climbing Steepest Ascent - Run 2 - Initial



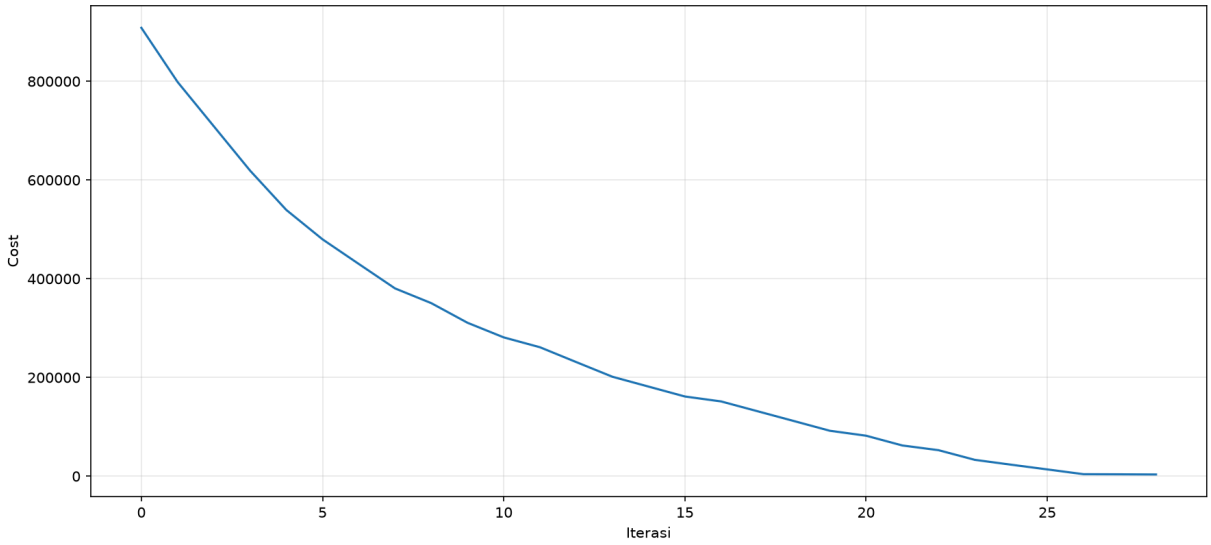

```
TOTAL VALUE di truk : 1925

[truckManhattanCafe] value=647  berat=148/500
  constraint: MELANGGAR -> fragile, clipping, fragile di bawah
[truckHishiAkebono] value=825  berat=175/500
  constraint: MELANGGAR -> clipping, melayang
[truckNishinoFlower] value=453  berat=47/500
  constraint: MELANGGAR -> clipping, melayang

DI DALAM PETI KONUS (id: x,y,z | w x l x h (orientasi))
--- truckManhattanCafe (11 paket) ---
  6: 4,4,3 | 1x3x2 [zxy] v=45 *fragile <- clipping dgn 5, clipping dgn 2, clipping dgn 13, clipping dgn 0, clipping dgn 13
  5: 3,11,2 | 1x4x3 [zxy] v=31  <-- clipping dgn 9, clipping dgn 2, clipping dgn 19, clipping dgn 7, clipping dgn 11, clipping dgn 8, clipping dgn 8, clipping dgn 13, di atas fragile 22
  4: 3,7,0 | 1x4x3 [zxy] v=31  <-- clipping dgn 9, clipping dgn 2, clipping dgn 19, clipping dgn 7, clipping dgn 11, clipping dgn 8, clipping dgn 8
  2: 2,3,0 | 4x1x4x1 [zxy] v=16 *fragile  <-- clipping dgn 6, clipping dgn 5, clipping dgn 19, clipping dgn 19, clipping dgn 7, clipping dgn 11, clipping dgn 8, clipping dgn 22, clipping dgn 19
  19: 7,10,2 | 3x3x1 [xzy] v=50  <-- clipping dgn 5, clipping dgn 2, clipping dgn 7, clipping dgn 11, clipping dgn 8, clipping dgn 8, clipping dgn 13, di atas fragile 22
  7: 5,13,0 | 4x4x3 [xzy] v=44  <-- clipping dgn 5, clipping dgn 2, clipping dgn 19, clipping dgn 11, clipping dgn 13, clipping dgn 13
  1: 0,7,1 | 3x3x4 [xzy] v=40 *fragile  <-- clipping dgn 6, clipping dgn 5, clipping dgn 19, clipping dgn 7, clipping dgn 11, clipping dgn 2, clipping dgn 4, clipping dgn 0, clipping dgn 22, clipping dgn 13, melayang
  8: 0,0,2 | 2x2x3 [xzy] v=24 *fragile  <-- clipping dgn 6, clipping dgn 5, clipping dgn 19, clipping dgn 19, clipping dgn 7, clipping dgn 11, clipping dgn 2, clipping dgn 22, clipping dgn 13, melayang
  0: 0,2,1 | 2x2x3x3 [zxy] v=37  <-- clipping dgn 6, clipping dgn 5, clipping dgn 19, clipping dgn 2, clipping dgn 19, clipping dgn 11, clipping dgn 2, clipping dgn 22, clipping dgn 13, melayang
  22: 3,6,0 | 8x3x2 [xzy] v=38 *fragile  <-- clipping dgn 2, clipping dgn 7, clipping dgn 11, clipping dgn 8, clipping dgn 8, clipping dgn 13
  14: 3,10,0 | 4x4x4 [zxy] v=48 *fragile  <-- clipping dgn 6, clipping dgn 5, clipping dgn 2, clipping dgn 19, clipping dgn 7, clipping dgn 11, clipping dgn 8, clipping dgn 8, clipping dgn 22
--- truckHishiAkebono (16 paket) ---
  17: 5,12,1 | 10x10x9 [xzy] v=39  <-- clipping dgn 17, clipping dgn 12, clipping dgn 21, clipping dgn 20, clipping dgn 20, clipping dgn 14, melayang
  27: 5,12,1 | 8x8x6 [zyx] v=67 *fragile  <-- clipping dgn 3, clipping dgn 25, clipping dgn 20, clipping dgn 29, melayang
  4: 9,29,3 | 5x3x6 [yxz] v=71  <-- clipping dgn 9, clipping dgn 27, melayang
  12: 0,21,0 | 8x8x8 [xyz] v=96  <-- clipping dgn 5, clipping dgn 16
  13: 0,21,1 | 10x3x2 [xyz] v=31  <-- clipping dgn 5, clipping dgn 16, melayang
  21: 11,16,1 | 4x8x6 [xzy] v=45 *fragile  <-- clipping dgn 5, clipping dgn 25, clipping dgn 14, melayang
  15: 5,1,3 | 6x6x8 [xzy] v=75  <-- clipping dgn 26, clipping dgn 26, melayang
  25: 5,18,1 | 3x2x5 [zxy] v=50  <-- clipping dgn 17, clipping dgn 20, clipping dgn 29, melayang
  26: 0,25,1 | 10x10x9 [xzy] v=39  <-- clipping dgn 17, clipping dgn 12, clipping dgn 21, clipping dgn 20, clipping dgn 16, melayang
  5: 0,25,1 | 10x10x9 [zyx] v=61 *fragile  <-- clipping dgn 4, clipping dgn 19, clipping dgn 21, melayang
  20: 0,15,1 | 2x3x6 [xzy] v=33 *fragile  <-- clipping dgn 3, clipping dgn 17, clipping dgn 29, melayang
  23: 8,25,1 | 2x6x5 [zyx] v=38 *fragile  <-- clipping dgn 4, clipping dgn 3, melayang
  29: 4,5,0 | 8x8x10 [xzy] v=75  <-- clipping dgn 5, clipping dgn 17, clipping dgn 21, clipping dgn 25, clipping dgn 29, clipping dgn 20, clipping dgn 14, clipping dgn 24, clipping dgn 16
  24: 5,1,3 | 6x6x8 [xzy] v=75  <-- clipping dgn 26, clipping dgn 26, clipping dgn 29, melayang
  14: 5,1,3 | 5x9x1 [yxz] v=67  <-- clipping dgn 15, clipping dgn 26, clipping dgn 29, clipping dgn 16, melayang
  16: 3,1,0 | 4x9x10 [xzy] v=93 *fragile  <-- clipping dgn 26, clipping dgn 29, clipping dgn 24
--- truckNishinoFlower (3 paket) ---
  28: 2,0,1 | 5x1x6 [yxz] v=61  <-- clipping dgn 1, melayang
  18: 2,0,1 | 5x1x6 [yxz] v=61
  3: 4,0,3 | 4x2x1 [zyx] v=56  <-- clipping dgn 28, melayang

DI LUAR / GUDANG (0 paket)
(kosong)
```

Hill Climbing Steepest Ascent - Run 2 - Final




```
TOTAL VALUE di truk : 731

[truckManhattanCafe] value=240  berat=37/500
  constraint: OK
[truckHishiAkebono] value=267  berat=59/500
  constraint: OK
[truckNishinoFlower] value=224  berat=54/500
  constraint: OK

DI DALAM PETI KONUS (id: x,y,z | w x l x h (orientasi))
--- truckManhattanCafe (3 paket) ---
  6: 4,4,3 | 1x3x2 [zxy] v=45 *fragile
  5: 3,11,2 | 1x4x3 [zxy] v=31
  8: 0,0,2 | 2x2x3 [zxy] v=24
--- truckHishiAkebono (5 paket) ---
  16: 3,1,0 | 4x9x10 [xzy] v=93 *fragile
  22: 3,6,0 | 8x3x2 [xzy] v=38 *fragile
  14: 3,10,0 | 4x4x4 [zxy] v=48
  0: 0,2,1 | 5x3x6 [zxy] v=67
  2: 0,0,0 | 5x8x10 [xyz] v=93 *fragile
--- truckNishinoFlower (4 paket) ---
  28: 3,7,4 | 5x1x4 [yxz] v=61
  27: 4,4,0 | 8x8x6 [yxz] v=24
  1: 2,4,4 | 4x2x1 [zyx] v=84
  4: 1,1,0 | 1x3x6 [yxz] v=71

DI LUAR / GUDANG (10 paket)
11: 9x9x10 v=90 berat=6 *fragile
29: 0x10x0 v=35 berat=12
17: 4x1x6 v=25 berat=10
26: 10x9x8 v=63 berat=6
13: 9x8x8 v=88 berat=19 *fragile
19: 1x8x1 v=56 berat=4
24: 10x9x4 v=78 berat=8 *fragile
15: 4x1x6 v=25 berat=10 *fragile
24: 9x1x1 v=92 berat=7
18: 4x1x1 v=26 berat=18
25: 1x1x3 v=69 berat=8
12: 10x3x2 v=23 berat=5
10: 10x10x9 v=39 berat=17 *fragile
9: 10x9x10 v=61 berat=13 *fragile
3: 10x10x9 v=39 berat=20
17: 4x1x6 v=67 berat=7 *fragile
15: 4x1x6 v=25 berat=10
21: 4x8x6 v=45 berat=15 *fragile
```

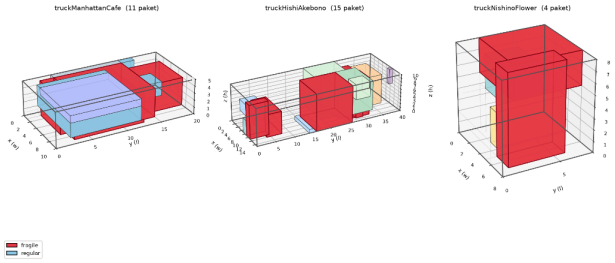
Hill Climbing Steepest Ascent - Run 2



Value	Cost	Durasi	Iterasi
731	3549	0.8s	29

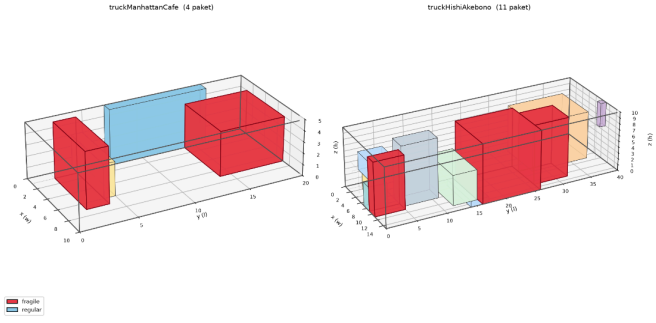
Run 3

Hill Climbing Steepest Ascent - Run 3 - Initial

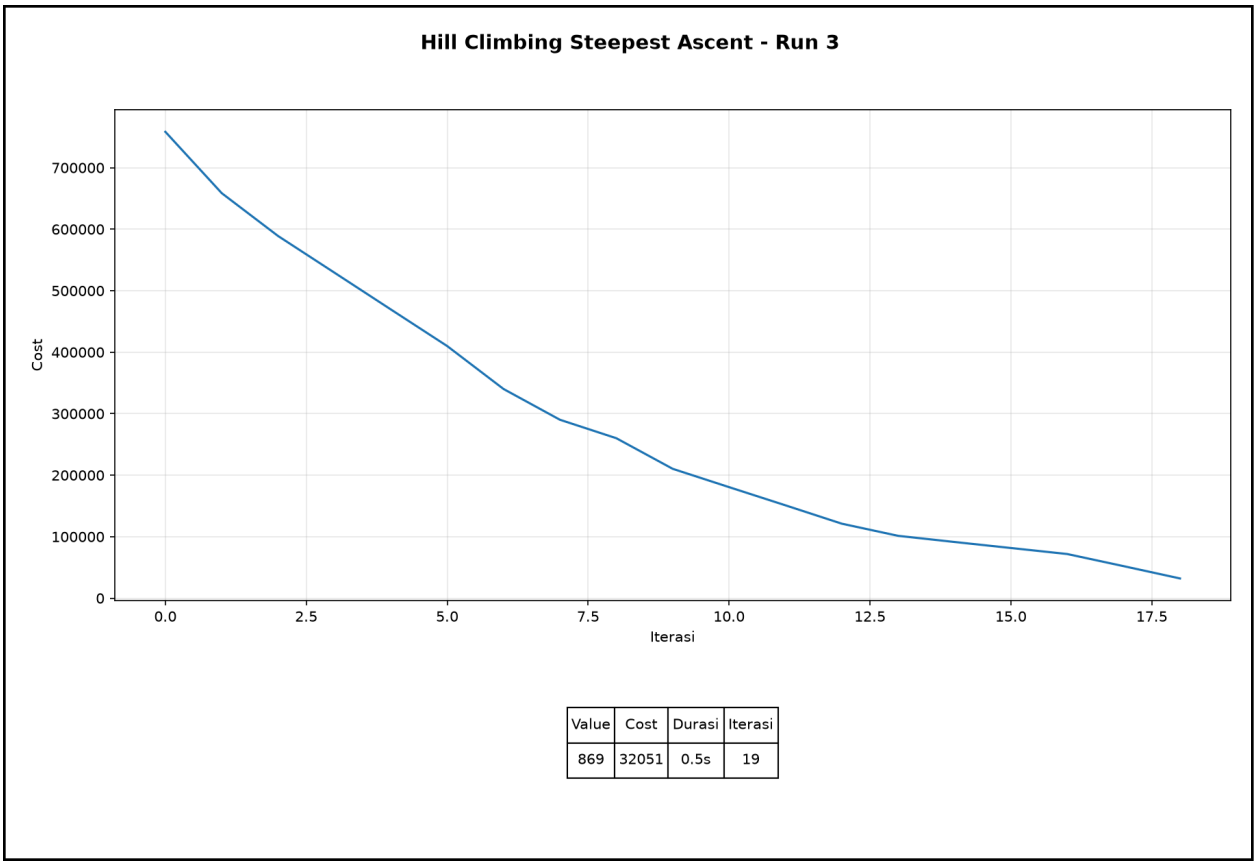



```
TOTAL VALUE di truk : 3625

[truckManhattanCafe] value=630  berat=130/500
  constraint: RELAXGAGAR -> clipping, melayang
[truckHishiAkebono] value=1955/590  berat=395/590
  constraint: RELAXGAGAR -> clipping, melayang
[truckNishinoFlower] value=240  berat=21/100
  constraint: RELAXGAGAR -> clipping, melayang

DI DALAM PETI KAYANG (id: x,y,z | w x l x h [orientasi])
  --- truckManhattanCafe (11 paket) ---
   29: 1,1,0 | 6x3x5 [xzy] v=30 *fragile  <- clipping dgn 6, clipping dgn 14, clipping dgn 30
   12: 2,6,1 | 3x10x2 [yxz] v=21  <-- clipping dgn 21, clipping dgn 0, clipping dgn 8, clipping dgn 14, clipping dgn 16, clipping dgn 15, melayang
   21: 7,14,0 | 5x5x4 [zyx] v=40  <- clipping dgn 27, clipping dgn 29
   25: 5,11,1 | 5x5x4 [zyx] v=41  <- clipping dgn 14, clipping dgn 19, clipping dgn 19
   0: 0,2,1 | 10x10x3 [xzy] v=67  <- clipping dgn 23, clipping dgn 12, clipping dgn 28, clipping dgn 8, clipping dgn 14, clipping dgn 16, clipping dgn 24, clipping dgn 10, clipping dgn 15, melayang
   8: 0,4,2 | 6x10x3 [xzy] v=61  <- clipping dgn 12, clipping dgn 0, clipping dgn 14, clipping dgn 16, clipping dgn 24, clipping dgn 10, clipping dgn 15, melayang
   14: 9,1,0 | 9x10x4 [xyz] v=76 *fragile  <-- clipping dgn 23, clipping dgn 12, clipping dgn 28, clipping dgn 8, clipping dgn 8, clipping dgn 10, clipping dgn 24, clipping dgn 10, clipping dgn 15
   19: 9,4,1 | 5x10x4 [zyx] v=97 *fragile  <- clipping dgn 17, clipping dgn 28, clipping dgn 8, clipping dgn 8, clipping dgn 14, clipping dgn 10, clipping dgn 15, melayang
   26: 0,9,0 | 5x10x3 [xzy] v=68  <- clipping dgn 23, clipping dgn 28, clipping dgn 8, clipping dgn 14, clipping dgn 16, clipping dgn 15
   18: 7,7,1 | 5x10x4 [zyx] v=48  <- clipping dgn 23, clipping dgn 28, clipping dgn 8, clipping dgn 8, clipping dgn 14, clipping dgn 16, clipping dgn 15
   15: 0,4,1 | 5x10x4 [zxy] v=51  <- clipping dgn 12, clipping dgn 0, clipping dgn 8, clipping dgn 14, clipping dgn 16, melayang
  --- truckHishiAkebono (15 paket) ---
   24: 7,14,0 | 6x4x4 [yxz] v=96  <- clipping dgn 22, clipping dgn 2, clipping dgn 11, clipping dgn 26, melayang
   23: 7,14,4 | 6x4x4 [yxz] v=58  <- clipping dgn 9
   10: 10,10,0 | 1x1x4 [zyx] v=26  <- melayang
   22: 5,17,1 | 10x1x8 [zyx] v=28 *fragile  <- clipping dgn 27, clipping dgn 2, clipping dgn 11, clipping dgn 29, clipping dgn 2, clipping dgn 29, melayang
   2: 8,20,0 | 5x9x10 [xyz] v=33 *fragile  <- clipping dgn 27, clipping dgn 22, clipping dgn 13, clipping dgn 29, clipping dgn 5, clipping dgn 29
   11: 2,18,0 | 10x4x9 [zyx] v=96 *fragile  <- clipping dgn 27, clipping dgn 22, clipping dgn 2, clipping dgn 29, clipping dgn 5, clipping dgn 29
   17: 0,13,0 | 5x10x4 [zyx] v=88 *fragile  <- clipping dgn 27, clipping dgn 22
   20: 0,11,1 | 9x10x4 [zyx] v=71  <- clipping dgn 27, clipping dgn 2, clipping dgn 11, clipping dgn 29, clipping dgn 29, clipping dgn 5, melayang
   3: 0,20,0 | 10x9x10 [xyz] v=39  <- clipping dgn 27, clipping dgn 2, clipping dgn 11, clipping dgn 29, clipping dgn 29, clipping dgn 9, clipping dgn 5
   6: 9,0,0 | 2x5x9 [yzx] v=95 *fragile  <- clipping dgn 15
   29: 1,10,0 | 8x10x8 [zyx] v=35  <- clipping dgn 22, clipping dgn 2, clipping dgn 11, clipping dgn 26, clipping dgn 3
   1: 2,3,0 | 2x3x6 [yzx] v=72
   9: 1,10,3 | 5x5x4 [zyx] v=91 *fragile  <- clipping dgn 19, clipping dgn 26, clipping dgn 3
   5: 1,21,5 | 5x3x4 [xzy] v=31  <- clipping dgn 29, clipping dgn 5, melayang
   7: 3,1,4 | 4x4x3 [yzx] v=64  <- clipping dgn 13, melayang
  --- truckNishinoFlower (4 paket) ---
   19: 3,2,4 | 5x3x2 [xzy] v=49  <- clipping dgn 17, clipping dgn 29, melayang
   27: 0,0,4 | 4x4x4 [zyx] v=24  <- clipping dgn 17, clipping dgn 29, melayang
   4: 0,0,2 | 1x4x3 [zyx] v=71  <- clipping dgn 1, melayang
   28: 3,1,0 | 2x1x6 [xzy] v=33 *fragile  <-  clipping dgn 25, clipping dgn 17, clipping dgn 4

DI LUAR / GUDANG (0 paket)
(kosong)
```

Hill Climbing Steepest Ascent - Run 3 - Final




```
TOTAL VALUE di truk : 869

[truckManhattanCafe] value=330  berat=47/500
  constraint: OK
[truckHishiAkebono] value=999  berat=551/500
  constraint: MELANGGAR -> clipping, melayang
[truckNishinoFlower] value=31  berat=19/500
  constraint: OK

DI DALAM PETI KAYANG (id: x,y,z | w x l x h [orientasi])
  --- truckManhattanCafe (4 paket) ---
   23: 1,2,0 | 6x2x5 [xzy] v=30 *fragile
   21: 1,14,0 | 6x6x4 [zyx] v=93 *fragile
   0: 0,2,1 | 10x10x3 [xzy] v=67
   4: 0,4,0 | 6x1x6 [zyx] v=72
  --- truckHishiAkebono (11 paket) ---
   15: 7,14,0 | 6x3x1 [yzx] v=56  <-- clipping dgn 9
   10: 10,10,0 | 1x1x4 [zyx] v=26  <-- melayang
   2: 8,20,0 | 5x9x10 [xyz] v=33 *fragile  <-- clipping dgn 29
   17: 0,13,0 | 5x10x4 [zyx] v=88 *fragile
   29: 1,10,0 | 8x10x8 [xzy] v=35  <-- clipping dgn 2
   1: 2,3,0 | 2x3x6 [yzx] v=60
   9: 6,16,0 | 9x10x10 [yxz] v=61 *fragile  <-- clipping dgn 19
   7: 3,1,4 | 4x4x3 [yzx] v=64
   13: 7,12,1 | 6x4x5 [yzx] v=25
   5: 1,21,5 | 5x3x4 [xzy] v=31
   28: 3,2,0 | 9x1x2 [yxz] v=41
  --- truckNishinoFlower (1 paket) ---
   3: 3,2,0 | 4x3x2 [yxz] v=31

DI LUAR / GUDANG (14 paket)
0: 10x10x10 v=67 berat=0
22: 6x2x10 v=30 berat=14 *fragile
18: 9x1x10 v=68 berat=1k
16: 4x10x9 v=91 berat=7 *fragile
3: 10x10x9 v=39 berat=2k
26: 10x9x8 v=43 berat=0
24: 2x6x10 v=52 berat=9 *fragile
20: 2x8x5 v=33 berat=9 *fragile
12: 9x8x6 v=68 berat=19 *fragile
8: 10x5x2 v=21 berat=5
25: 5x3x3 v=88 berat=0
14: 9x10x10 v=76 *fragile
11: 9x9x10 v=90 berat=k *fragile
27: 4x4x4 v=24 berat=13
```



2.3.1.2 Analisis

Berdasarkan hasil tersebut, didapatkan informasi sebagai berikut:

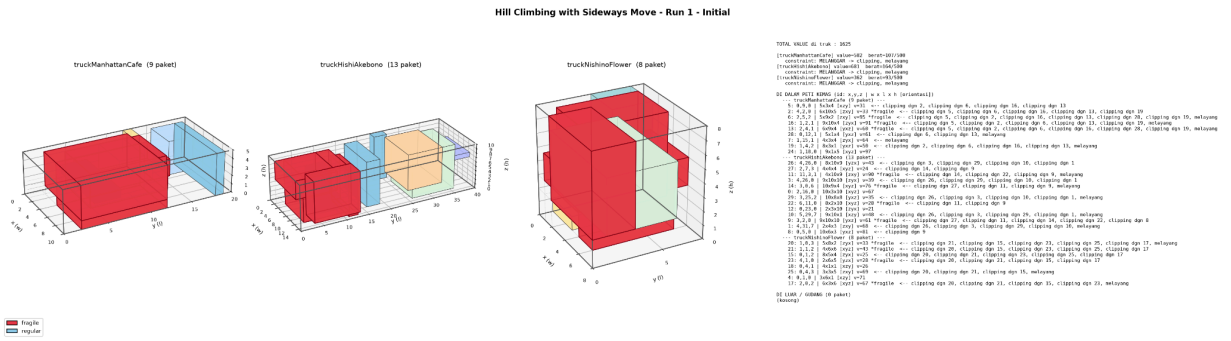
1) Run 1 menghasilkan final state dengan total value sebesar 1129, cost sebesar 80471, durasi selama 0.7 detik, dan iterasi sebanyak 14. Namun, terdapat 8 paket yang melanggar constraint (clipping dan melayang) di truckHishiAkebono.
2) Run 2 menghasilkan final state dengan total value sebesar 731, cost sebesar 3549, durasi selama 0.8 detik, dan iterasi sebanyak 29. Tidak ada paket yang melanggar constraint.
3) Run 3 menghasilkan final state dengan total value sebesar 869, cost sebesar 32051, durasi selama 0.5 detik, dan iterasi sebanyak 19. Namun, terdapat 5 paket yang melanggar constraint (clipping dan melayang) di truckHishiAkebono.

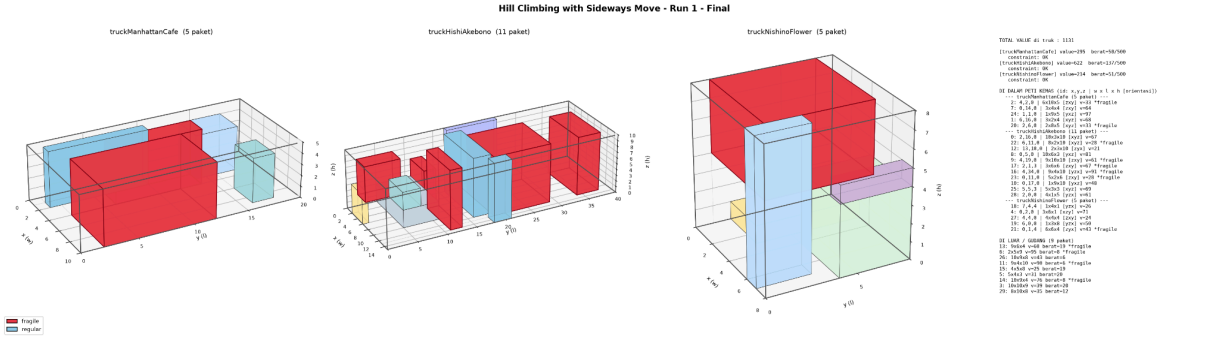
Berdasarkan 3 run yang telah dijalankan, algoritma Steepest Ascent merupakan algoritma dengan durasi paling cepat dan iterasinya cenderung lebih sedikit dibandingkan algoritma lainnya. Meskipun begitu, algoritma ini menunjukkan hasil cost yang sangat tidak konsisten (80471, 3549, dan 32051). Selain itu, algoritma ini sering terjebak pada local optimum. Hal ini bisa dilihat pada Run 1 dan Run 3 yang masih terdapat beberapa paket yang melanggar constraint (clipping dan melayang), di mana algoritma ini berhenti sangat cepat (pada 14 dan 19 iterasi) dengan cost yang yang sangat besar (80471 dan 32051). Hal ini mungkin disebabkan oleh sifat algoritma ini yang hanya mau menerima neighbor dengan cost lebih rendah sehingga tidak berani melewati lembah untuk memperbaiki pelanggaran tersebut.

2.3.2. Sideways Move Hill Climbing

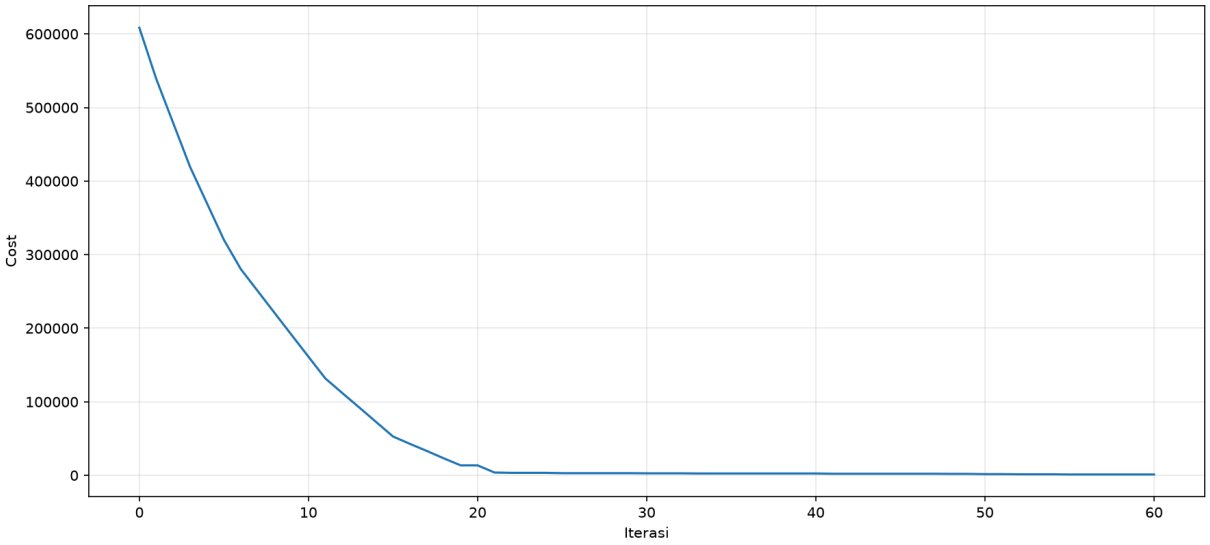
2.3.2.1 Hasil

Run 1



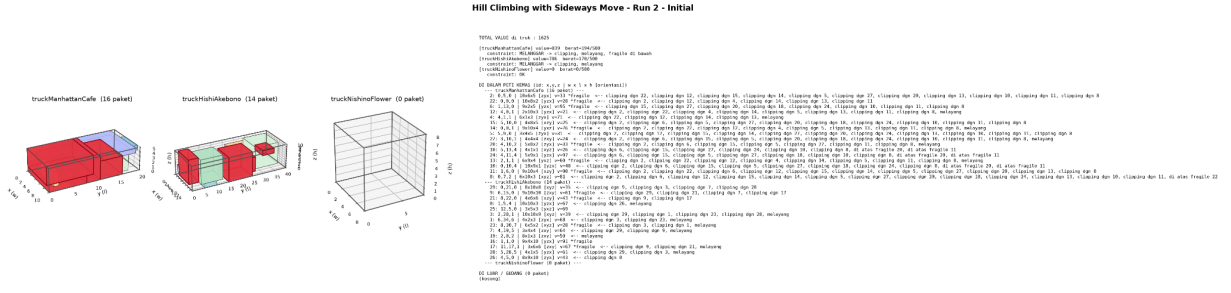


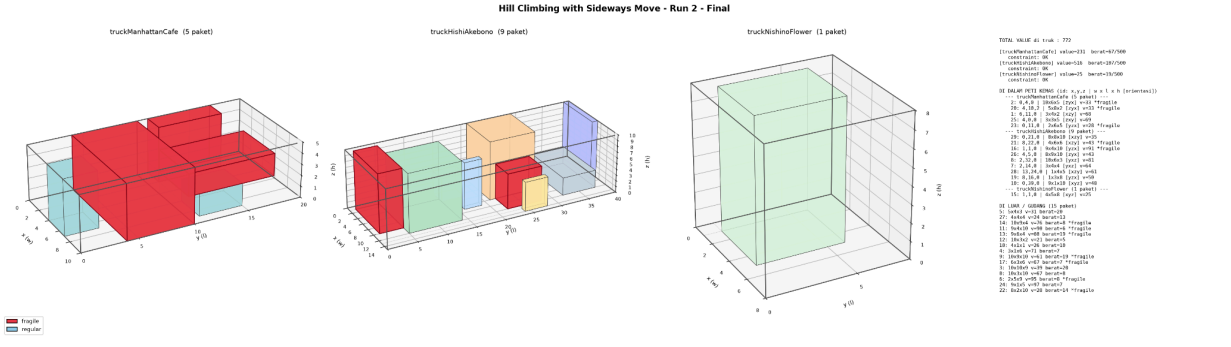
Hill Climbing with Sideways Move - Run 1

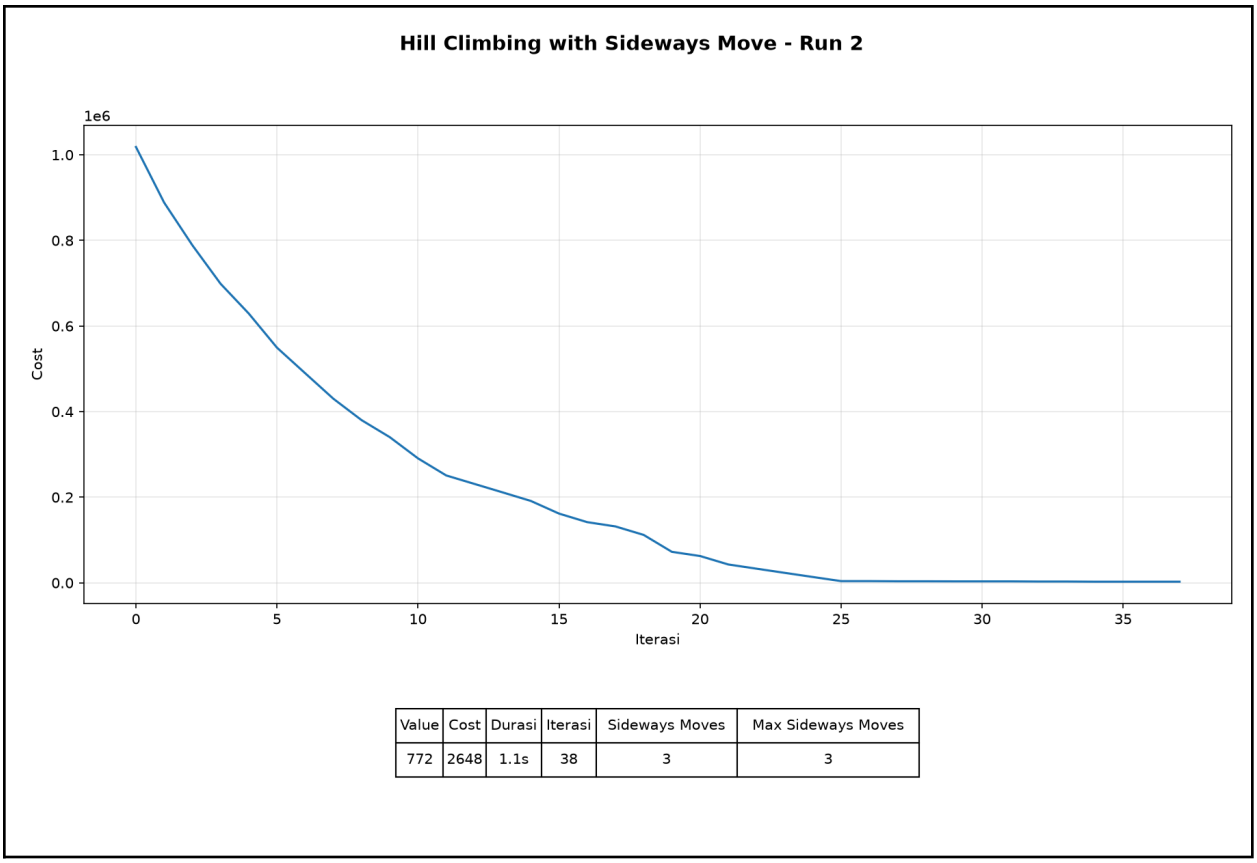


Value	Cost	Durasi	Iterasi	Sideways Moves	Max Sideways Moves
1131	1229	1.1s	61	5	10

Run 2

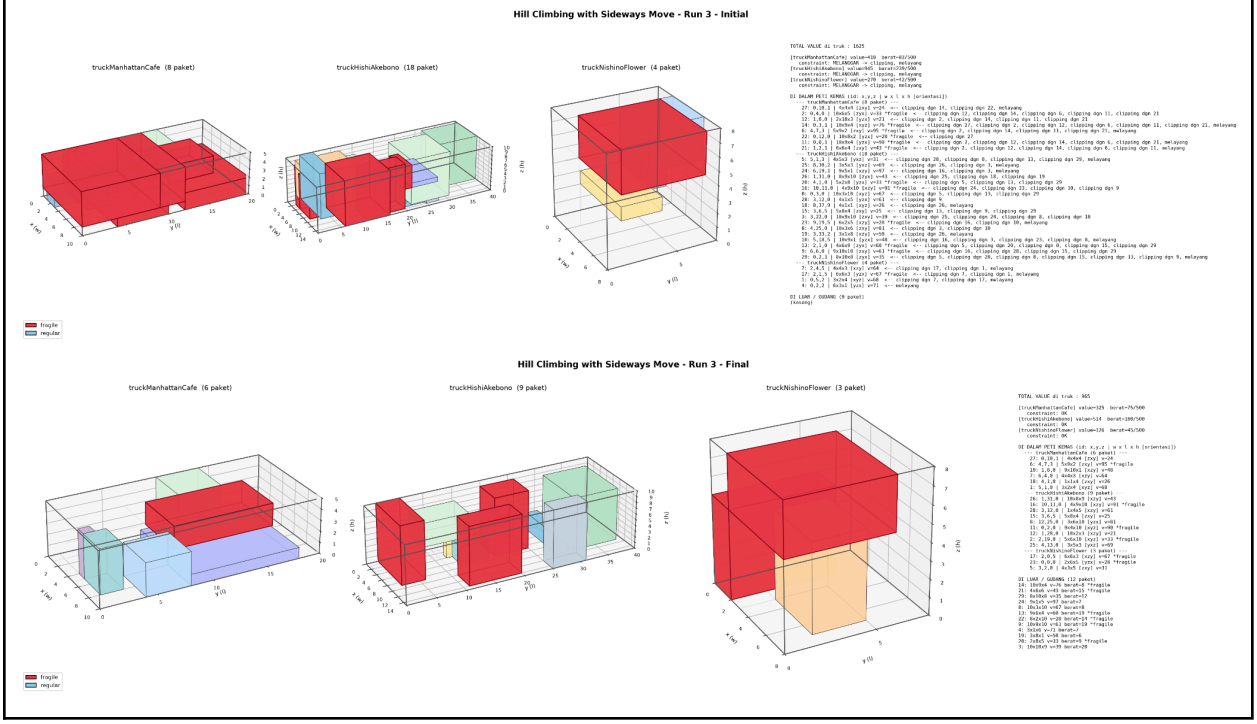


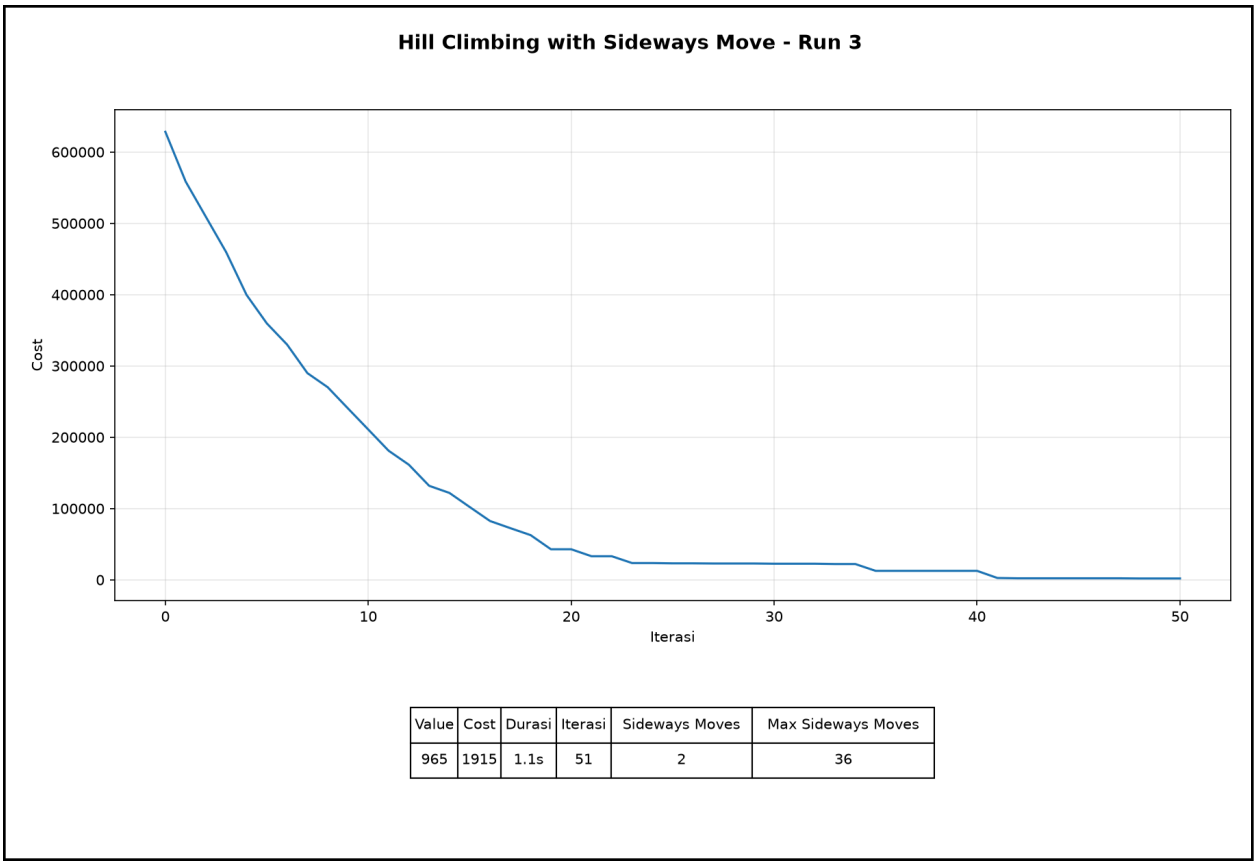




Value	Cost	Durasi	Iterasi	Sideways Moves	Max Sideways Moves
772	2648	1.1s	38	3	3

Run 3





Value	Cost	Durasi	Iterasi	Sideways Moves	Max Sideways Moves
965	1915	1.1s	51	2	36

2.3.2.2 Analisis

Berdasarkan hasil tersebut, didapatkan informasi sebagai berikut:

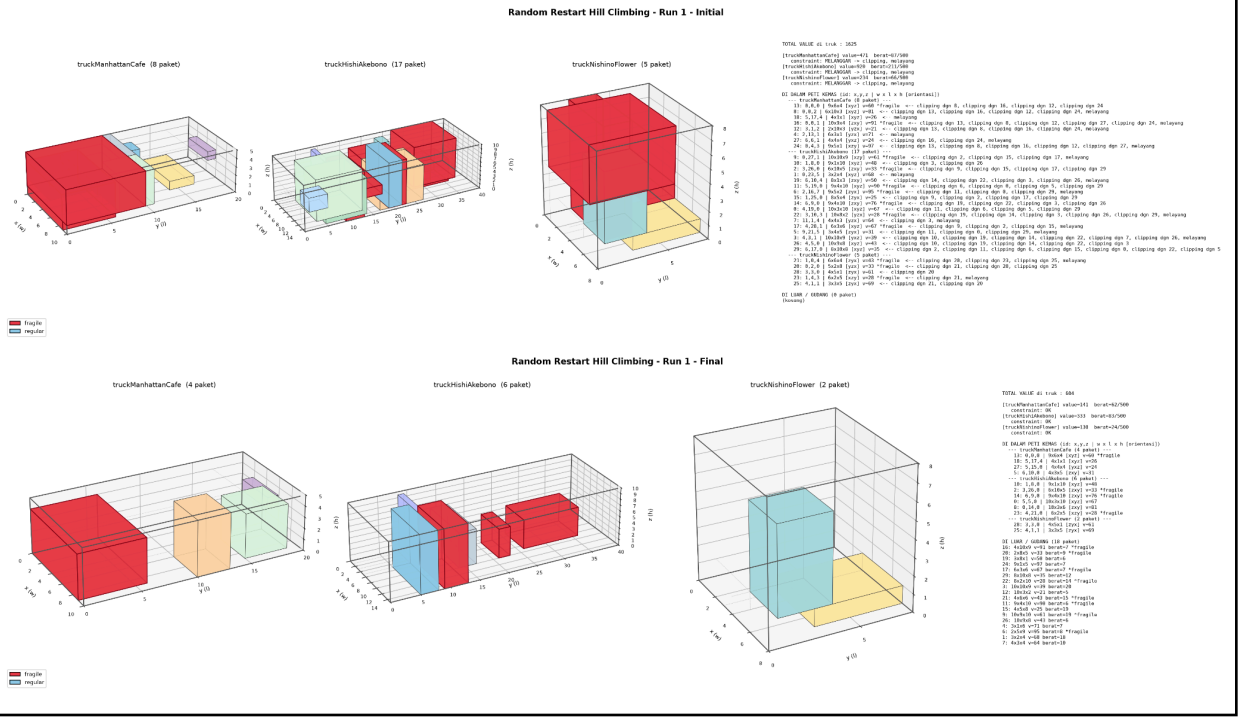
1) Run 1 menghasilkan final state dengan total value sebesar 1131, cost sebesar 1229, durasi selama 1.1 detik, iterasi sebanyak 61, dan 5/10 sideways move. Tidak ada paket yang melanggar constraint.
2) Run 2 menghasilkan final state dengan total value sebesar 772, cost sebesar 2648, durasi selama 1.1 detik, iterasi sebanyak 38, dan 3/3 sideways move. Tidak ada paket yang melanggar constraint.
3) Run 3 menghasilkan final state dengan total value sebesar 965, cost sebesar 1915, durasi selama 1.1 detik, iterasi sebanyak 51, dan 2/36 sideways move. Tidak ada paket yang melanggar constraint.

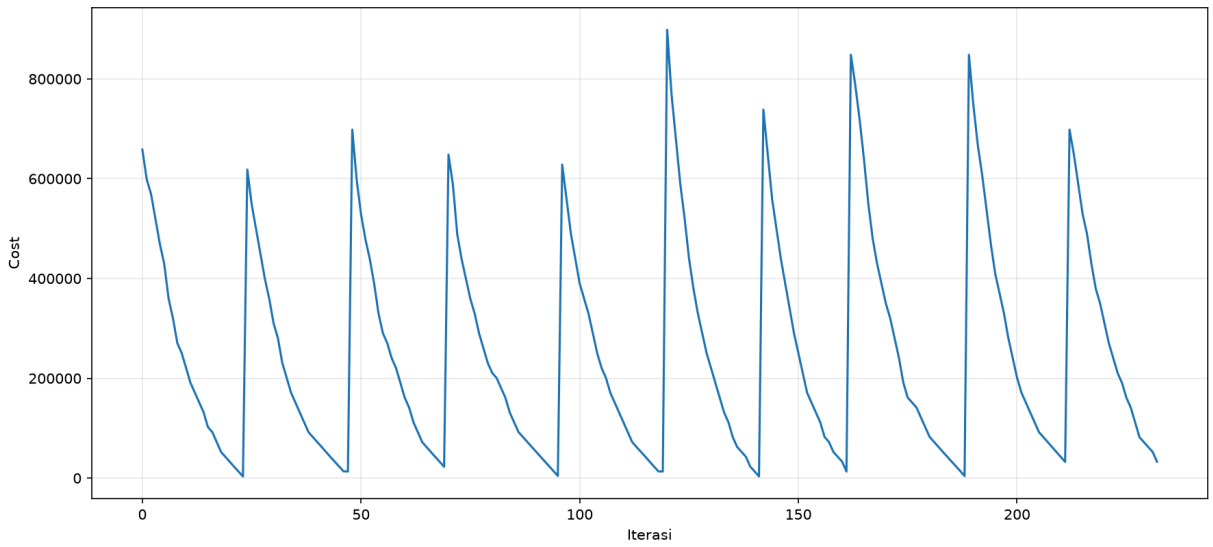
Berdasarkan 3 run yang telah dijalankan, algoritma Sideways Move jauh lebih baik daripada Steepest Ascent. Dengan mengizinkan perpindahan ke neighbor dengan cost yang sama, algoritma ini dapat keluar dari local optimum. Dapat dilihat pada ketiga run, algoritma ini berhasil menemukan solusi yang valid (tidak ada pelanggaran constraint sama sekali) dengan cost yang jauh lebih rendah dan cukup konsisten (1229 - 2648) dibandingkan Steepest Ascent. Waktu eksekusinya sedikit lebih lama karena adanya tambahan iterasi untuk melakukan sideways move.

2.3.3. Random Restart Hill Climbing

2.3.3.1 Hasil

Run 1

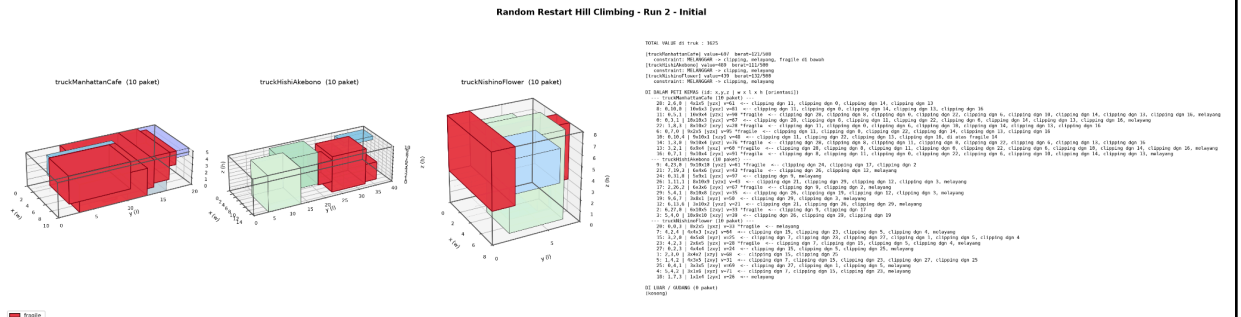


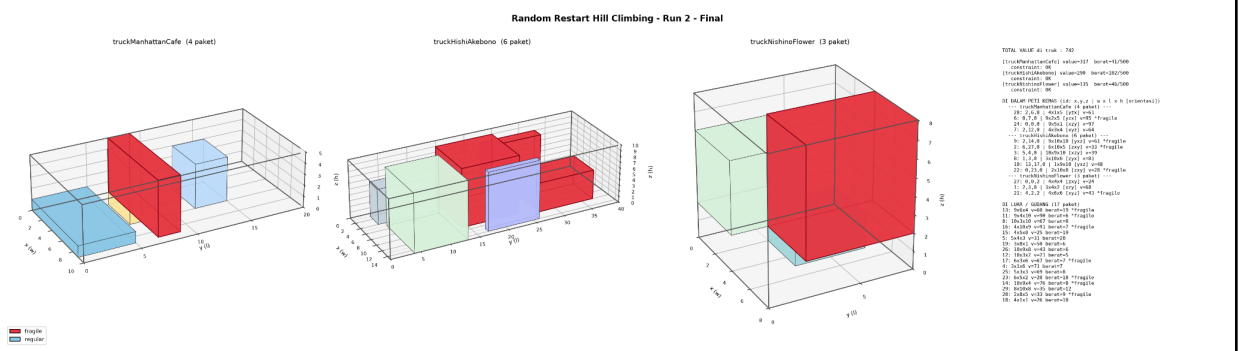


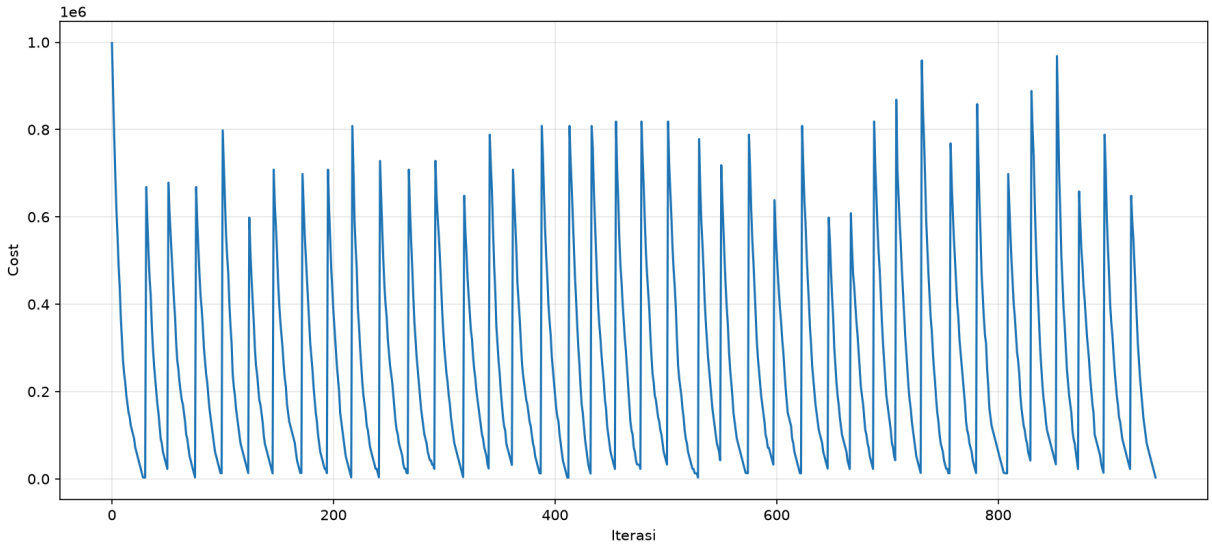
Value	Cost	Durasi	Iterasi	Restarts	Avg Iter/Restart
604	3296	6.0s	233	10	23.3

Iterasi per restart = [24, 24, 22, 26, 24, 22, 20, 27, 23, 21]

Run 2



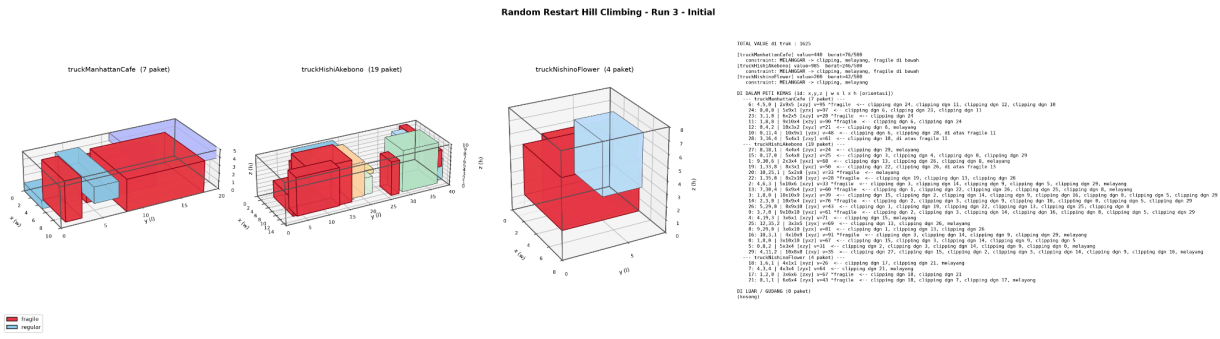


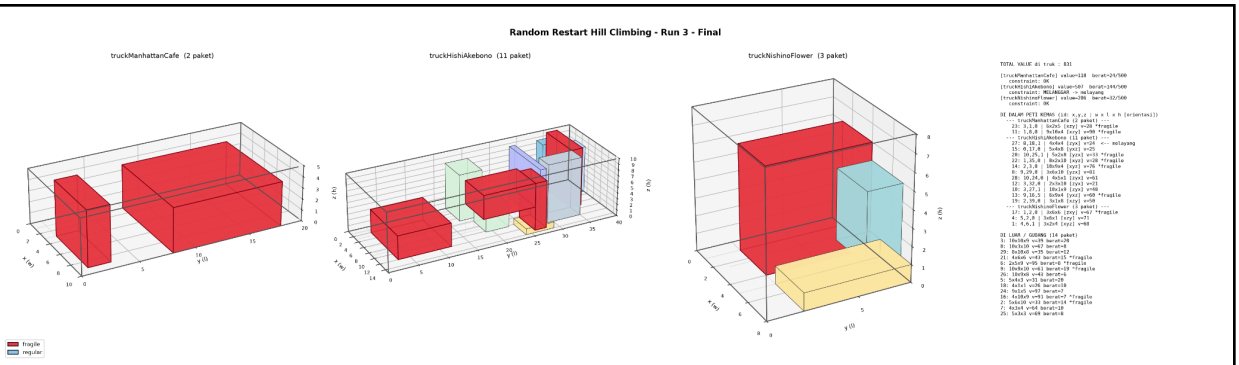


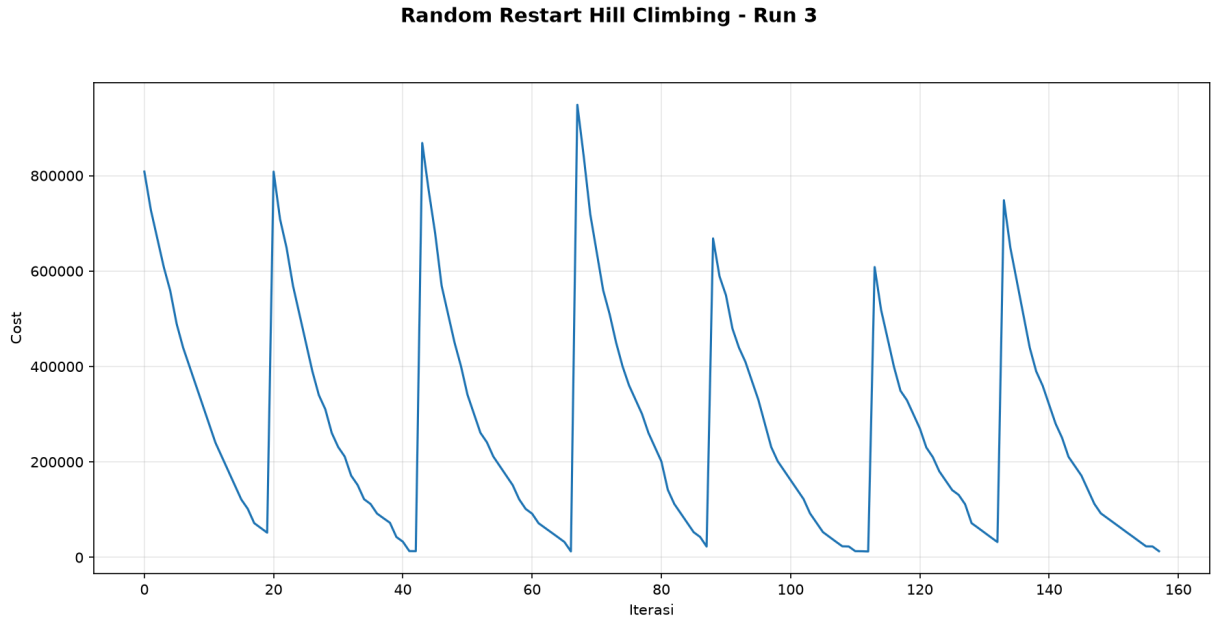
Value	Cost	Durasi	Iterasi	Restarts	Avg Iter/Restart
742	2758	43.7s	943	40	23.6

Iterasi per restart = [31, 20, 25, 24, 24, 22, 26, 23, 22, 25, 26, 24, 26, 23, 21, 26, 25, 20, 22, 23, 24, 28, 20, 25, 23, 25, 24, 20, 21, 20, 23, 26, 24, 28, 21, 23, 20, 23, 24, 23]

Run 3







Value	Cost	Durasi	Iterasi	Restarts	Avg Iter/Restart
831	12449	9.5s	158	7	22.6

Iterasi per restart = [20, 23, 24, 21, 25, 20, 25]

2.3.3.2 Analisis

Berdasarkan hasil tersebut, didapatkan informasi sebagai berikut:

1) Run 1 menghasilkan final state dengan total value sebesar 604, cost sebesar 3296, durasi selama 6.0 detik, total iterasi sebanyak 233, jumlah restart sebanyak 10, dan rata-rata iterasi per restart sebanyak 23.3. Tidak ada paket yang melanggar constraint.
2) Run 2 menghasilkan final state dengan total value sebesar 742, cost sebesar 2758, durasi selama 43.7 detik, total iterasi sebanyak 943, jumlah restart sebanyak 40, dan rata-rata iterasi per restart sebanyak 23.6. Tidak ada paket yang melanggar constraint.
3) Run 3 menghasilkan final state dengan total value sebesar 831, cost sebesar 12449, durasi selama 9.5 detik, total iterasi sebanyak 158, jumlah restart sebanyak 7, dan

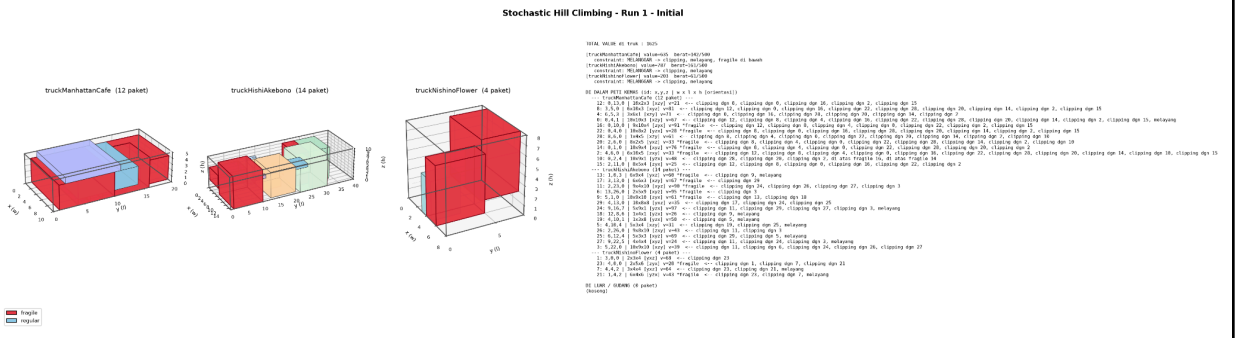
rata-rata iterasi per restart sebanyak 22.6. Namun, terdapat 1 paket yang melanggar constraint (melayang) di truckHishiAkebono.

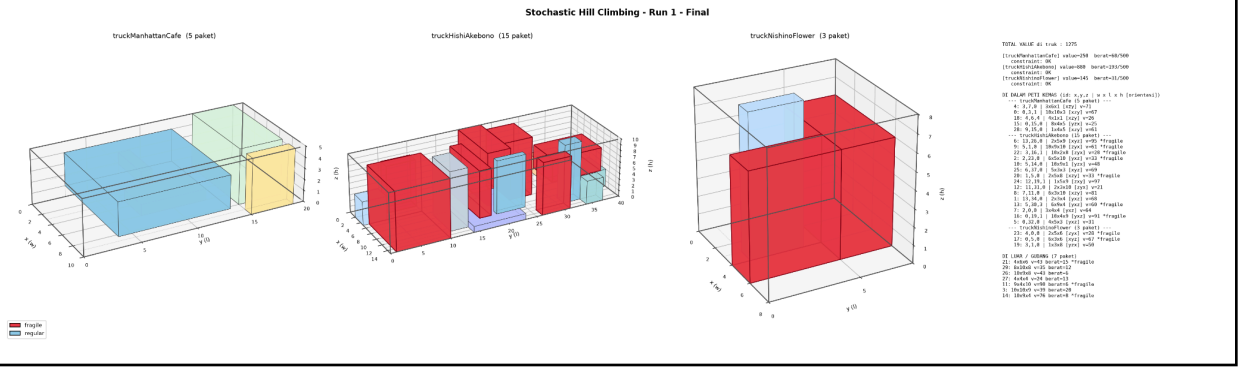
Berdasarkan 3 run yang telah dijalankan, algoritma Random Restart memiliki waktu eksekusi paling lama dibanding algoritma Hill Climbing lainnya. Hal ini terjadi karena algoritma harus menjalankan proses Hill Climbing berulang kali dari initial state yang baru. Secara umum, hasil algoritma ini cukup bervariasi dan kurang konsisten, pada Run 1 dan Run 2 algoritma berhasil menemukan solusi tanpa adanya pelanggaran (costnya sebesar 3296 dan 2758), namun pada Run 3 (dengan jumlah restart sebanyak 7, paling sedikit di antara run lainnya) masih terdapat 1 paket yang melayang (costnya sebesar 12449). Kedekatan dengan global optima bergantung pada seberapa banyak restart yang diizinkan. Pada Run 2, maxRestart diatur hingga 40 kali, jauh lebih tinggi dari Run lainnya. Hal ini menyebabkan waktu eksekusi meningkat secara drastis menjadi 43.7 detik dengan total 943 iterasi. Namun, meningkatnya batas restart tersebut tidak membuat peningkatan kualitas solusi yang signifikan, cost akhir Run 2 (2758) tidak jauh berbeda dengan Run 1 (3296). Hal ini menunjukkan bahwa penambahan jumlah restart tidak menjamin penemuan solusi yang jauh lebih optimal dan bisa saja malah hanya menambah beban komputasi. Oleh karena itu, diperlukan tuning parameter maxRestarts agar menghasilkan solusi yang optimal dan tidak membuat beban komputasi membengkak.

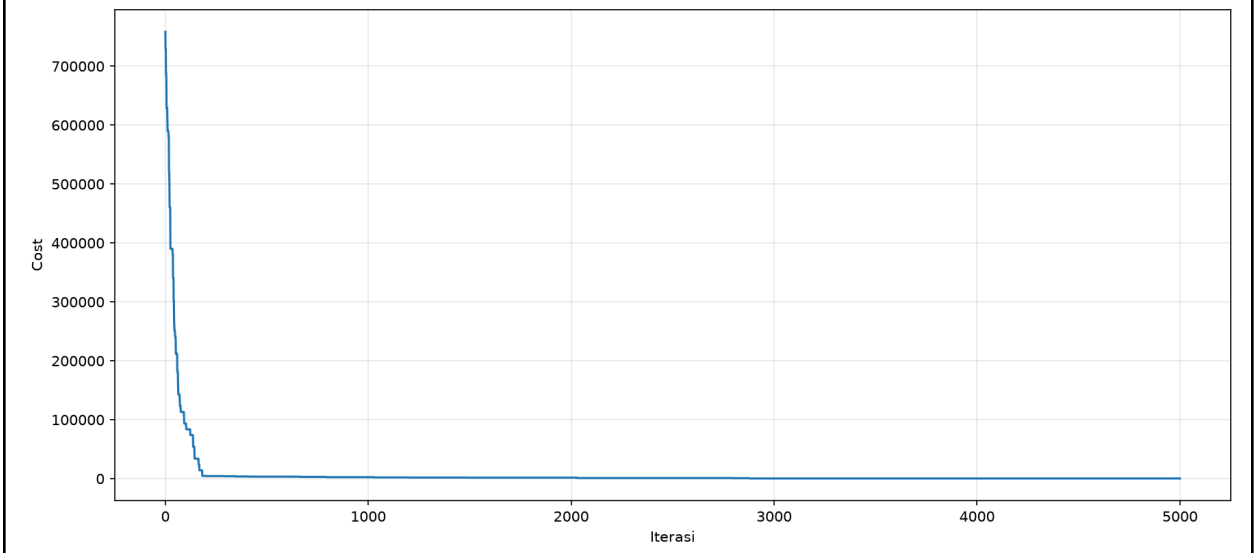
2.3.4. Stochastic Hill Climbing

2.3.4.1 Hasil

Run 1

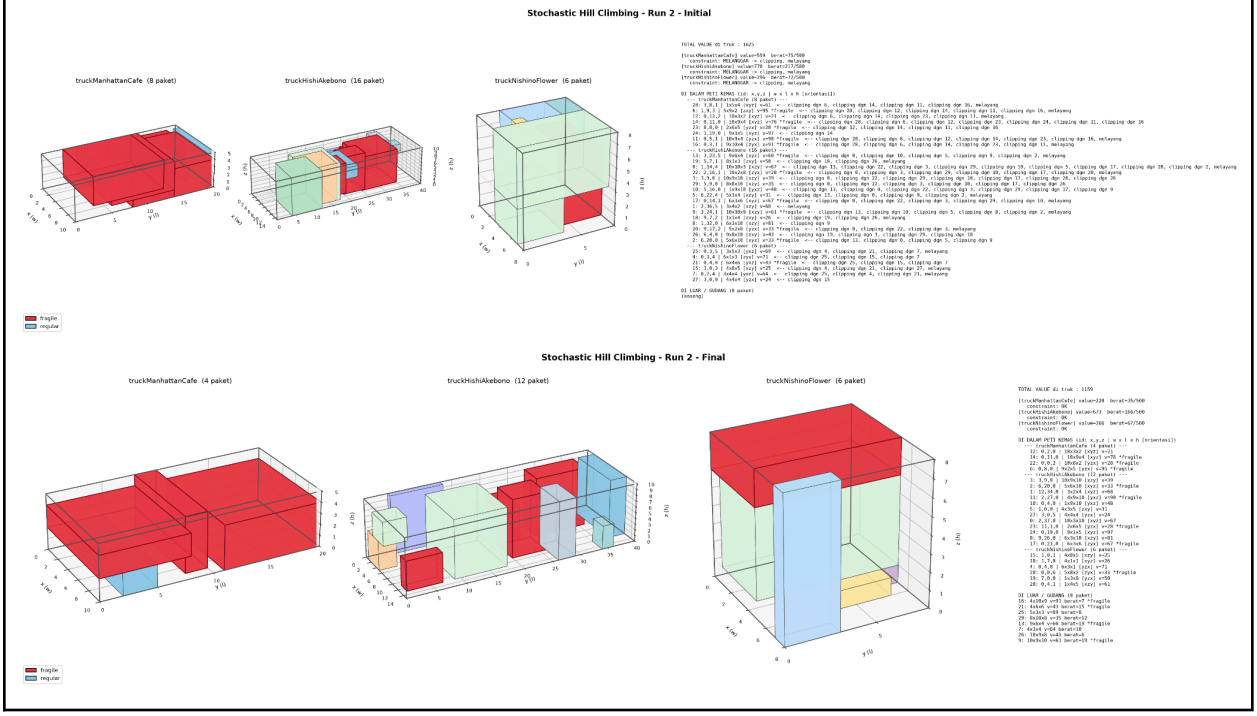




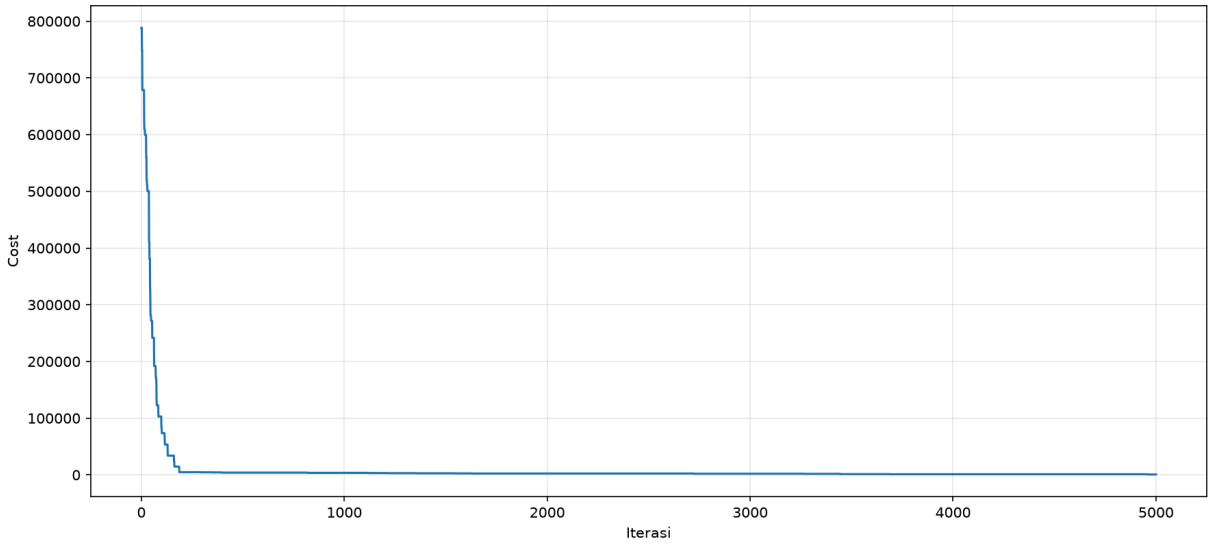


Value	Cost	Durasi	Iterasi
1275	325	2.3s	5000

Run 2

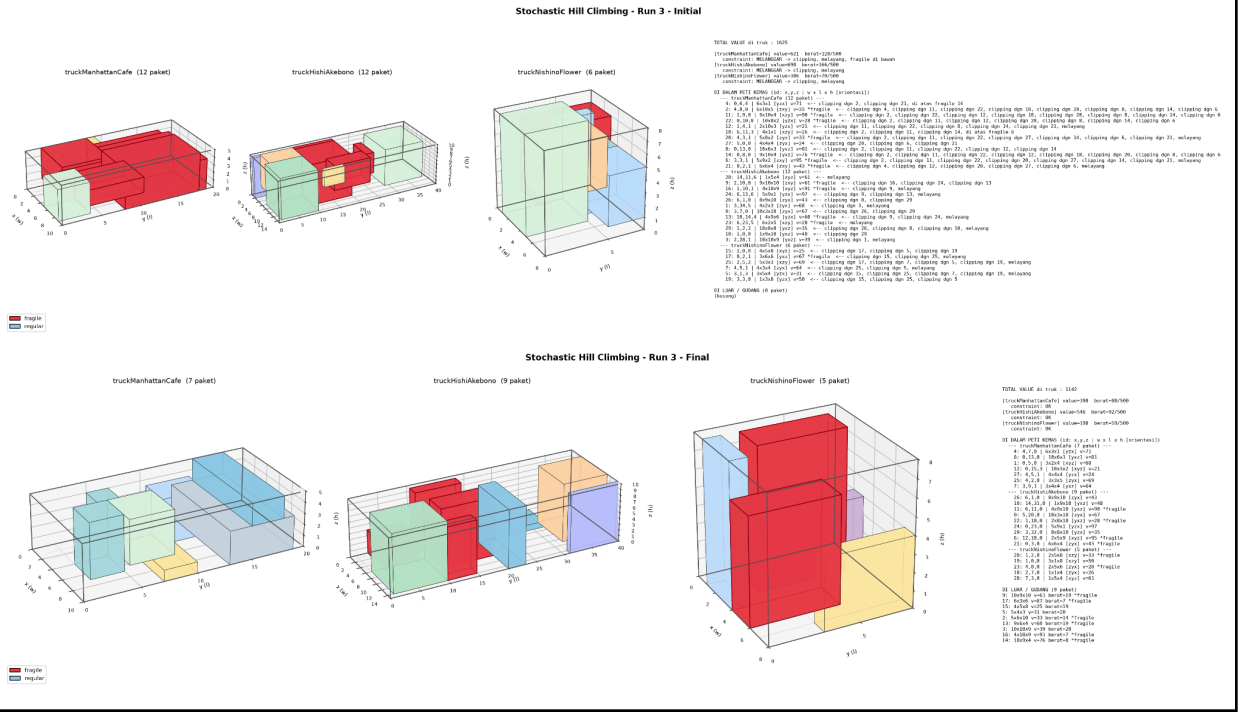


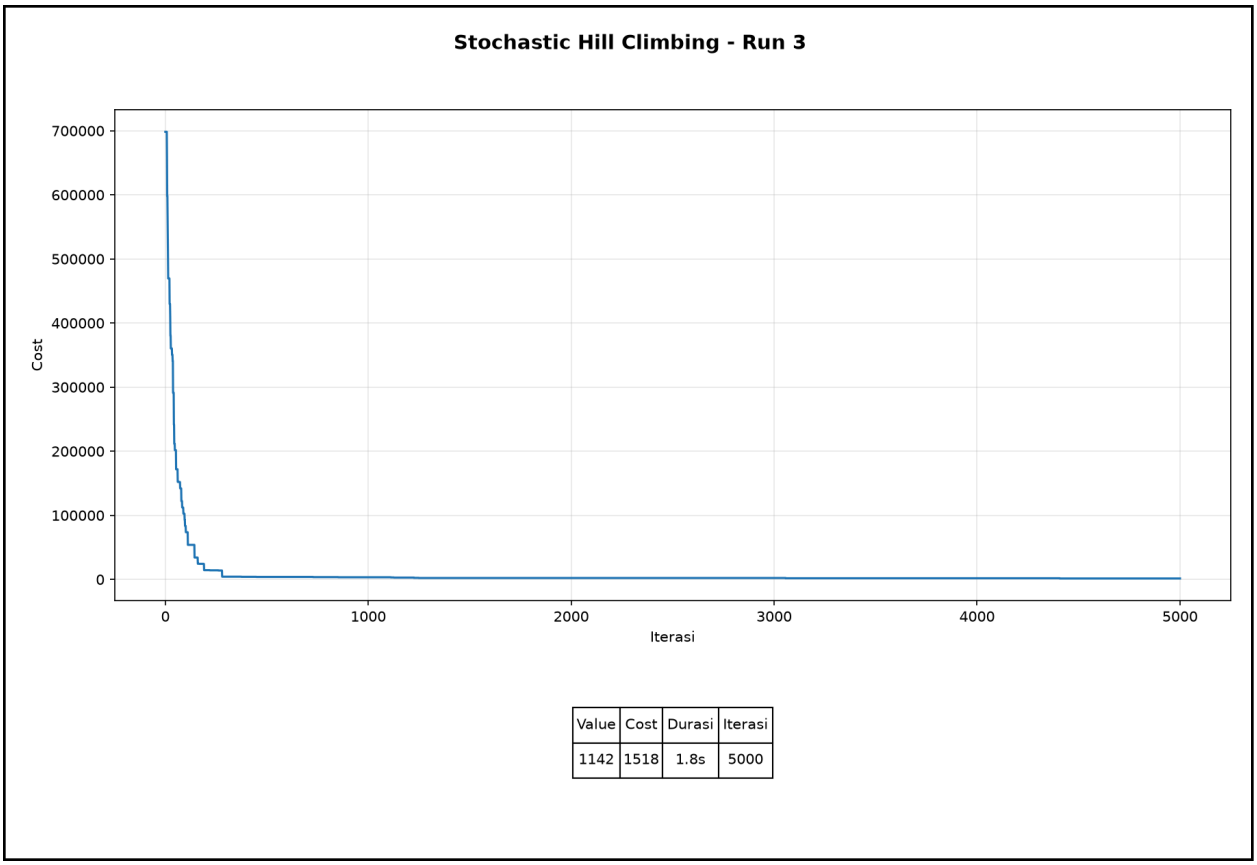
Stochastic Hill Climbing - Run 2



Value	Cost	Durasi	Iterasi
1159	761	1.6s	5000

Run 3





Value	Cost	Durasi	Iterasi
1142	1518	1.8s	5000

2.3.4.2 Analisis

Berdasarkan hasil tersebut, didapatkan informasi sebagai berikut:

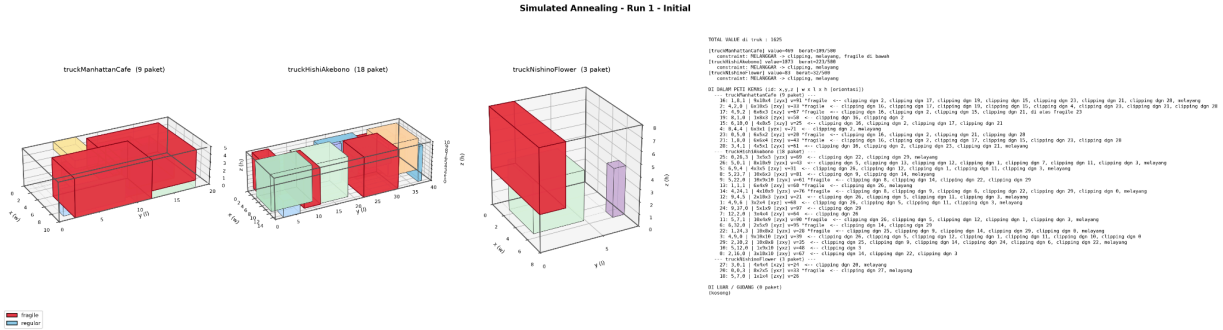
1) Run 1 menghasilkan final state dengan total value sebesar 1275, cost sebesar 325, durasi selama 2.3 detik, dan iterasi sebanyak 5000. Tidak ada paket yang melanggar constraint.
2) Run 2 menghasilkan final state dengan total value sebesar 1159, cost sebesar 761, durasi selama 1.6 detik, dan iterasi sebanyak 5000. Tidak ada paket yang melanggar constraint.
3) Run 3 menghasilkan final state dengan total value sebesar 1142, cost sebesar 1518, durasi selama 1.8 detik, dan iterasi sebanyak 5000. Tidak ada paket yang melanggar constraint.

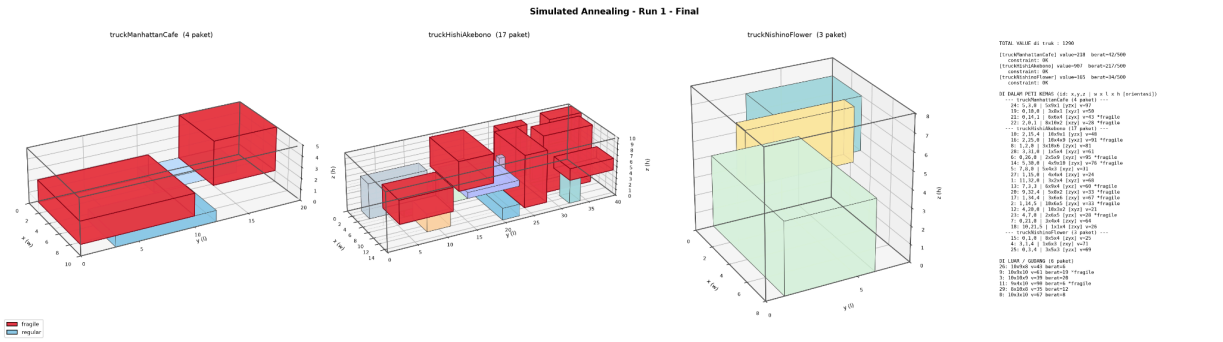
Berdasarkan 3 Run yang telah dijalankan, algoritma Stochastic Hill Climbing memberikan hasil yang paling baik dan stabil dibandingkan algoritma Hill Climbing lainnya. Walaupun hanya mengevaluasi satu neighbor acak per iterasi, algoritma ini dijalankan hingga batas maksimalnya yaitu 5000 iterasi. Jumlah iterasi yang sangat besar tersebut menyebabkan algoritma ini dapat melakukan random walk dengan ruang eksplorasi yang luar sehingga menemukan konfigurasi dengan cost sangat rendah (325 - 1518) dan value tinggi (1142 - 1275) tanpa melanggar constraint apa pun.

2.3.5. Simulated Annealing

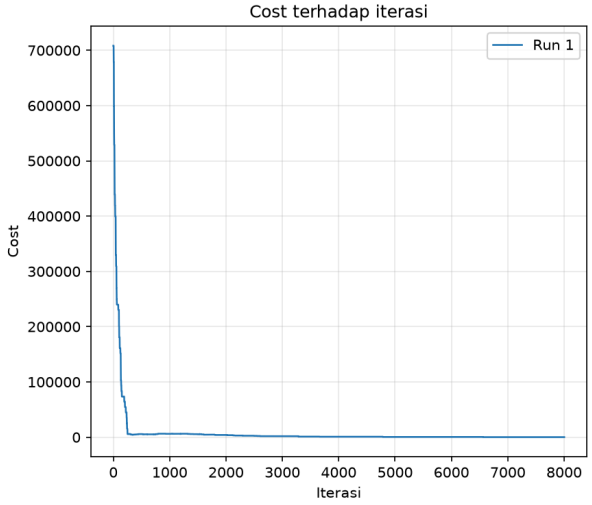
2.3.5.1 Hasil

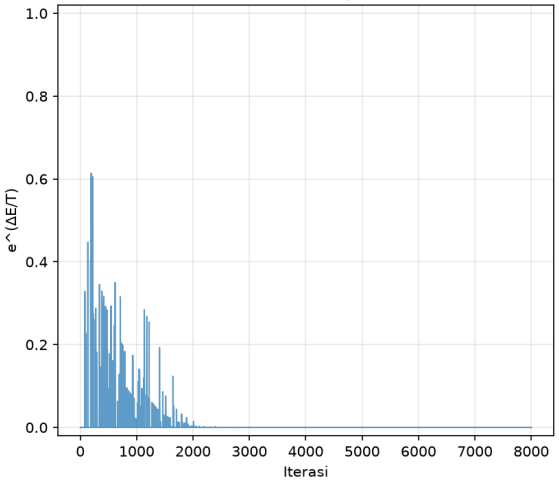
Run 1





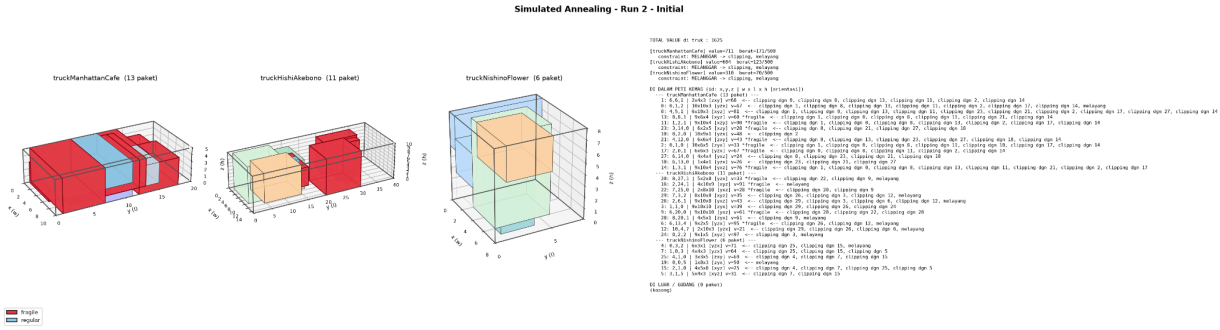
Simulated Annealing - Run 1

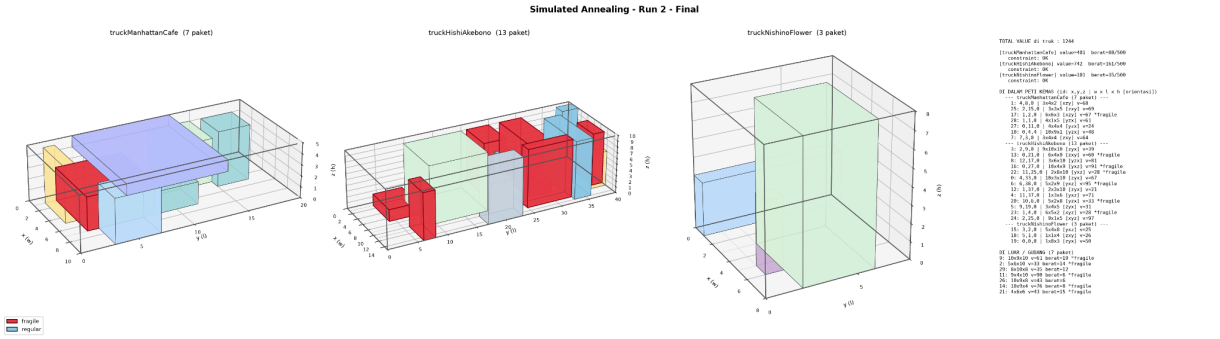




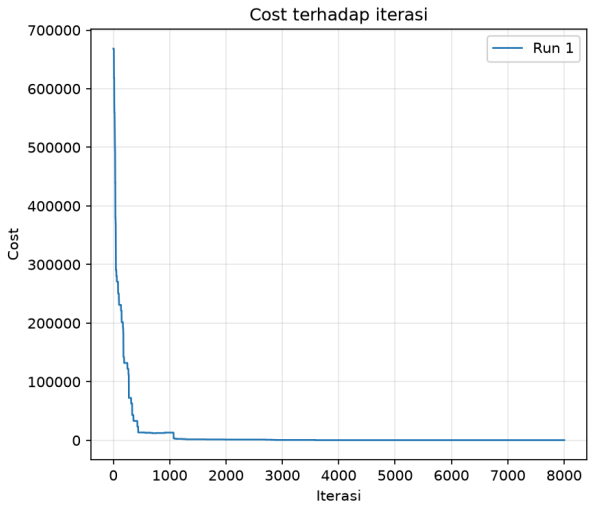
Value	Cost	Durasi (s)	Iterasi	Stuck (Frekuensi move lebih buruk ditolak)
1290	130	26.7	8003	5897

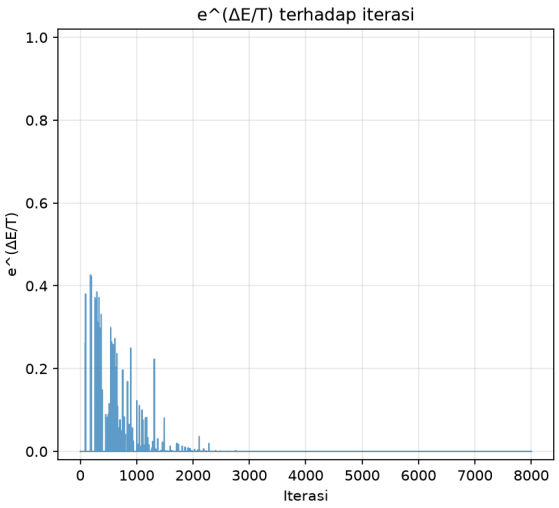
Run 2





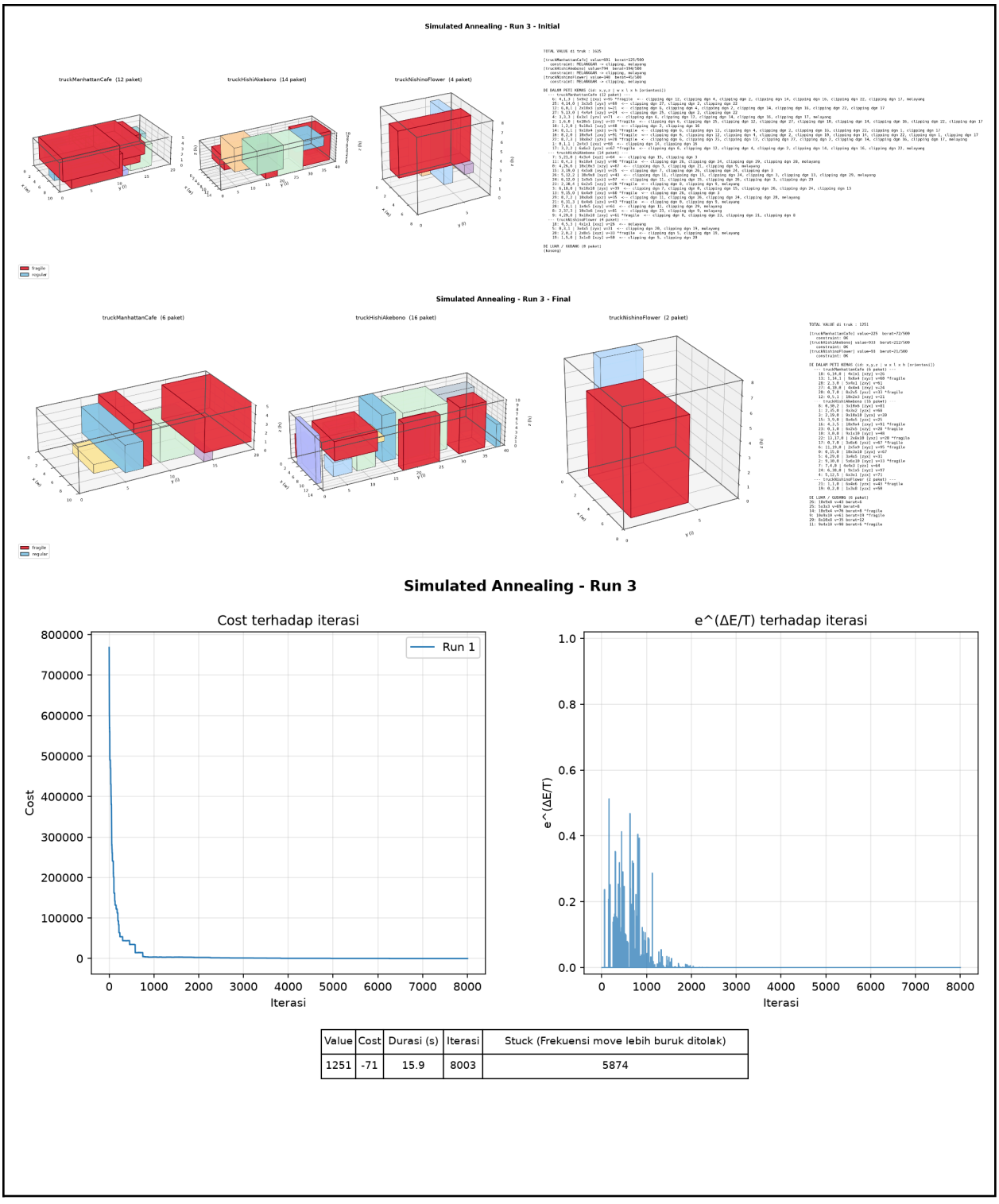
Simulated Annealing - Run 2





Value	Cost	Durasi (s)	Iterasi	Stuck (Frekuensi move lebih buruk ditolak)
1244	356	18.1	8003	5868

Run 3



Value	Cost	Durasi (s)	Iterasi	Stuck (Frekuensi move lebih buruk ditolak)
1251	-71	15.9	8003	5874

2.3.5.2 Analisis

Berdasarkan hasil tersebut, didapatkan informasi sebagai berikut:

1) Run 1 menghasilkan final state dengan total value sebesar 1290, cost sebesar 130, durasi selama 26.7 detik, dan iterasi sebanyak 8003. Tidak ada paket yang melanggar constraint.
2) Run 2 menghasilkan final state dengan total value sebesar 1244, cost sebesar 356, durasi selama 18.1 detik, dan iterasi sebanyak 8003. Tidak ada paket yang melanggar constraint.
3) Run 3 menghasilkan final state dengan total value sebesar 1251, cost sebesar -71, durasi selama 15.9 detik, dan iterasi sebanyak 8003. Tidak ada paket yang melanggar constraint.

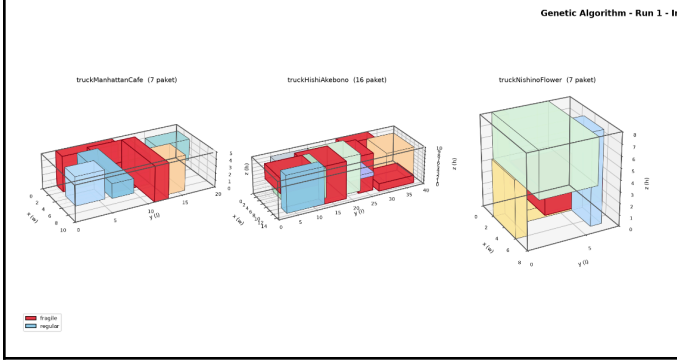
Berdasarkan 3 Run yang telah dijalankan, algoritma Simulated Annealing memberikan pula hasil value yang cukup konsisten di antara algoritma local search yang lain, dengan cost akhir yang dihasilkan memiliki rentang yang cukup besar antar-run, tetapi masih dapat dikatakan rata-rata cukup kecil jika dibandingkan dengan beberapa algoritma Hill Climbing. Sementara itu, jumlah iterasi yang sama pada tiap run tersebut disebabkan oleh penggunaan parameter cooling schedule yang sama, yakni T = 300 dan cooldown rate = 0.999 dengan proses pencarian berhenti ketika suhu tidak lagi lebih besar dari 0.1. Walau memiliki jumlah iterasi yang sama, waktu eksekusi mereka berbeda. Hal ini dikarenakan proses pencarian dan evaluasi kandidat pada tiap iterasi tidak selalu membutuhkan yang sama. Dibandingkan dengan varian Hill Climbing, SA mampu mendekati nilai optimal dengan lebih baik (cost lebih rendah dan value tinggi). Hal ini terjadi karena mekanisme yang SA tidak hanya menerima neighbor yang lebih baik, tetapi juga memiliki probabilitas untuk menerima neighbor yang lebih buruk di awal pencarian (saat suhu T masih tinggi). Kemampuan untuk melewati lembah ini membuat SA tidak mudah terjebak di local optimum seperti yang sering dialami oleh Hill Climbing Steepest Ascent.

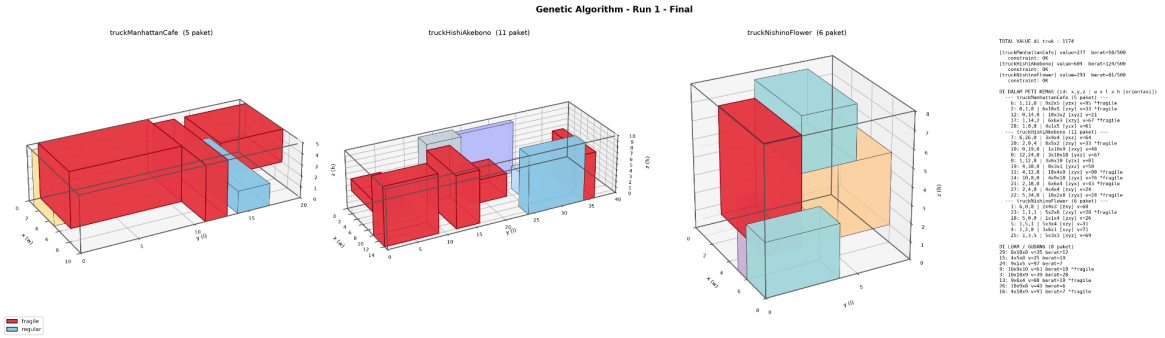
2.3.6. Genetic Algorithm

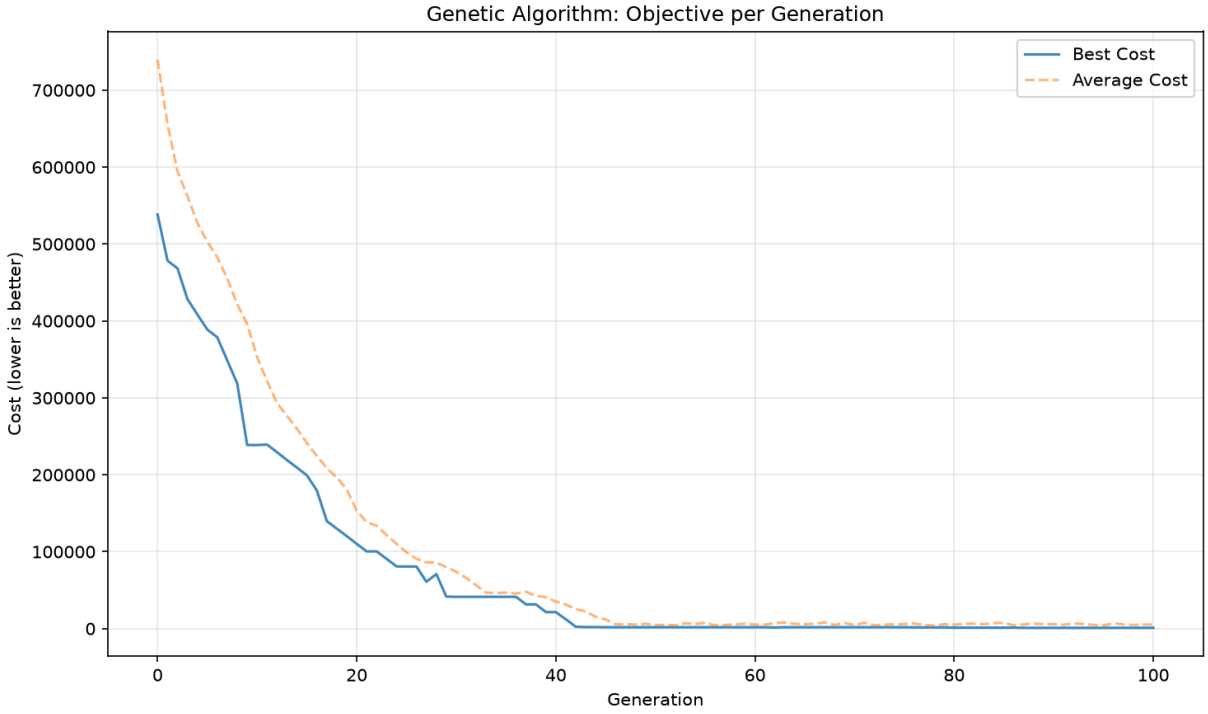
2.3.6.1. Jumlah Populasi = 100 (sebagai kontrol) dan Jumlah Iterasi = 100

2.3.6.1.1. Hasil

Run 1

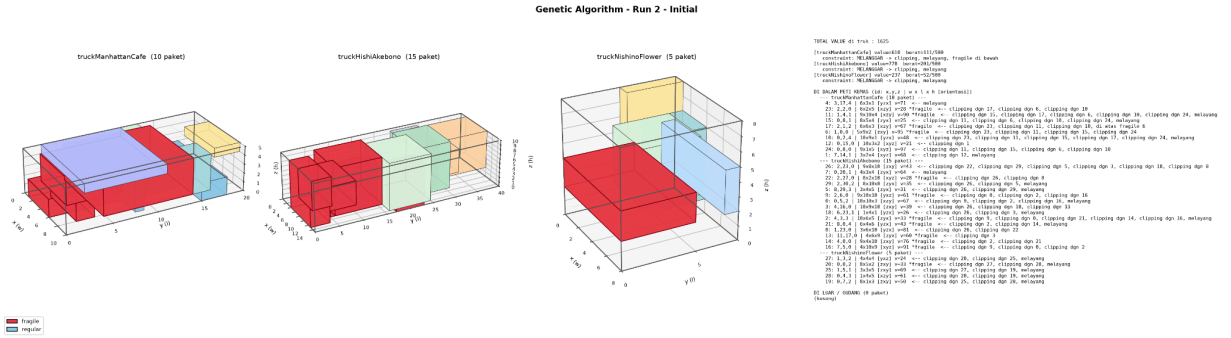


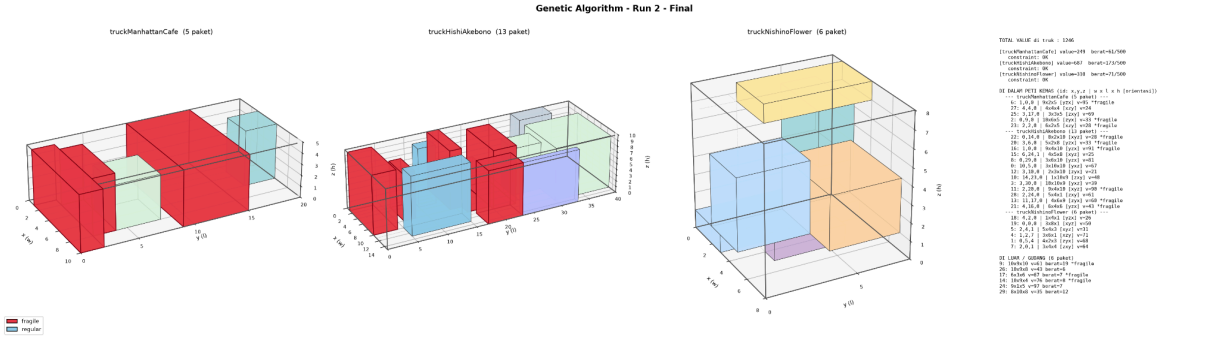


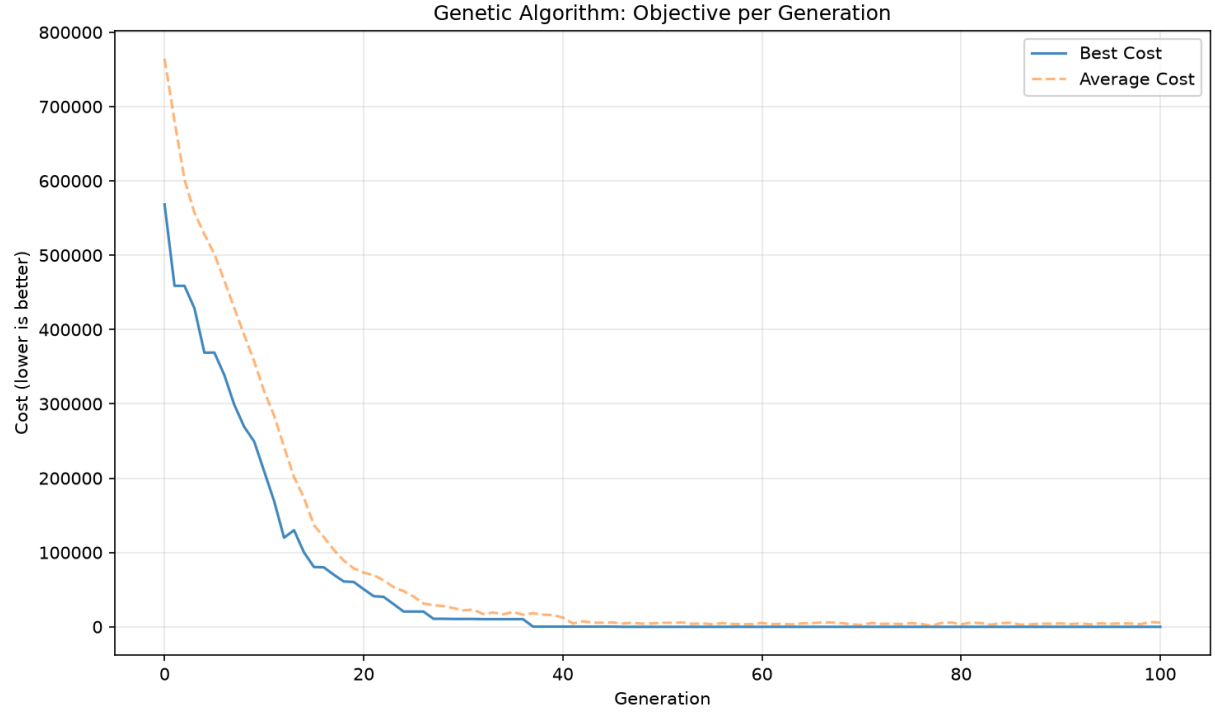


Populasi = 100
Jumlah generasi = 100
value akhir = 1174 | cost = 1006 | durasi = 133.4s

Run 2





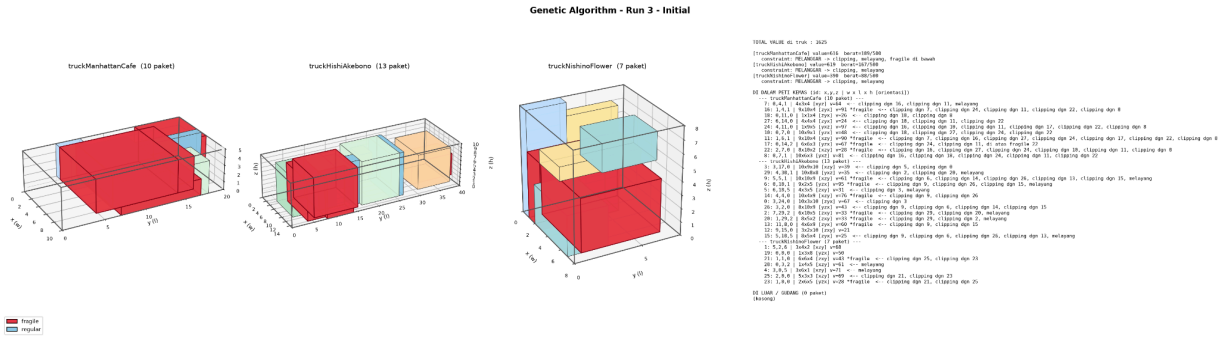


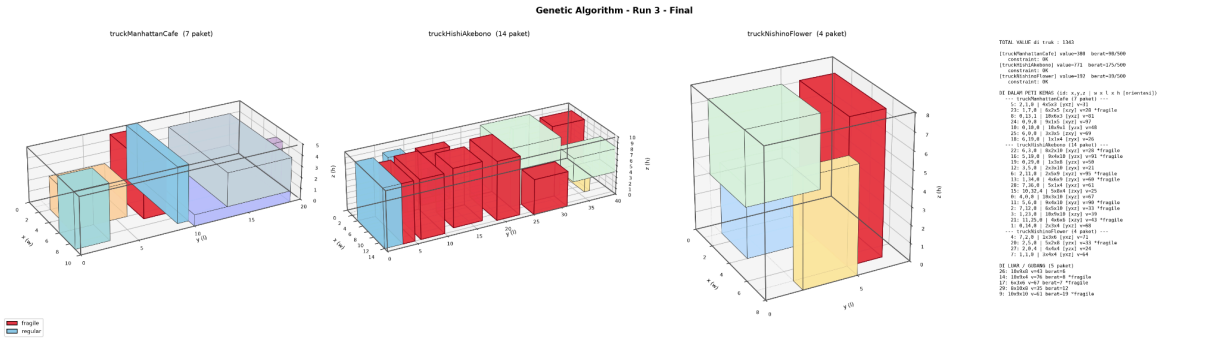
Populasi = 100

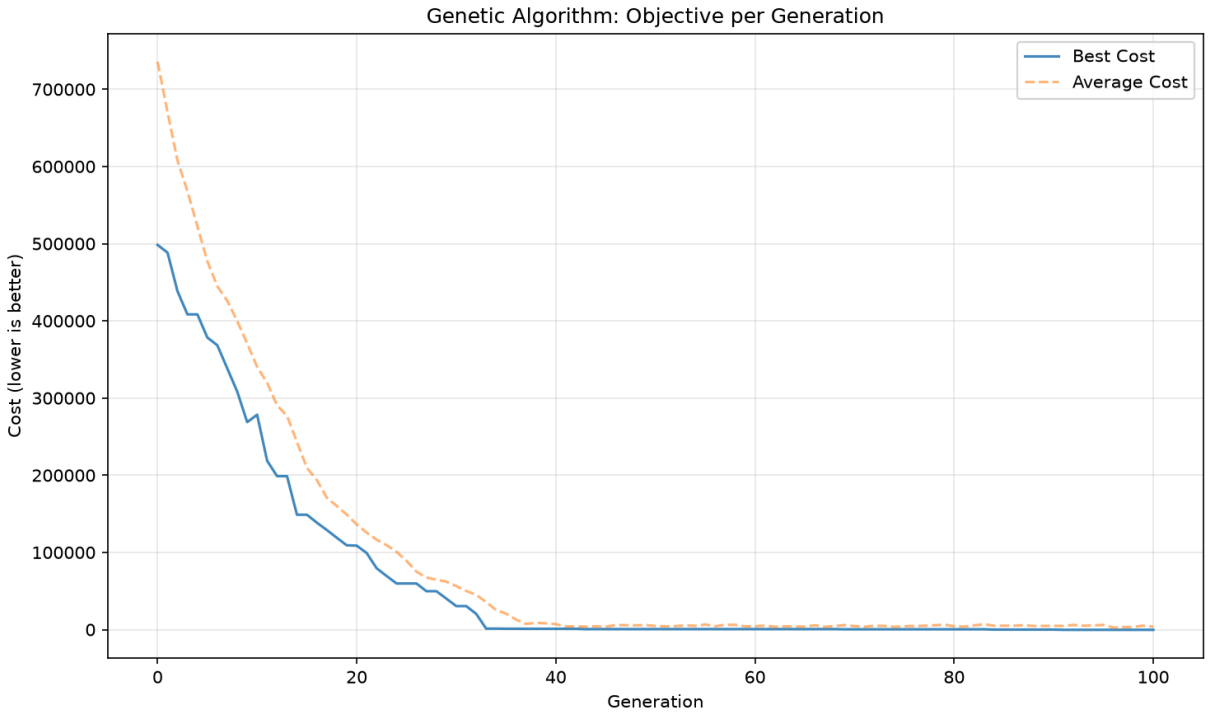
Jumlah generasi = 100

value akhir = 1246 | cost = -66 | durasi = 123.1s

Run 3







Populasi = 100

Jumlah generasi = 100

value akhir = 1343 | cost = -303 | durasi = 120.2s

2.3.6.1.2. Analisis

Berdasarkan hasil yang sudah digenerasi, diketahui info sebagai berikut:

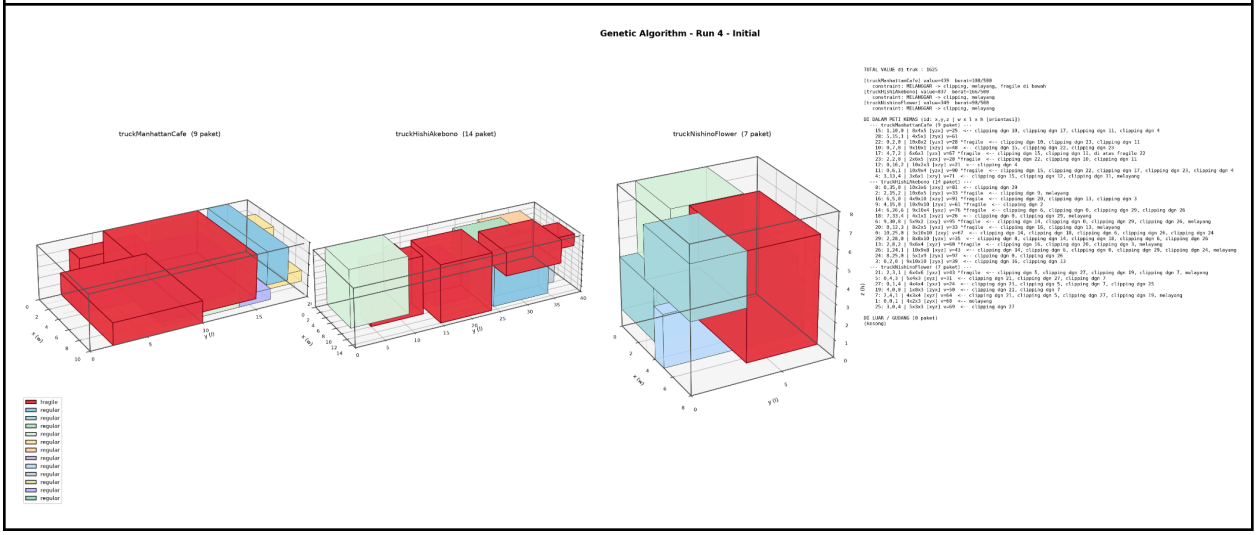
1) Run 1 menghasilkan value akhir sebanyak 1174, cost sebanyak 1006, dan memiliki durasi 133.4 detik. Tidak ada paket yang melanggar constraint.
2) Run 2 menghasilkan value akhir sebanyak 1246, cost sebanyak -66, dan memiliki durasi 123.1 detik. Tidak ada paket yang melanggar constraint.
3) Run 3 menghasilkan value akhir sebanyak 1343, cost sebanyak -303, dan memiliki durasi 120.2 detik. Tidak ada paket yang melanggar constraint.

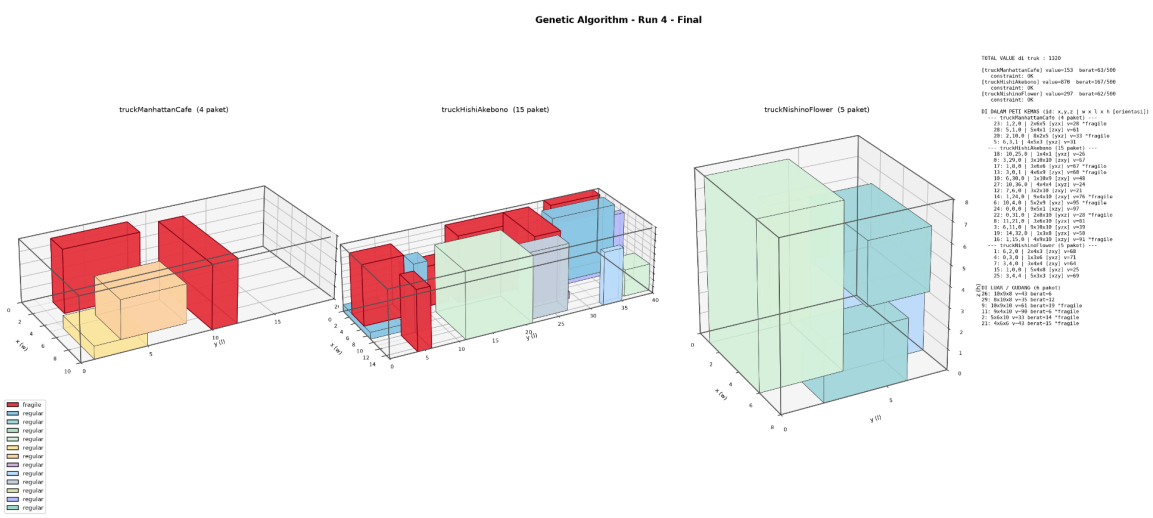
Genetic algorithm memberikan hasil value yang konsisten, tapi hasil cost yang tidak konsisten. Durasi untuk eksekusi program cukup lama.

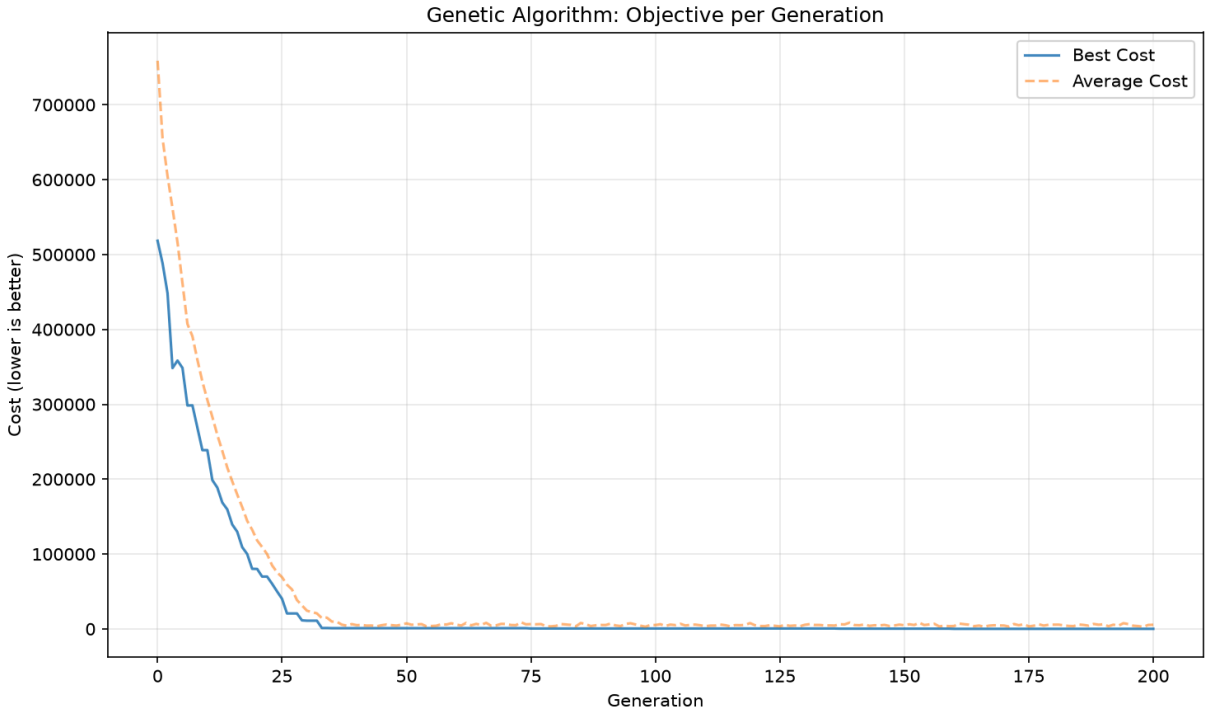
2.3.6.2. Jumlah Populasi = 100 (sebagai kontrol) dan Jumlah Iterasi = 200

2.3.6.2.1. Hasil

Run 1



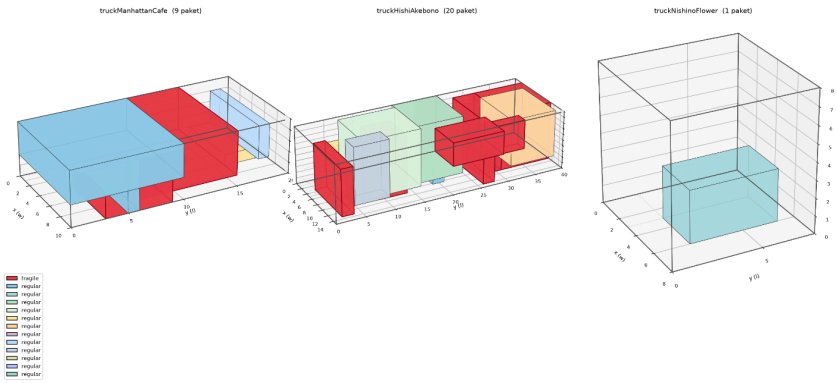




Populasi = 100
Jumlah generasi = 200
Value di truk = 1320 | Cost = 120 | Durasi = 112.4s

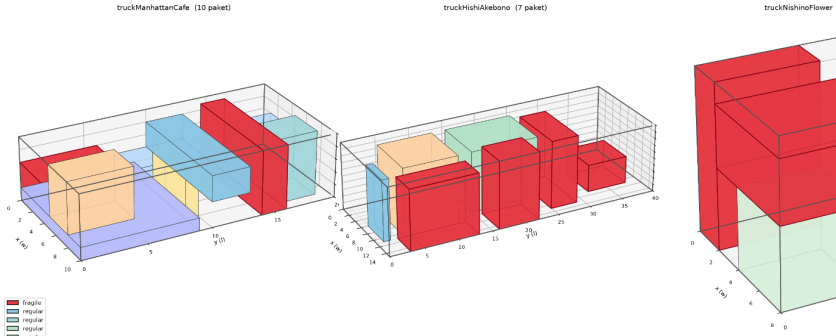
Run 2

Genetic Algorithm - Run 5 - Initial

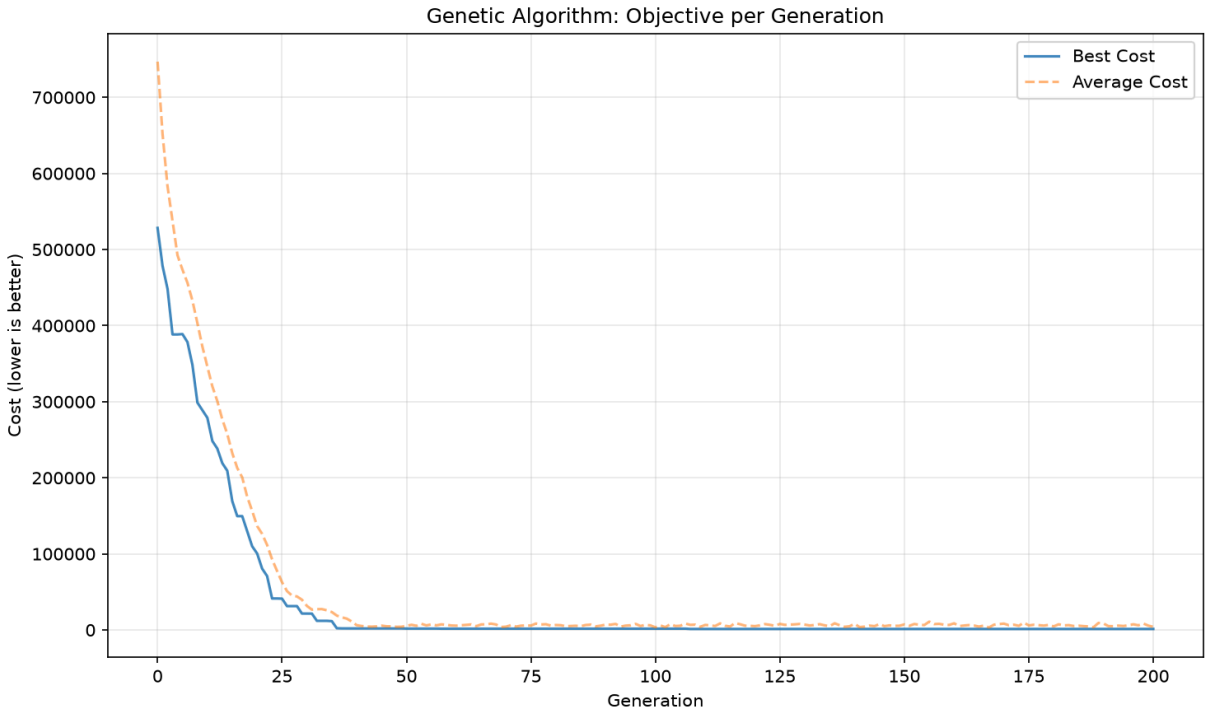



```
TOTAL VALUE di truk : 1025

[truckManhattanCafe] value=501  berat=78/500
  constraint: MELANGGAR -> clipping, melayang, fragile di bawah
[truckHishiAkebono] value=975  berat=276/500
  constraint: MELANGGAR -> clipping, melayang, fragile di bawah
[truckNishinoFlower] value=89  berat=8/500
  constraint: OK

DI DALAM PETI KEMAS (id: x,y,z | w x l x h [orientasi])
  --- truckManhattanCafe (9 paket) ---
   16: 0,0,1 | 10x5x4 [xyz] v=91 *fragile  <-- clipping dgn 0, clipping dgn 17, clipping dgn 10, clipping dgn 7
   24: 1,5,0 | 9x1x5 [yyz] v=37  <-- clipping dgn 6, clipping dgn 17, clipping dgn 10
   17: 1,6,0 | 9x4x1 [yyz] v=37  <-- clipping dgn 6
   0: 0,0,2 | 10x10x1 [xzy] v=62  <-- clipping dgn 16, clipping dgn 24, clipping dgn 17, clipping dgn 16, clipping dgn 7, melayang
   10: 1,12,0 | 4x1x1 [xyz] v=26
   17: 6,1,0 | 9x4x1 [xyz] v=67 *fragile  <-- clipping dgn 19, clipping dgn 24, clipping dgn 0, clipping dgn 7
   19: 2,17,2 | 8x1x1 [xyz] v=58
   7: 1,7,0 | 4x4x4 [yyz] v=64 *fragile  <-- clipping dgn 20, clipping dgn 24, clipping dgn 0
   7: 1,7,0 | 4x3x4 [xyz] v=64  <-- clipping dgn 16, clipping dgn 0, clipping dgn 17
  --- truckHishiAkebono (20 paket) ---
   11: 2,27,0 | 9x4x10 [xyz] v=90 *fragile  <-- clipping dgn 9, clipping dgn 27, clipping dgn 6, clipping dgn 34, clipping dgn 13
   15: 9,10,4 | 6x9x5 [xzy] v=25  <-- clipping dgn 39, clipping dgn 7, clipping dgn 23, melayang
   4: 3,0,0 | 3x3x10 [xyz] v=90 *fragile  <-- clipping dgn 11
   5: 2,30,0 | 9x10x10 [xyz] v=41 *fragile  <-- clipping dgn 27, clipping dgn 14, clipping dgn 5, clipping dgn 29
   22: 1,1,1 | 10x2x8 [yyx] v=28 *fragile  <-- melayang
   20: 11,26,4 | 2x8x5 [xyz] v=33 *fragile  <-- clipping dgn 6, clipping dgn 13, clipping dgn 29, melayang
   27: 1,27,4 | 4x4x4 [xyz] v=24  <-- clipping dgn 13, clipping dgn 9, clipping dgn 6, clipping dgn 16, melayang
   3: 3,0,2 | 3x10x1 [xyz] v=62 *fragile  <-- clipping dgn 11, clipping dgn 3
   21: 9,23,4 | 6x4x4 [yyx] v=43 *fragile  <-- clipping dgn 29, melayang
   28: 9,5,3 | 4x5x1 [yyx] v=51  <-- clipping dgn 3, clipping dgn 22, melayang
   26: 6,17,4 | 9x9x10 [yyx] v=43  <-- clipping dgn 35, clipping dgn 12, clipping dgn 21
   0: 8,5,0 | 3x6x10 [yzx] v=81  <-- clipping dgn 2, clipping dgn 3
   6: 5,20,0 | 9x3x10 [yyz] v=90 *fragile  <-- clipping dgn 11, clipping dgn 26, clipping dgn 27, clipping dgn 34
   2: 0,0,0 | 10x10x3 [xyz] v=28  <-- clipping dgn 2, clipping dgn 28, clipping dgn 8, clipping dgn 23
   14: 1,27,0 | 6x9x10 [yyx] v=76 *fragile  <-- clipping dgn 11, clipping dgn 9, clipping dgn 27, clipping dgn 9, clipping dgn 29
   23: 1,7,1 | 2x6x5 [yyx] v=28 *fragile  <-- clipping dgn 15, clipping dgn 28, clipping dgn 3, melayang
   1: 8,34,1 | 2x4x2 [yyx] v=68  <-- melayang
   1: 7,30,4 | 5x6x1 [yyz] v=31  <-- clipping dgn 9, clipping dgn 29, melayang
   12: 4,20,5 | 6x6x1 [yyz] v=60 *fragile  <-- clipping dgn 11, clipping dgn 20, ci atas fragile 6
   13: 3,0,3 | 10x6x4 [xyz] v=71  <-- clipping dgn 9, clipping dgn 20, clipping dgn 34, clipping dgn 5, melayang
  --- truckNishinoFlower (1 paket) ---
   25: 3,2,0 | 3x3x3 [yyz] v=69

DI LUAR / GUDANG (0 paket)
(kosong)
```

Genetic Algorithm - Run 5 - Final

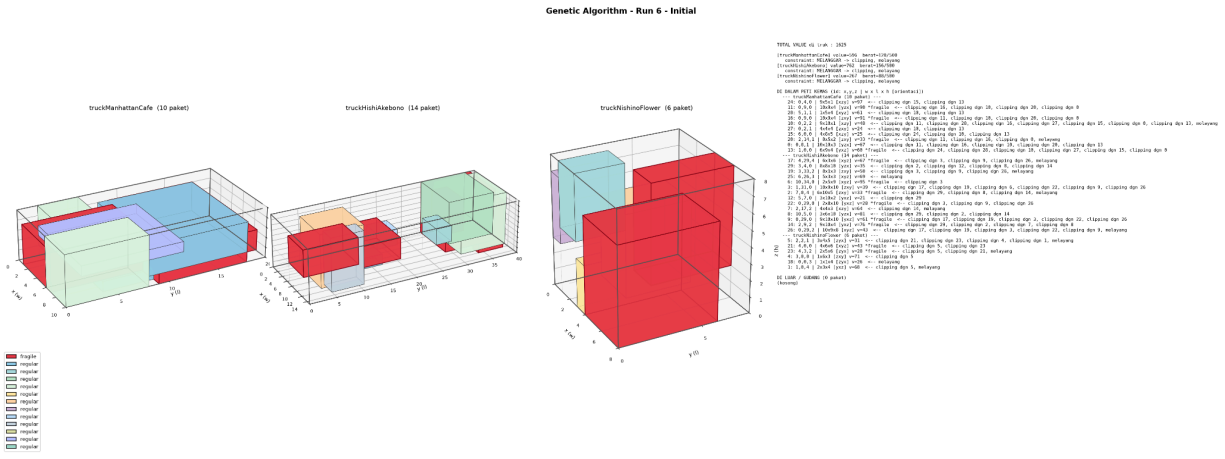


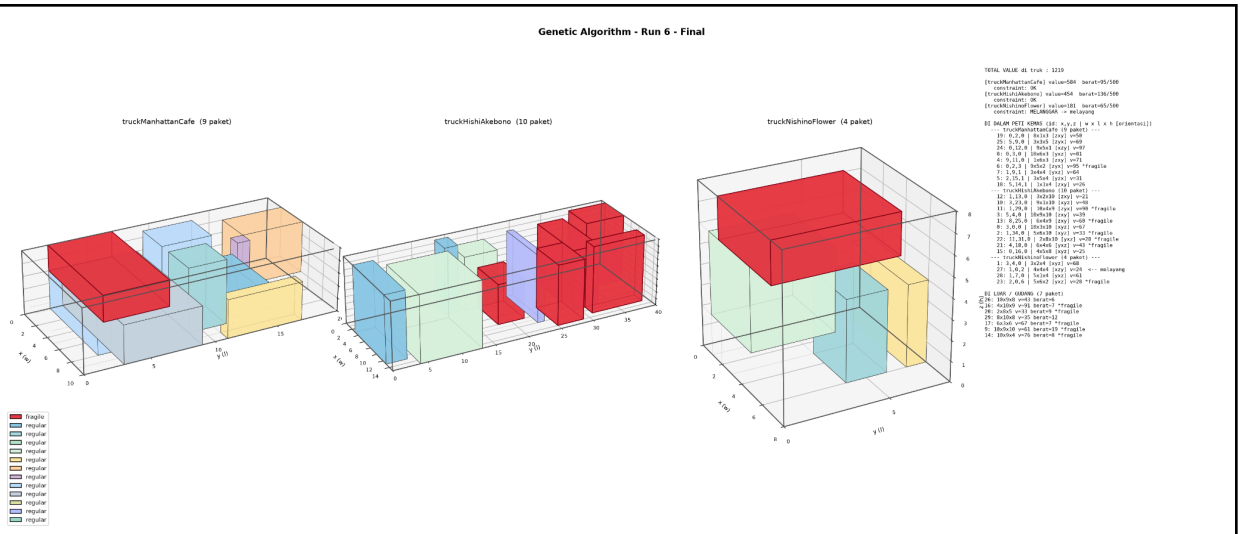
```
TOTAL VALUE di truk : 1172

[truckManhattanCafe] value=493  berat=117/500
  constraint: OK
[truckHishiAkebono] value=417  berat=68/500
  constraint: OK
[truckNishinoFlower] value=263  berat=54/500
  constraint: OK

DI DALAM PETI KEMAS (id: x,y,z | w x l x h [orientasi])
  --- truckManhattanCafe (10 paket) ---
   19: 0,0,1 | 9x5x4 [xyz] v=91 *fragile
   3: 0,0,0 | 10x5x1 [xyz] v=28
   5: 3,0,1 | 3x3x4 [yyx] v=36
   25: 6,16,0 | 3x3x5 [xyy] v=69
   6: 1,14,0 | 9x2x5 [yzx] v=95 *fragile
   10: 0,0,6 | 10x5x1 [yyx] v=68
   7: 1,16,0 | 4x4x3 [yzx] v=64
   28: 3,9,0 | 5x1x4 [yyz] v=61
   18: 0,12,0 | 4x1x1 [xyz] v=26
   8: 1,7,0 | 4x3x4 [xyz] v=201
  --- truckHishiAkebono (7 paket) ---
   11: 4,25,0 | 9x4x10 [xyz] v=90 *fragile
   2: 10,16,0 | 5x4x10 [xyz] v=51 *fragile
   14: 11,3,0 | 4x10x9 [yyx] v=76 *fragile
   24: 6,1,0 | 3x1x9 [zyx] v=97
   29: 3,4,0 | 8x8x10 [xzy] v=35
   4: 3,0,0 | 3x3x10 [xyz] v=93
   21: 5,37,0 | 6x6x4 [yyz] v=43 *fragile
  --- truckNishinoFlower (5 paket) ---
   20: 0,0,0 | 2x5x8 [xyz] v=33 *fragile
   27: 4,0,0 | 4x4x4 [yzx] v=24
   1: 2,6,0 | 4x2x3 [zyx] v=68
   4: 0,7,0 | 3x1x6 [xyz] v=71
   17: 2,6,4 | 6x6x1 [xyy] v=67 *fragile

DI LUAR / GUDANG (0 paket)
(kosong)
```

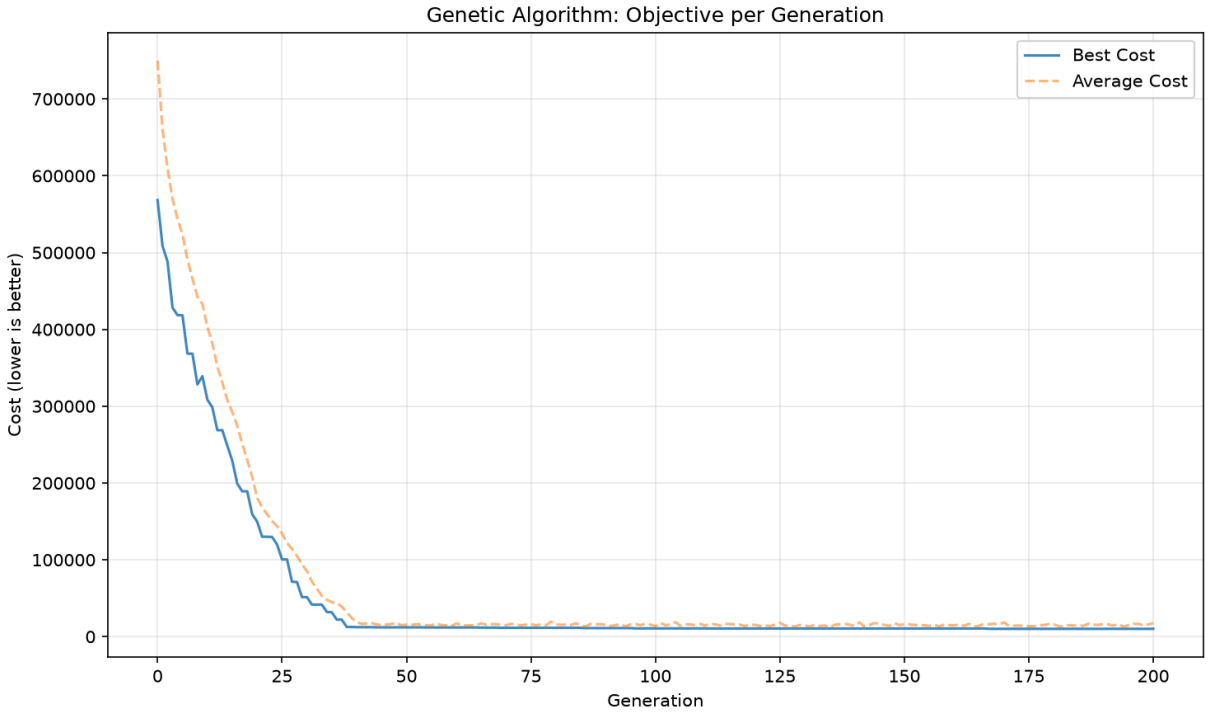


Populasi = 100
Jumlah generasi = 200
Value di truk = 1173 | Cost = 1327 | Durasi = 99.1s

Run 3







Populasi = 100
Jumlah generasi = 200
Value di truk = 1219 | Cost = 10141 | Durasi = 97.5s

2.3.6.2.2. Analisis

Berdasarkan hasil yang sudah digenerasi, diketahui info sebagai berikut:

1) Run 1 menghasilkan value akhir sebanyak 1320, cost sebanyak 120, dan memiliki durasi 112.4 detik. Tidak ada paket yang melanggar constraint.
2) Run 2 menghasilkan value akhir sebanyak 1173, cost sebanyak 1327, dan memiliki durasi 99.1 detik. Tidak ada paket yang melanggar constraint.

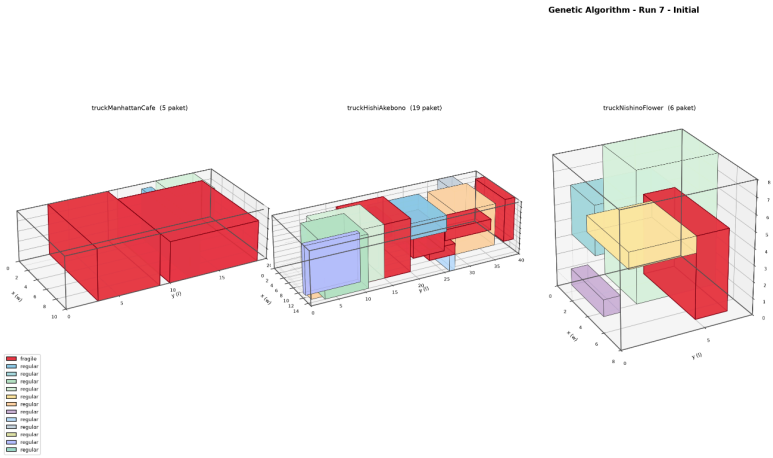
3) Run 3 menghasilkan value akhir sebanyak 1219, cost sebanyak 10141, dan memiliki durasi 97.5 detik. Tidak ada paket yang melanggar constraint.

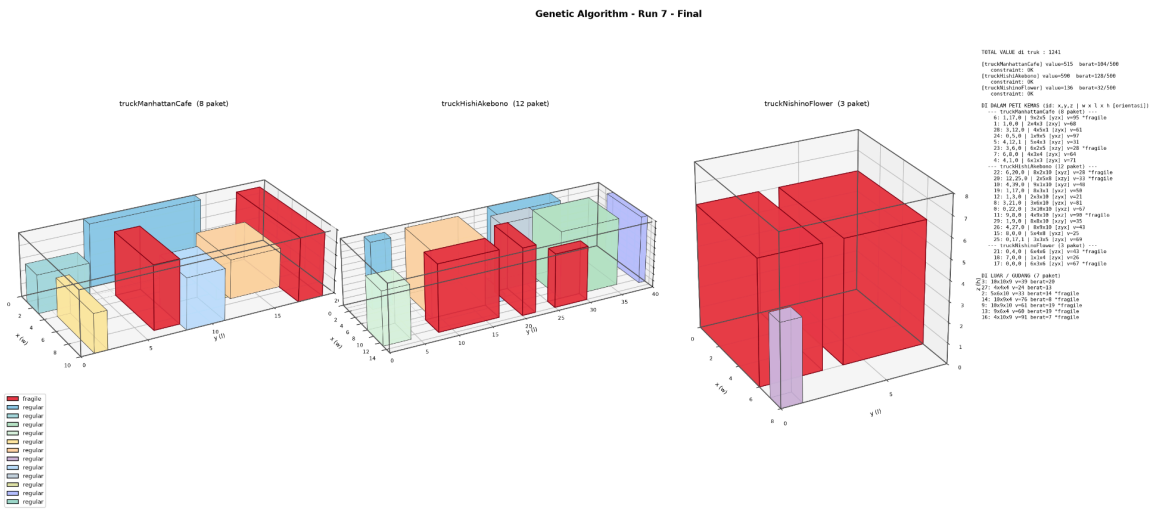
Cost yang dihasilkan cukup beragam. Namun, value yang dihasilkan cukup konsisten, sama seperti jumlah iterasi = 100. Durasi eksekusi membutuhkan sekitar 100 detik.

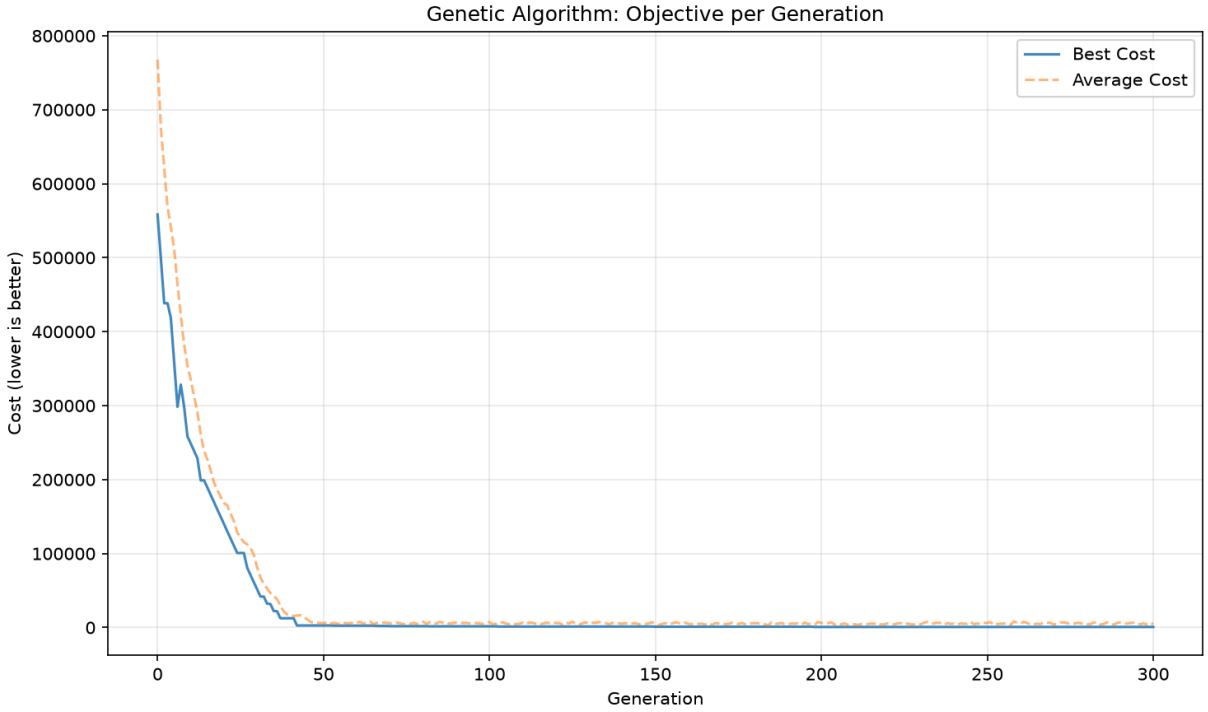
2.3.6.3. Jumlah Populasi = 100 (sebagai kontrol) dan Jumlah Iterasi = 300

2.3.6.3.1. Hasil

Run 1

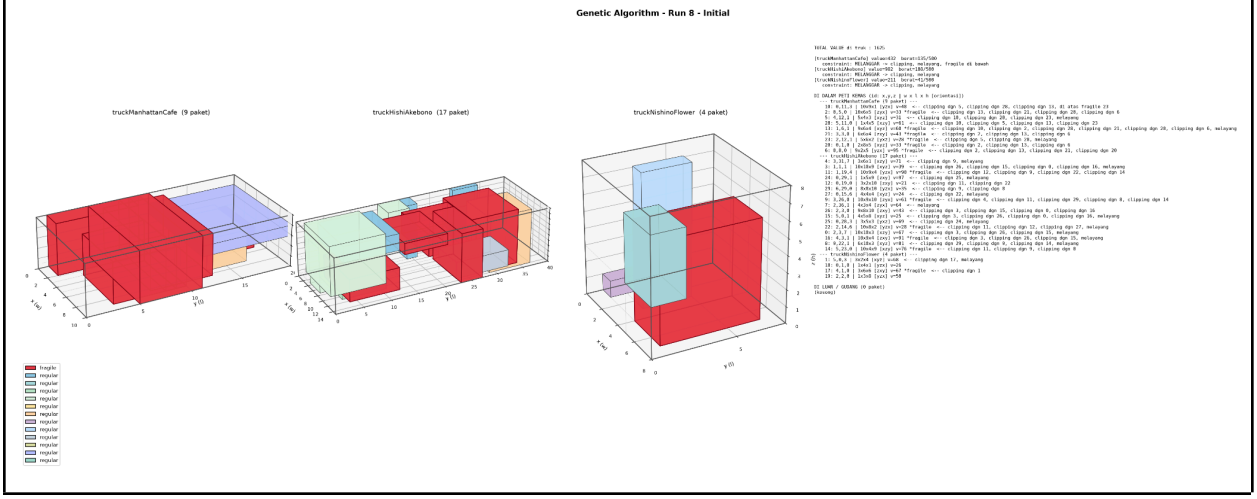


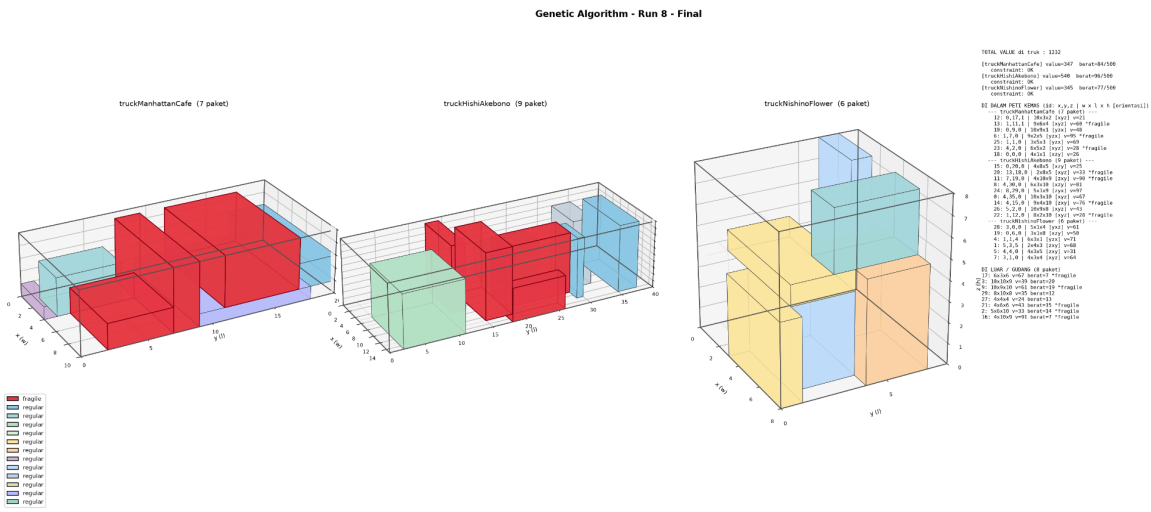


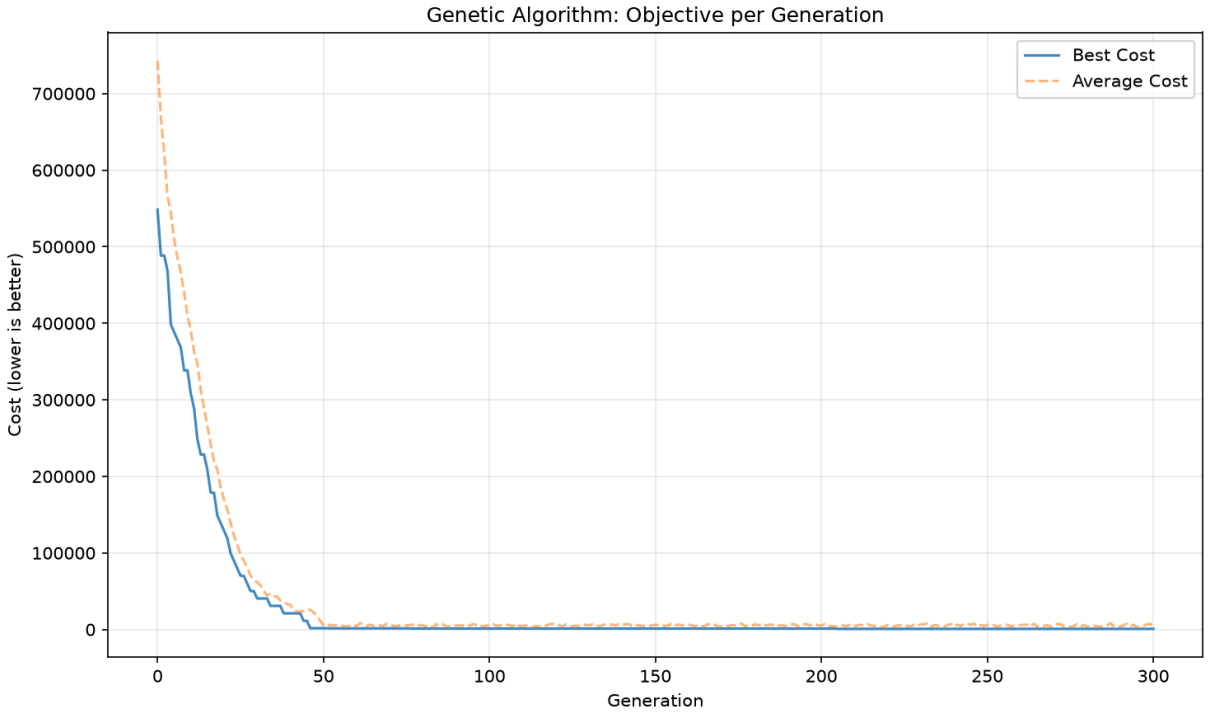


Populasi = 100
Jumlah generasi = 300
value akhir = 1241 | cost = 759 | durasi = 159.1s

Run 2



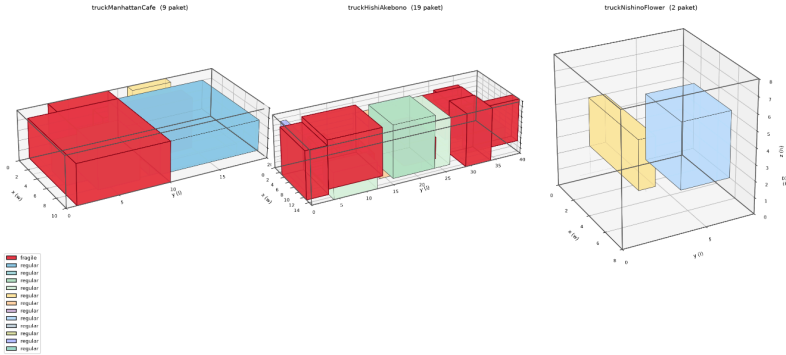




Populasi = 100
Jumlah generasi = 300
value akhir = 1232 | cost = 908 | durasi = 138.6s

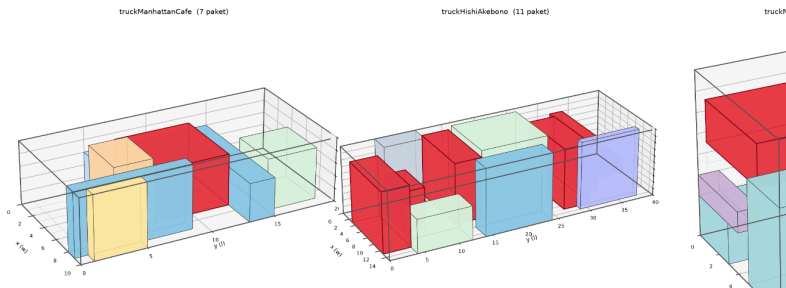
Run 3

Genetic Algorithm - Run 9 - Initial

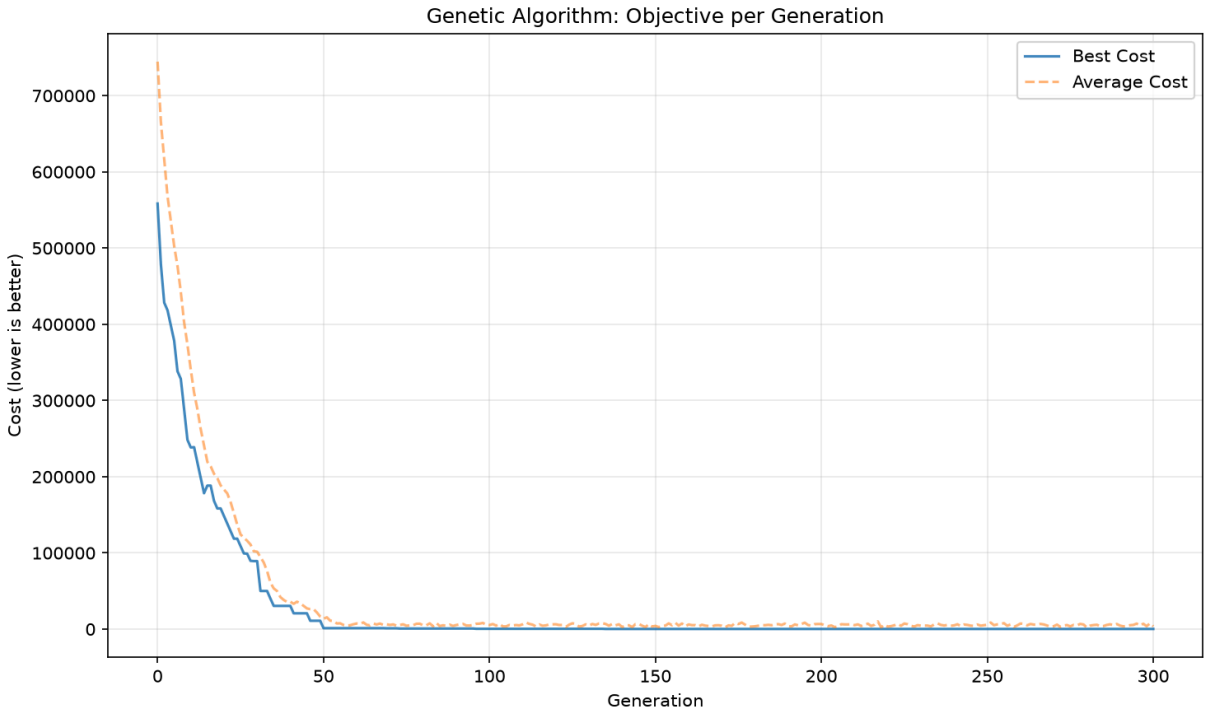



```
TOTAL VALUE dt truk : 1675

[truckManhattanCafe] value=600  berat=101/500
  constraint: OK | melayang, melayang
[truckHishiAkebono] value=850  berat=246/500
  constraint: OK | clipping, melayang
[truckNishinoFlower] value=225  berat=17/500
  constraint: MELANGGAR -> melayang

DI DALAM PETI KEMAS (id: x_y_z | w x l x h [orientasi])
  --- truckManhattanCafe (9 paket) ---
   27: 0,5,1 | 4x4x4 [xyz] v=24  <- clipping dgn 25, clipping dgn 18, clipping dgn 1, melayang
   25: 5,1,1 | 3x5x3 [yxz] v=45  <- clipping dgn 27, clipping dgn 14, clipping dgn 21, clipping dgn 1, melayang
   14: 0,1,0 | 10x4x4 [xyz] v=76 *fragile <- clipping dgn 27, clipping dgn 15, clipping dgn 21, clipping dgn 24, clipping dgn 1, clipping dgn 8, clipping dgn 6
   15: 5,1,0 | 5x4x4 [xyz] v=40 *fragile <- clipping dgn 14, clipping dgn 1, clipping dgn 8, melayang
   24: 3,5,0 | 5x5x1 [yxz] v=47  <- clipping dgn 14, clipping dgn 6
    1: 0,2,1 | 4x4x2 [xyz] v=64  <- clipping dgn 27, clipping dgn 25, clipping dgn 14, clipping dgn 21, melayang
   21: 0,1,1 | 5x4x3 [xyz] v=60  <- clipping dgn 27, clipping dgn 6, clipping dgn 20
    6: 4,4,0 | 4x4x1 [xyz] v=81 *fragile <- clipping dgn 14, clipping dgn 21, clipping dgn 24, clipping dgn 8
   20: 0,1,0 | 1x4x5 [xyz] v=47  <- clipping dgn 8
    --- truckHishiAkebono (19 paket) ---
   29: 4,10,0 | 8x8x10 [xyz] v=92  <- clipping dgn 28, clipping dgn 3, clipping dgn 5, clipping dgn 13, clipping dgn 12
   28: 0,10,0 | 4x8x10 [xyz] v=80  <- clipping dgn 3, clipping dgn 5
   22: 12,0,2 | 2x10x8 [zxy] v=38 *fragile <- clipping dgn 11, clipping dgn 2, melayang
   18: 2,12,1 | 4x1x1 [xyz] v=26  <- melayang
   26: 2,11,0 | 10x8x9 [xyz] v=36  <- clipping dgn 28, clipping dgn 3, clipping dgn 13, clipping dgn 12
   23: 0,0,0 | 4x5x2 [xyz] v=40 *fragile <- clipping dgn 11, melayang
    3: 8,10,0 | 2x8x2 [yxz] v=31  <- clipping dgn 19
   19: 8,0,1 | 6x5x2 [xyz] v=28 *fragile <- clipping dgn 9, clipping dgn 12, clipping dgn 15, melayang
    5: 3,17,0 | 10x10x9 [yxz] v=39  <- clipping dgn 28, clipping dgn 29, clipping dgn 11, clipping dgn 5, clipping dgn 13, clipping dgn 12
   11: 0,12,0 | 3x8x4 [xyz] v=91 *fragile  <- clipping dgn 22
   13: 5,10,0 | 10x5x8 [yxz] v=96 *fragile  <- clipping dgn 22, clipping dgn 29, clipping dgn 3, clipping dgn 2, melayang
    2: 5,25,2 | 4x3x5 [zxy] v=31  <- clipping dgn 26, clipping dgn 5, melayang
   17: 0,17,1 | 5x8x4 [xyz] v=80 *fragile  <- clipping dgn 29, clipping dgn 3, clipping dgn 12, melayang
    9: 3,6,1 | 10x10x5 [xyz] v=81 *fragile <- clipping dgn 17, clipping dgn 23, clipping dgn 12, clipping dgn 15, melayang
   12: 0,15,0 | 4x4x4 [yxz] v=24  <- clipping dgn 17, clipping dgn 23
   27: 7,15,4 | 2x10x5 [yzx] v=71  <- clipping dgn 29, clipping dgn 26, clipping dgn 23, clipping dgn 5, clipping dgn 13, clipping dgn 9, melayang
   15: 11,4,0 | 4x8x5 [xzy] v=25  <- clipping dgn 23, clipping dgn 9
   16: 3,8,0 | 5x8x4 [yxz] v=91 *fragile <- clipping dgn 10
   10: 1,19,0 | 5x8x1 [yxz] v=45  <- clipping dgn 8
    --- truckNishinoFlower (2 paket) ---
    4: 0,2,2 | 6x3x1 [zyx] v=71  <- melayang
    7: 3,6,2 | 4x3x4 [xyz] v=64  <- melayang

DI LUAR / GERBONG (0 paket)
(kosong)
```

Genetic Algorithm - Run 9 - Final



```
TOTAL VALUE dl truk : 1206

[truckManhattanCafe] value=327  berat=82/500
  constraint: OK
[truckHishiAkebono] value=640  berat=142/500
  constraint: OK
[truckNishinoFlower] value=331  berat=62/500
  constraint: OK

DI DALAM PETI KEMAS (id: x,y,z | w x l x h [orientasi])
  --- truckManhattanCafe (7 paket) ---
   19: 0,5,0 | 1x8x3 [yxz] v=58
   24: 8,0,0 | 1x9x5 [yxz] v=97
   11: 1,7,0 | 6x4x5 [yxz] v=93 *fragile
   21: 5,0,0 | 4x4x5 [yxz] v=81
   20: 9,1,0 | 1x4x5 [yxz] v=61
   12: 0,13,0 | 2x2x3 [xzy] v=21
   27: 5,15,0 | 4x4x4 [yxz] v=24
  --- truckHishiAkebono (11 paket) ---
   11: 0,12,0 | 3Bx4x9 [yxz] v=98 *fragile
   10: 13,29,0 | 1x9x10 [yxz] v=48
   22: 0,32,0 | 2Bx2x8 [zyx] v=28 *fragile
   18: 2,32,1 | 4x1x1 [yxz] v=26 *fragile
    8: 12,14,0 | 3x10x10 [yxz] v=67
   16: 3,0,0 | 3Bx4x9 [yxz] v=91 *fragile
   23: 2,34,0 | 1x6x2 [yxz] v=20 *fragile
    0: 0,5,0 | 1x8x2B [yzx] v=81
    3: 1,36,0 | 9x3Bx10 [yxz] v=39
   15: 11,4,0 | 4x8x5 [xzy] v=25
   17: 0,17,1 | 5x8x4 [xyz] v=80 *fragile
  --- truckNishinoFlower (6 paket) ---
   10: 0,0,2 | 4x1x1 [xzy] v=26
   25: 5,0,0 | 3x3x5 [xzy] v=69
   20: 1,0,5 | 5x8x2 [zyx] v=33 *fragile
    1: 0,0,0 | 6x6x2 [xzy] v=68
    4: 1,6,4 | 6x3x1 [yzx] v=71
    7: 4,5,0 | 4x3x4 [xzy] v=64

DI LUAR / GERBONG (9 paket)
- 26: 10x5x8 v=43 berat=6
- 6: 2x5x9 v=95 berat=8 *fragile
- 2: 5x8x10 v=33 berat=34 *fragile
- 9: 10x9x10 v=61 berat=19 *fragile
- 13: 5x8x4 v=66 berat=26 *fragile
- 29: 6x10x8 v=35 berat=12
```



Populasi = 100
Jumlah generasi = 300
value akhir = 1298 | cost = 262 | durasi = 160.5s

2.3.6.3.2. Analisis

Berdasarkan hasil yang sudah digenerasi, diketahui info sebagai berikut:

4) Run 1 menghasilkan value akhir sebanyak 1241, cost sebanyak 759, dan memiliki durasi 159.1 detik. Tidak ada paket yang melanggar constraint.
5) Run 2 menghasilkan value akhir sebanyak 1173, cost sebanyak 1327, dan memiliki durasi 99.1 detik. Tidak ada paket yang melanggar constraint.
6) Run 3 menghasilkan value akhir sebanyak 1219, cost sebanyak 10141, dan memiliki durasi 97.5 detik. Tidak ada paket yang melanggar constraint.

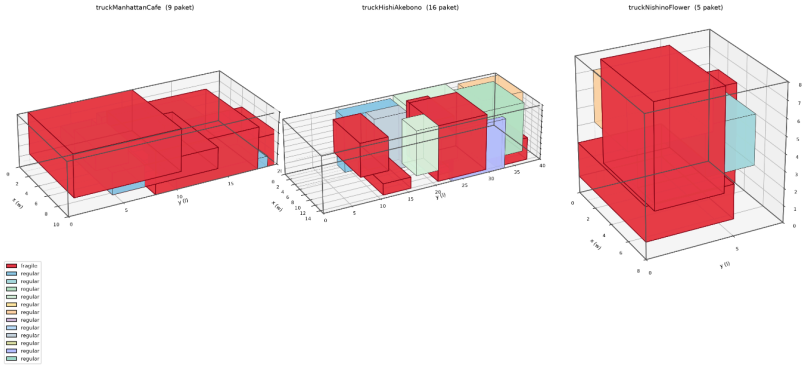
Sama seperti percobaan sebelumnya, cost yang dihasilkan cukup beragam. Namun, value yang dihasilkan lebih konsisten dibandingkan percobaan sebelumnya. Durasi eksekusi membutuhkan waktu yang lebih dari 90 detik.

2.3.6.4. Jumlah Iterasi = 150 (sebagai kontrol) dan Jumlah Populasi = 100

2.3.6.4.1. Hasil

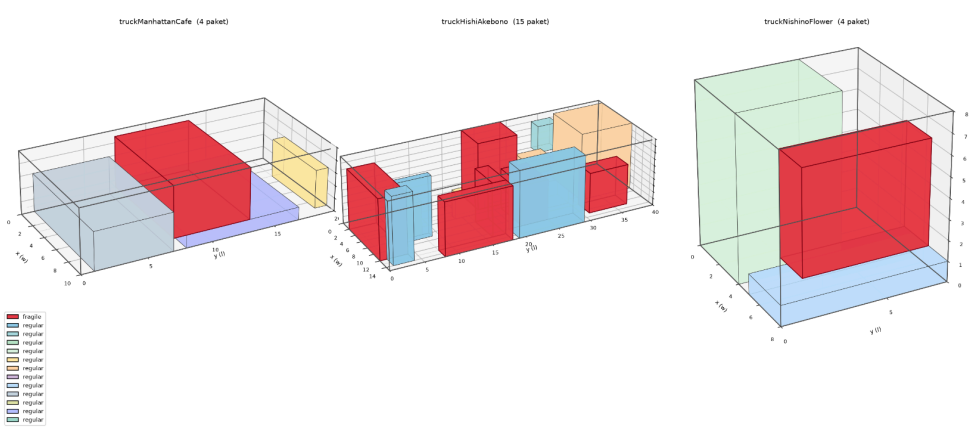
Run 1

Genetic Algorithm - Run 10 - Initial

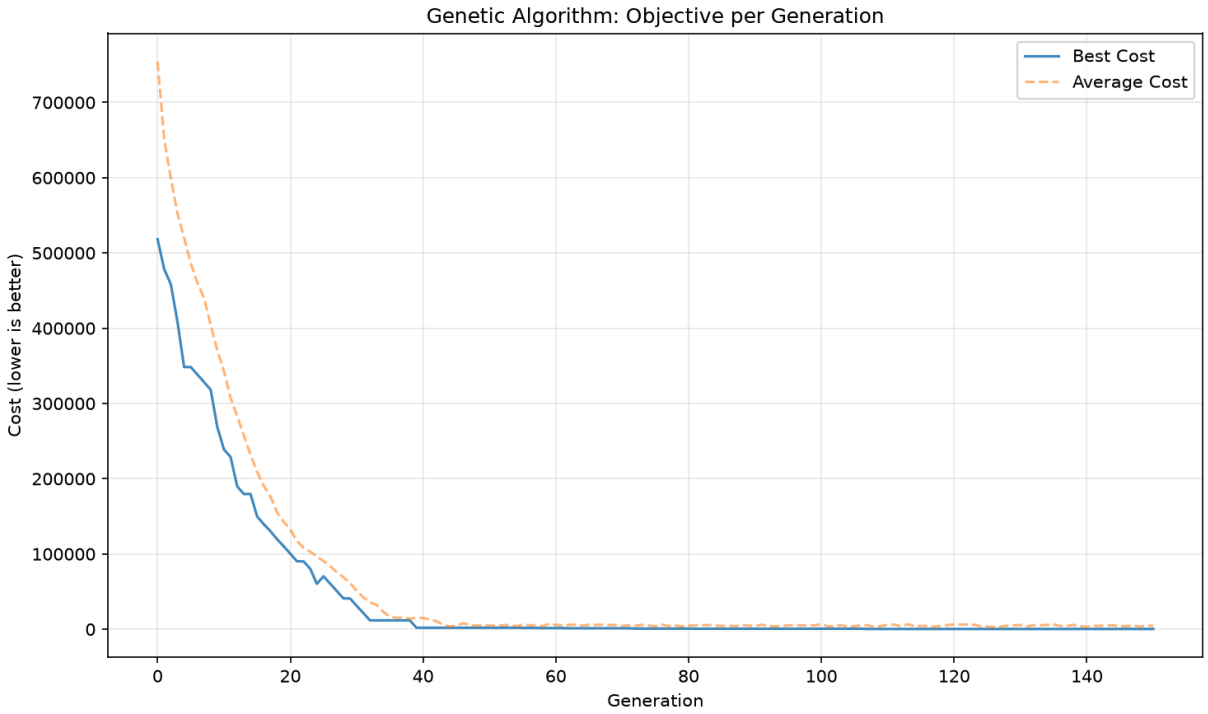



```
TOTAL VALUE di truk : 1625

[truckManhattanCafe] value=336  berat=61/500
  constraint: MELANGGAR -> clipping, melayang, fragile di bawah
[truckHishiAkebono] value=600  berat=711/500
  constraint: MELANGGAR -> clipping, melayang
[truckNishinoFlower] value=689  berat=53/500
  constraint: MELANGGAR -> clipping, melayang

DI DALAM PETI KEMAS (id: x,y,z | w x l x h [orientasi])
--- truckManhattanCafe (9 paket) ---
  13: 0,0,0 | 10x8x4 [xyz] v=480 *fragile <- clipping dgn 17, clipping dgn 16, clipping dgn 15, clipping dgn 22, clipping dgn 27, clipping dgn 24
  17: 0,10,0 | 6x4x4 [yzx] v=92 *fragile <- clipping dgn 11, clipping dgn 27, clipping dgn 24
  16: 0,3,1 | 5x10x4 [zyx] v=93 *fragile <- clipping dgn 13, clipping dgn 19, clipping dgn 1, clipping dgn 22, clipping dgn 12
  19: 5,7,1 | 5x3x4 [yzx] v=60 <- clipping dgn 12, clipping dgn 1, melayang
  1: 5,0,1 | 5x10x3 [zyx] v=150 <- clipping dgn 12, melayang
  22: 2,4,1 | 6x10x2 [xzy] v=30 *fragile <- clipping dgn 11, clipping dgn 16, clipping dgn 27, clipping dgn 17
  27: 2,11,0 | 4x4x4 [zxy] v=24 <- clipping dgn 11, clipping dgn 17, clipping dgn 22, clipping dgn 24
  12: 0,4,1 | 10x3x2 [xzy] v=23 <- clipping dgn 16, clipping dgn 22, melayang
  24: 0,10,4 | 6x5x2 [xzy] v=20 <- clipping dgn 11, clipping dgn 17, clipping dgn 27
--- truckHishiAkebono (16 paket) ---
  18: 5,23,5 | 6x1x1 [xyz] v=36 <- clipping dgn 4, clipping dgn 16, clipping dgn 9, melayang
  10: 14,23,0 | 1x10x9 [zxy] v=90 <- clipping dgn 7
  29: 1,32,0 | 8x8x10 [yzx] v=95 <- clipping dgn 15, clipping dgn 26
  15: 9,32,0 | 1x10x10 [yzx] v=100 *fragile <- clipping dgn 26, melayang
  6: 5,31,0 | 4x5x2 [zyx] v=36 *fragile <- clipping dgn 26, clipping dgn 8
  2: 0,39,0 | 9x10x10 [zyx] v=390 <- clipping dgn 18, clipping dgn 6, clipping dgn 14, clipping dgn 9
  4: 5,39,0 | 1x8x5 [zxy] v=71 <- clipping dgn 6, clipping dgn 14, clipping dgn 9
  13: 5,32,0 | 9x8x4 [xyz] v=69 *fragile <- clipping dgn 29, clipping dgn 26
  14: 0,32,0 | 5x8x4 [xyz] v=160 *fragile <- clipping dgn 10, clipping dgn 9, clipping dgn 4, clipping dgn 9, clipping dgn 7
  9: 1,9,2 | 9x10x10 [yxz] v=67 <- clipping dgn 1
  11: 9,17,2 | 5x4x8 [yxz] v=25 <- melayang
  28: 5,14,0 | 1x5x4 [xyz] v=63 <- clipping dgn 0, clipping dgn 8
  8: 4,21,0 | 10x9x10 [xyz] v=81 *fragile <- clipping dgn 3, clipping dgn 4, clipping dgn 14, clipping dgn 7
  3: 0,21,0 | 4x6x8 [yzx] v=192 *fragile <- clipping dgn 14, clipping dgn 9
  7: 11,21,0 | 4x3x4 [xyz] v=84 <- clipping dgn 30, clipping dgn 14, clipping dgn 9, melayang
  26: 1,31,0 | 10x8x9 [xzy] v=42 <- clipping dgn 29, clipping dgn 12
--- truckNishinoFlower (5 paket) ---
  20: 0,0,0 | 6x5x3 [yzx] v=90 *fragile <- clipping dgn 23, melayang
  5: 0,1,4 | 4xxx2 [yxz] v=45 <- clipping dgn 23, clipping dgn 21, clipping dgn 25, melayang
  23: 1,5,0 | 2x5x4 [zyx] v=39 *fragile <- clipping dgn 5, clipping dgn 21, clipping dgn 25
  21: 1,1,2 | 6x4x6 [yxz] v=63 *fragile <- clipping dgn 20, clipping dgn 5, clipping dgn 23, melayang
  25: 0,5,2 | 5x3x3 [xyz] v=90 <- clipping dgn 5, clipping dgn 23, melayang

DI LUAR / GUDANG (0 paket)
(kosong)
```

Genetic Algorithm - Run 10 - Final

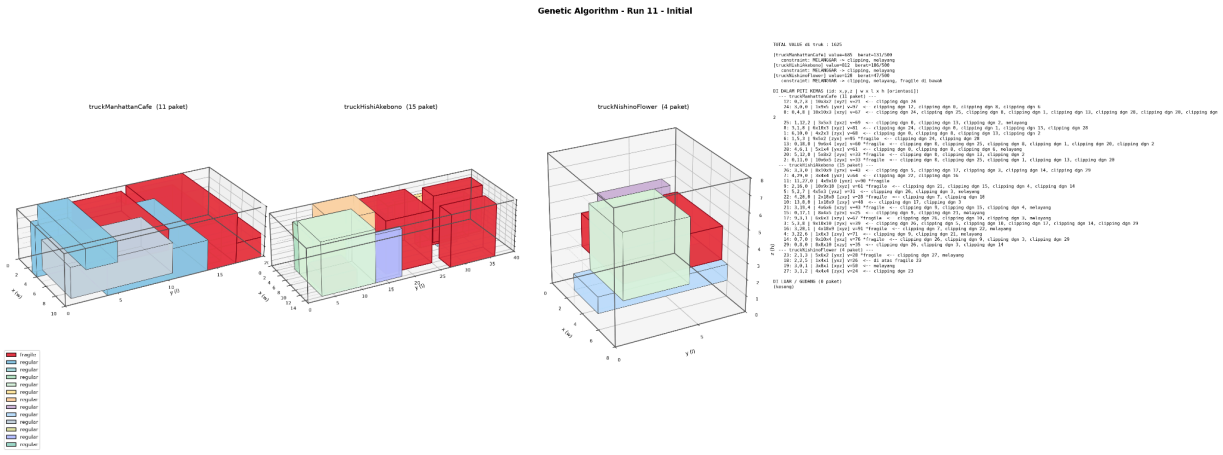


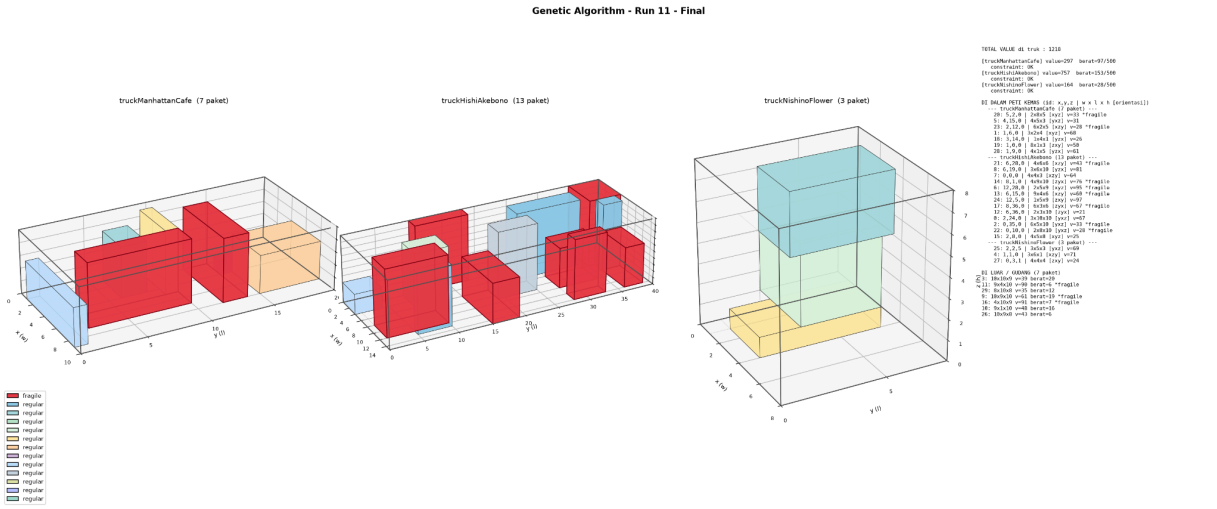

```
TOTAL VALUE di truk : 1197

[truckManhattanCafe] value=260  berat=61/500
  constraint: OK
[truckHishiAkebono] value=660  berat=170/500
  constraint: OK
[truckNishinoFlower] value=129  berat=53/500
  constraint: OK

DI DALAM PETI KEMAS (id: x,y,z | w x l x h [orientasi])
--- truckManhattanCafe (4 paket) ---
  13: 1,7,1 | 9x8x4 [xyz] v=60 *fragile
  4: 3,19,0 | 6x1x1 [zyx] v=71
  10: 0,0,0 | 10x9x1 [yzx] v=60
  8: 0,1,0 | 10x6x3 [yzx] v=81
--- truckHishiAkebono (15 paket) ---
  28: 5,14,0 | 1x5x4 [xyz] v=63
  22: 2,0,0 | 2x10x8 [zxy] v=20 *fragile
  2: 0,39,0 | 9x10x10 [zyx] v=390
  16: 2,6,0 | 10x4x8 [yxz] v=91 *fragile
  17: 10,31,0 | 3x6x6 [yxz] v=67 *fragile
  24: 0,4,0 | 1x1x9 [zxy] v=87
  21: 0,21,0 | 6x4x6 [yzx] v=61 *fragile
  3: 0,10,0 | 4x6x10 [xyz] v=33 *fragile
  12: 13,1,0 | 2x3x10 [zyx] v=23
  11: 9,17,2 | 5x4x8 [yxz] v=25
  9: 12,19,0 | 3x10x10 [yxz] v=0/
  7: 0,36,0 | 3x4x4 [zyx] v=84
  5: 1,36,0 | 3x4x5 [zyx] v=31
  20: 0,16,0 | 5x2x8 [yzx] v=33 *fragile
  25: 5,5,0 | 3x3x5 [zxy] v=69
--- truckNishinoFlower (4 paket) ---
  23: 1,5,0 | 2x5x4 [zyx] v=39 *fragile
  15: 6,1,1 | 2x6x5 [yxz] v=28 *fragile
  19: 5,0,0 | 5x8x1 [xyz] v=50
  15: 0,0,0 | 4x5x8 [xyz] v=25

DI LUAR / GUDANG (7 paket)
27: 4x4x4 v=24 berat=11
26: 1x10x9 v=42 berat=12
14: 10x5x6 v=76 berat=6 *fragile
6: 10x9x10 v=61 berat=19 *fragile
1: 5x4x10 v=90 berat=6 *fragile
18: 2x5x5 v=95 berat=8 *fragile
3: 10x10x9 v=30 berat=20
```

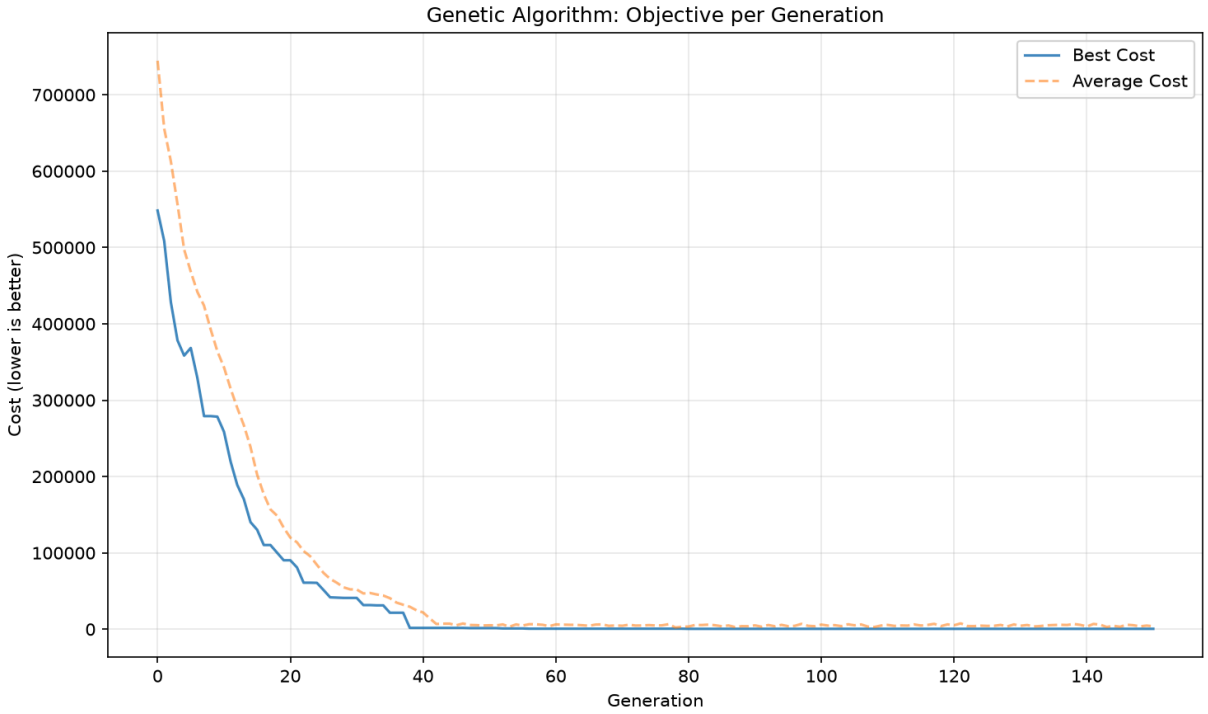


Populasi = 100
Jumlah generasi = 150
value akhir = 1197 | cost = 403 | durasi = 83.1s

Run 2



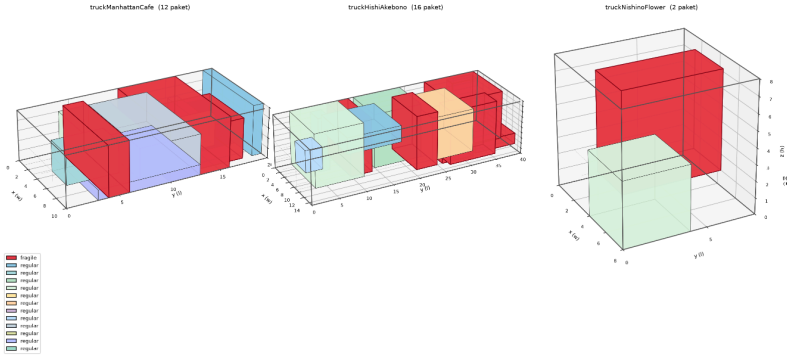




Populasi = 100
Jumlah generasi = 150
value akhir = 1218 | cost = 502 | durasi = 85.1s

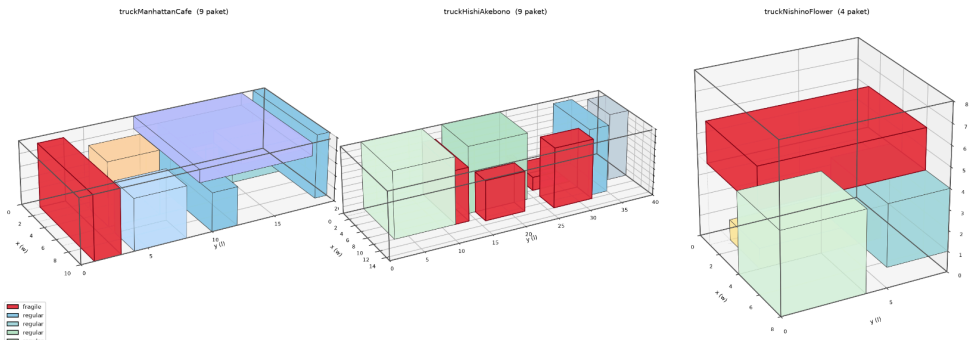
Run 3

Genetic Algorithm - Run 12 - Initial

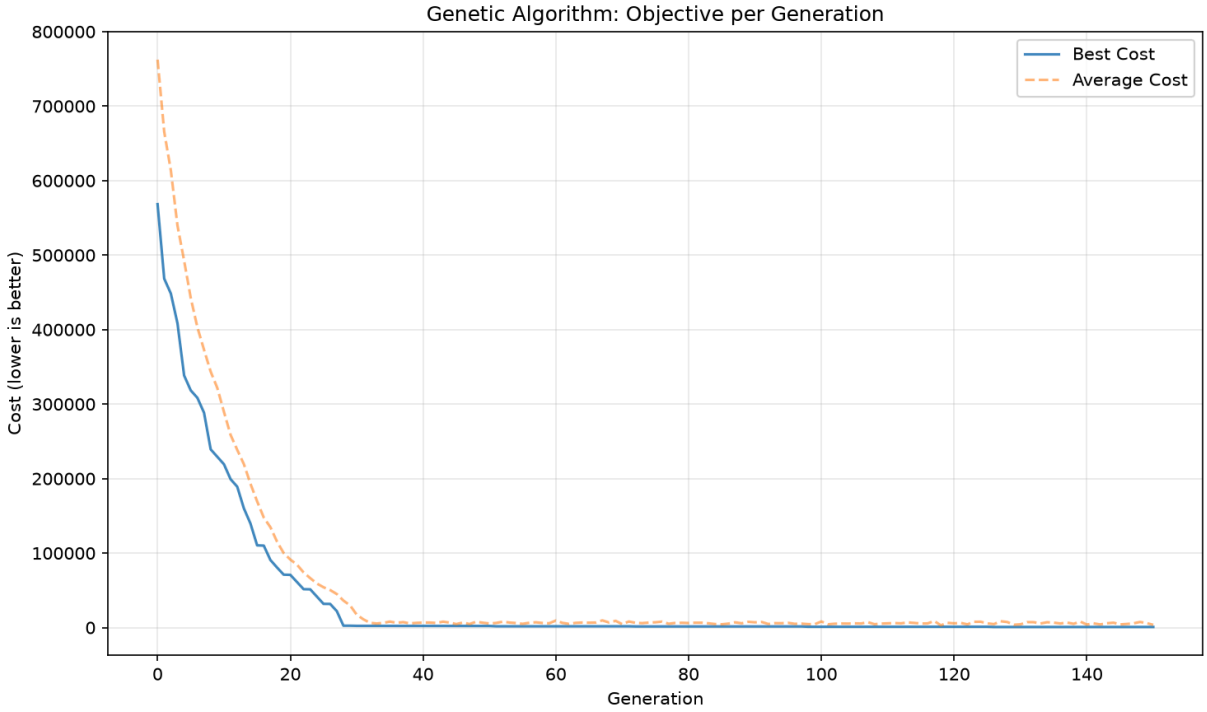



```
TOTAL VALUE di truk : 1625

[truckManhattanCafe] value=741  berat=153/500
  constraint: MELANGGAR → clipping, melayang
[truckHishiAkebono] value=756  berat=391/500
  constraint: MELANGGAR → clipping, melayang
[truckNishinoFlower] value=128  berat=37/500
  constraint: MELANGGAR → clipping, melayang

DI DALAM PETI KONG (id: x,y,z | w x l x h (orientasi))
  --- truckManhattanCafe (12 paket) ---
   25: 0,4,0 | 8x5x4 [yxz] v=25  <-- clipping dgn 25, clipping dgn 10, clipping dgn 0, clipping dgn 6
   24: 0,29,0 | 8x2x4 [xyz] v=97
   4: 8,9,2 | 1x6x3 [zxy] v=73  <-- clipping dgn 2, clipping dgn 9, clipping dgn 33, melayang
   6: 8,0,0 | 1x9x2 [yxz] v=73  <-- clipping dgn 4
   20: 4,13,0 | 9x1x5 [xyz] v=61  <-- clipping dgn 35, clipping dgn 2, clipping dgn 21
   1: 2,35,0 | 2x5x3 [zxy] v=60  <-- clipping dgn 39, clipping dgn 2, clipping dgn 21
   3: 0,0,0 | 2x2x10 [xyz] v=57  <-- clipping dgn 4, clipping dgn 20, clipping dgn 2, clipping dgn 21
   18: 1,1,0 | 9x10x1 [xyz] v=48  <-- clipping dgn 35, clipping dgn 19, clipping dgn 2, clipping dgn 21, clipping dgn 6
   2: 8,38,0 | 10x6x5 [yxz] v=33 *fragile <-- clipping dgn 4, clipping dgn 20, clipping dgn 1, clipping dgn 19, clipping dgn 30, clipping dgn 0, clipping dgn 21
   0: 8,7,1 | 10x8x3 [yxz] v=81  <-- clipping dgn 35, clipping dgn 4, clipping dgn 2, clipping dgn 21
   21: 3,12,0 | 6x6x4 [yxz] v=43 *fragile <-- clipping dgn 4, clipping dgn 20, clipping dgn 1, clipping dgn 19, clipping dgn 30, clipping dgn 2, clipping dgn 0
   23: 0,0,0 | 4x4x10 [xyz] v=41 *fragile <-- clipping dgn 35, clipping dgn 25, clipping dgn 38
  --- truckHishiAkebono (16 paket) ---
   11: 0,18,0 | 9x4x10 [xyz] v=90 *fragile <-- clipping dgn 3, clipping dgn 0
   38: 2,29,0 | 1x5x8 [zxy] v=39  <-- clipping dgn 21
   19: 0,27,0 | 3x5x8 [zxy] v=36  <-- clipping dgn 9, melayang
   10: 9,27,0 | 4x9x8 [xyz] v=36 *fragile <-- clipping dgn 9, clipping dgn 22, clipping dgn 5, clipping dgn 11, clipping dgn 29
   9: 8,35,1 | 10x3x9 [xzy] v=61 *fragile <-- clipping dgn 29, clipping dgn 14, clipping dgn 22, clipping dgn 5, clipping dgn 13, clipping dgn 29, melayang
   23: 3,21,5 | 6x3x5 [xzy] v=28 *fragile <-- clipping dgn 35, clipping dgn 26, melayang
   8: 0,0,0 | 10x10x2 [xyz] v=24  <-- clipping dgn 3
   33: 3,23,0 | 9x4x10 [yzx] v=81 *fragile <-- clipping dgn 25, clipping dgn 29, clipping dgn 26
   12: 2,0,0 | 2x2x18 [zxy] v=23  <-- clipping dgn 3
   2: 8,3,0 | 10x9x10 [zxy] v=39  <-- clipping dgn 11, clipping dgn 12, clipping dgn 0
   22: 2,32,1 | 10x8x2 [yzx] v=28 *fragile <-- clipping dgn 14, clipping dgn 5, melayang
   29: 0,0,0 | 10x10x1 [xyz] v=10  <-- clipping dgn 35, clipping dgn 25, clipping dgn 29, melayang
   13: 0,28,8 | 4x6x8 [zxy] v=68 *fragile <-- clipping dgn 18, clipping dgn 5, clipping dgn 29
   0: 2,8,5 | 10x10x3 [xzy] v=67  <-- clipping dgn 11, clipping dgn 2, clipping dgn 26, melayang
   26: 2,25,1 | 6x9x8 [xzy] v=35  <-- clipping dgn 38, clipping dgn 24, clipping dgn 9, clipping dgn 16, clipping dgn 5, clipping dgn 13, melayang
   28: 0,0,0 | 8x5x10 [xyz] v=25  <-- clipping dgn 23, clipping dgn 39, clipping dgn 0
  --- truckNishinoFlower (2 paket) ---
   27: 4,0,0 | 6x4x4 [xyz] v=24  <-- clipping dgn 17
   17: 3,1,2 | 3x6x6 [zxy] v=87 *fragile <-- clipping dgn 27, melayang

DI LUAR / GERBONG (0 paket)
(kosong)
```

Genetic Algorithm - Run 12 - Final



```
TOTAL VALUE di truk : 1171

[truckManhattanCafe] value=535  berat=106/500
  constraint: OK
[truckHishiAkebono] value=439  berat=121/500
  constraint: OK
[truckNishinoFlower] value=197  berat=37/500
  constraint: OK

DI DALAM PETI KEMAS (id: x,y,z | w x l x h (orientasi))
  --- truckManhattanCafe (9 paket) ---
   5: 3,0,0 | 1x4x5 [zyx] v=31
   12: 0,10,0 | 3Bx2x3 [xzy] v=21
   7: 7,4,0 | 1x4x4 [yxz] v=46
   17: 0,0,0 | 2x4x3 [yxz] v=86 *fragile
   1: 2,15,0 | 2x4x3 [zxy] v=60
   19: 0,11,3 | 3x8x1 [xyz] v=50
   28: 3,14,3 | 4x5x1 [zyx] v=61
   18: 0,9,6 | 10x9x1 [yzx] v=60
   24: 0,19,0 | 9x1x5 [xyz] v=97
  --- truckNishinoAkebono (9 paket) ---
   22: 0,12,0 | 10x2x8 [zyx] v=28 *fragile
   13: 0,28,8 | 4x6x8 [zxy] v=68 *fragile
   0: 2,37,0 | 4x3x10 [xzy] v=81
   8: 1,32,0 | 10x3x10 [xyz] v=67
   11: 8,20,0 | 4x6x9 [zyx] v=60 *fragile
   26: 0,15,0 | 8x9x10 [zyx] v=43
   3: 0,3,0 | 10x9x10 [zxy] v=39
   17: 0,16,0 | 3x6x6 [yxz] v=67 *fragile
   2: 8,3,0 | 10x9x10 [zxy] v=39
  --- truckNishinoFlower (4 paket) ---
   25: 3,5,1 | 5x3x3 [zxy] v=69
   4: 1,1,0 | 3x6x1 [zxy] v=71
   27: 4,0,0 | 4x4x4 [yzx] v=24
   20: 1,0,1 | 5x8x2 [zyx] v=33 *fragile

DI LUAR / GERBONG (8 paket)
  35: 0x5x6 v=75 berat=19
  29: 0x10x9 v=35 berat=32
  16: 4x10x9 v=91 berat=7 *fragile
  21: 4x6x6 v=43 berat=35 *fragile
  2: 5x6x10 v=33 berat=14 *fragile
  31: 9x4x10 v=90 berat=6 *fragile
  14: 10x9x4 v=76 berat=8 *fragile
  9: 10x5x10 v=61 berat=19 *fragile
```



Populasi = 100
Jumlah generasi = 150
value akhir = 1171 | cost = 829 | durasi = 67.6s

2.3.6.4.2. Analisis

Berdasarkan hasil yang sudah digenerasi, diketahui info sebagai berikut:

7) Run 1 menghasilkan value akhir sebanyak 1197, cost sebanyak 403, dan memiliki durasi 83.1 detik. Tidak ada paket yang melanggar constraint.
8) Run 2 menghasilkan value akhir sebanyak 1218, cost sebanyak 502, dan memiliki durasi 85.1 detik. Tidak ada paket yang melanggar constraint.
9) Run 3 menghasilkan value akhir sebanyak 1171, cost sebanyak 829, dan memiliki durasi 67.6 detik. Tidak ada paket yang melanggar constraint.

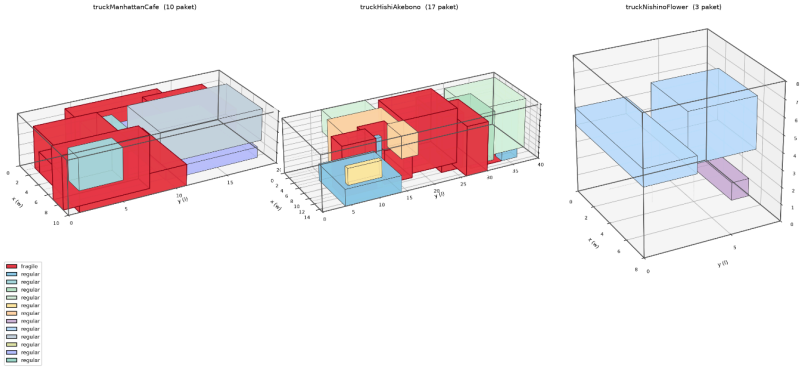
Cost yang dihasilkan cukup beragam. Namun, value yang dihasilkan lebih konsisten dibandingkan percobaan sebelumnya. Durasi eksekusi membutuhkan waktu sekitar 70 hingga 80 detik.

2.3.6.5. Jumlah Iterasi = 150 (sebagai kontrol) dan Jumlah Populasi = 200

2.3.6.5.1. Hasil

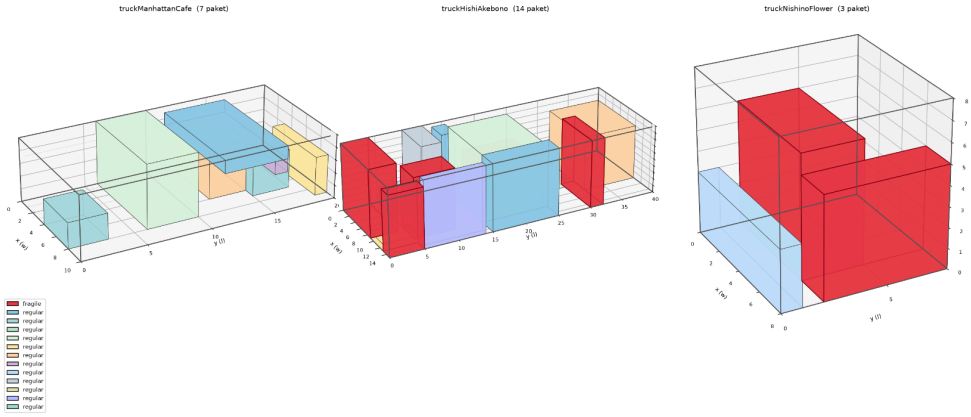
Run 1

Genetic Algorithm - Run 13 - Initial

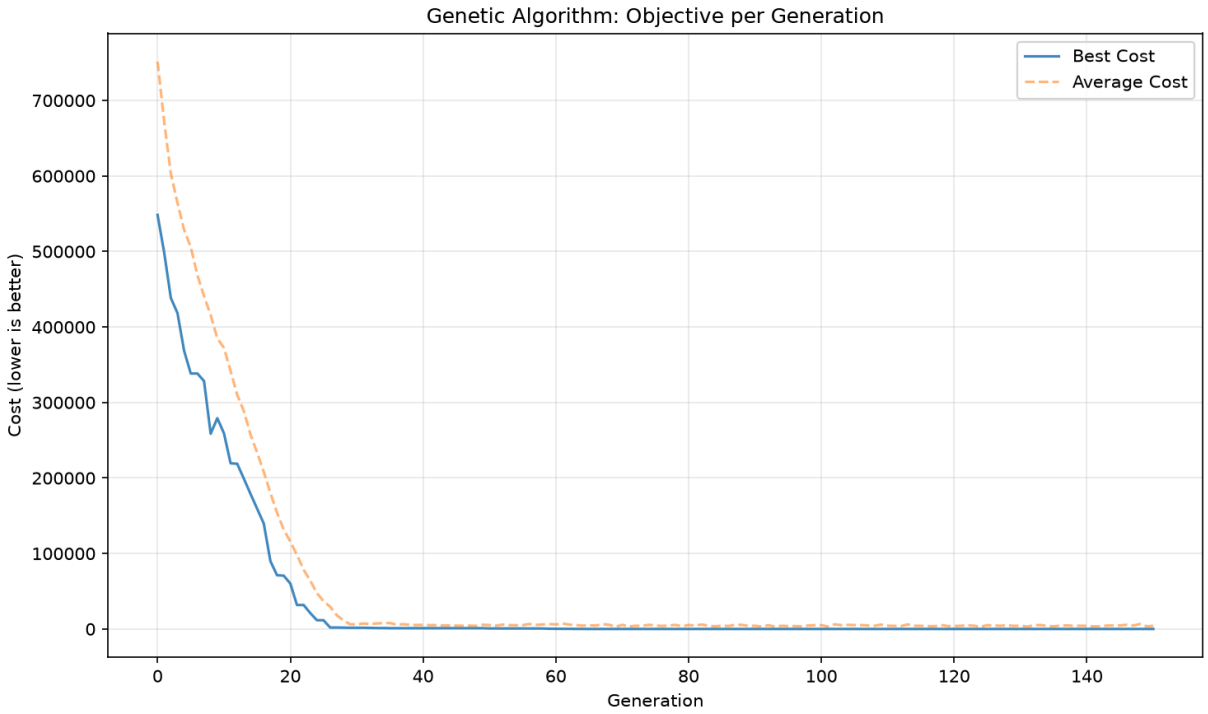



```
TOTAL VALUE di truk : 1625

[truckManhattanCafe] value=593  berat=120/500
  constraint: MELANGGAR -> clipping, melayang, fragile di bawah
[truckHishiAkebono] value=592  berat=206/500
  constraint: MELANGGAR -> clipping, melayang
[truckNishinoFlower] value=440  berat=28/500
  constraint: MELANGGAR -> melayang

DI DALAM MISI KERBS (id: x,y,z | w x l x h [orientasi])
  --- truckManhattanCafe (10 paket) ---
   0: 0,0,0 | 3x5x3 [xyz] v=90  <-- clipping dgn 22, clipping dgn 20, clipping dgn 21
   1: 0,5,3 | 2x6x3 [xyz] v=48  <-- clipping dgn 20, 4i alas fragile 22
  22: 2,1,0 | 8x10x7 [xzy] v=70 *fragile <-  clipping dgn 35, clipping dgn 36, clipping dgn 31, clipping dgn 18, clipping dgn 39, clipping dgn 6
  20: 5,9,0 | 1x5x4 [xyz] v=61  <-- clipping dgn 25, clipping dgn 22, clipping dgn 21, clipping dgn 8, clipping dgn 18
  21: 1,1,0 | 6x8x4 [xzy] v=63 *fragile <-- clipping dgn 29, clipping dgn 22, clipping dgn 20, clipping dgn 6
  35: 0,0,7 | 3x5x2 [xyz] v=30 *fragile <-  clipping dgn 31, clipping dgn 18, fragile 22
  17: 2,11,2 | 4x6x4 [xzy] v=61 *fragile <-  clipping dgn 6, clipping dgn 8
  18: 0,9,1  | 10x9x1 [yxz] v=90  <-- clipping dgn 22, clipping dgn 20, clipping dgn 6, melayang
  36: 7,8,0  | 3x8x5 [xyz] v=36 *fragile  <-  clipping dgn 1, clipping dgn 22
   6: 1,4,0 | 2x9x5 [xzy] v=96 *fragile  <-  clipping dgn 22, clipping dgn 21, clipping dgn 17, clipping dgn 18
  --- truckHishiAkebono (17 paket) ---
   8: 5,4,0 | 10x10x3 [xzy] v=67  <-- clipping dgn 13, clipping dgn 2, clipping dgn 24, clipping dgn 29, clipping dgn 36
   5: 10,14,6 | 5x3x4 [xzy] v=31  <-- clipping dgn 2, melayang
  23: 0,10,0 | 5x8x2 [yxz] v=30 *fragile <-- clipping dgn 14, clipping dgn 3, clipping dgn 26
  13: 0,5,0 | 6x6x9 [zyx] v=98 *fragile  <-- clipping dgn 8, clipping dgn 24
  19: 0,0,0 | 5x5x5 [xyz] v=75 *fragile  <-  fragile 22
  27: 1,7,1 | 4x4x4 [xzy] v=24  <-- clipping dgn 5, melayang
   9: 1,17,0 | 9x10x10 [yxz] v=61 *fragile <-- clipping dgn 14, clipping dgn 29, clipping dgn 11, clipping dgn 26, clipping dgn 39
   2: 2,12,1 | 10x5x6 [yzx] v=52 *fragile <-  clipping dgn 6, clipping dgn 5, clipping dgn 25, clipping dgn 39, clipping dgn 16, melayang
  14: 5,16,0 | 10x6x9 [xzy] v=76 *fragile <-- clipping dgn 23, clipping dgn 9, clipping dgn 3, clipping dgn 11, clipping dgn 26
  15: 0,0,0 | 4x4x4 [xzy] v=24  <-- clipping dgn 29, melayang
  25: 8,7,6  | 5x8x4 [zxy] v=35  <-- clipping dgn 2, clipping dgn 29, clipping dgn 26, melayang
  29: 8,8,0  | 8x10x8 [yxz] v=95  <-- clipping dgn 8, clipping dgn 9, clipping dgn 2, clipping dgn 15, clipping dgn 16
   3: 2,19,1  | 10x10x9 [xyz] v=99 <-- clipping dgn 23, clipping dgn 12, clipping dgn 27, clipping dgn 14, clipping dgn 39, melayang
  11: 1,22,0 | 9x9x10 [yxz] v=90 *fragile <-- clipping dgn 9, clipping dgn 14, clipping dgn 26
  12: 1,13,0 | 3x7x10 [zyx] v=21 *fragile <-- clipping dgn 14, clipping dgn 26, clipping dgn 3, clipping dgn 11
  39: 4,15,0 | 4x10x9 [yzx] v=91 *fragile <-- clipping dgn 9, clipping dgn 7, clipping dgn 15, clipping dgn 29
   4: 11,5,2 | 1x5x2 [zxy] v=71  <-- melayang
  --- truckNishinoFlower (3 paket) ---
  16: 0,0,0 | 8x13x1 [xzy] v=58  <-- melayang
  28: 9,9,4 | 4x3x1 [xyz] v=36  <-- melayang
   7: 3,3,4 | 6x6x3 [xzy] v=64  <-- melayang

DI LUAR / GERBANG (0 paket)
(kosong)
```


Genetic Algorithm - Run 13 - Final

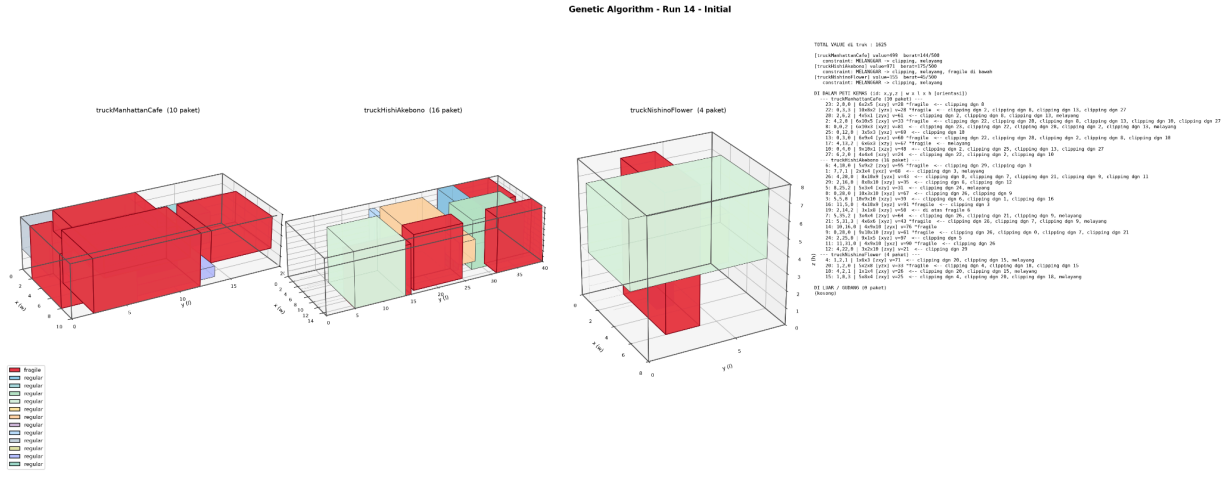


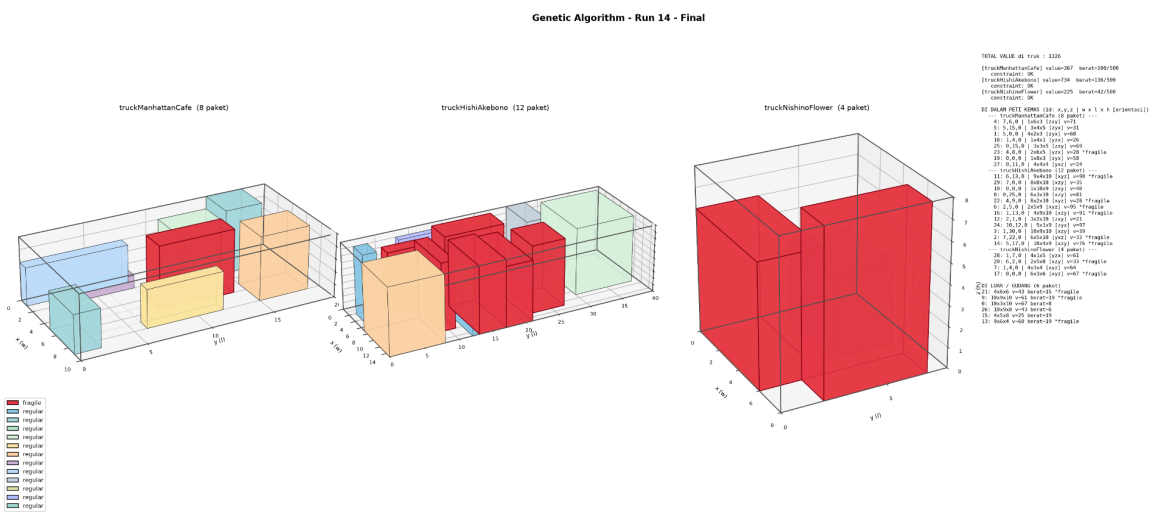

```
TOTAL VALUE di truk : 1279

[truckManhattanCafe] value=387  berat=89/500
  constraint: OK
[truckHiliAkebono] value=747  berat=179/500
  constraint: OK
[truckNiliznoFlower] value=145  berat=31/500
  constraint: OK

DI DALAM MISI KERBS (id: x,y,z | w x l x h [orientasi])
  --- truckManhattanCafe (7 paket) ---
   15: 0,6,0 | 8x9x5 [yxz] v=25
   25: 2,15,0 | 5x3x3 [xzy] v=69
   24: 1,11,6 | 9x5x1 [xzy] v=97
    4: 3,19,0 | 6x3x3 [zyx] v=71
    1: 4,0,0 | 4x5x2 [xzy] v=48
   18: 0,13,3 | 4x1x1 [xyz] v=29
   10: 6,15,0 | 4x3x3 [xzy] v=28
  --- truckHishiAkebono (14 paket) ---
   16: 0,0,0 | 9x4x10 [yzx] v=91 *fragile
    8: 12,15,0 | 3x10x10 [yxz] v=67
   10: 14,5,0 | 2x9x10 [yxz] v=48
   20: 9,0,0 | 4x5x1 [zyx] v=61
   27: 8,9,0 | 4x4x4 [yxz] v=24
   23: 0,10,0 | 5x8x2 [yxz] v=30 *fragile
    8: 0,9,0 | 6x5x10 [xzy] v=81
    6: 13,0,0 | 2x5x9 [xyz] v=95 *fragile
   22: 7,90,0 | 8x2x10 [xyz] v=28 *fragile
   12: 1,13,0 | 3x7x10 [zyx] v=21
    3: 2,15,0 | 10x9x10 [zyx] v=39
   29: 0,32,0 | 10x8x8 [yxz] v=35
   26: 0,0,0 | 10x10x3 [xzy] v=67
   28: 0,18,6 | 7x8x5 [xyz] v=53 *fragile
  --- truckNishinoFlower (3 paket) ---
   19: 0,0,0 | 8x1x1 [xzy] v=58
   21: 6,2,0 | 2x6x5 [yzx] v=28 *fragile
   17: 0,2,0 | 6x3x6 [xzy] v=87 *fragile

DI LUAR / GERBANG (0 paket)
   9: 10x9x10 v=91 berat=19 *fragile
  14: 10x9x6 v=76 berat=8 *fragile
  26: 10x9x8 v=91 berat=6
   2: 5x6x10 v=31 berat=14 *fragile
  13: 9x6x10 v=90 berat=6 *fragile
  21: 4x6x6 v=43 berat=15 *fragile
```

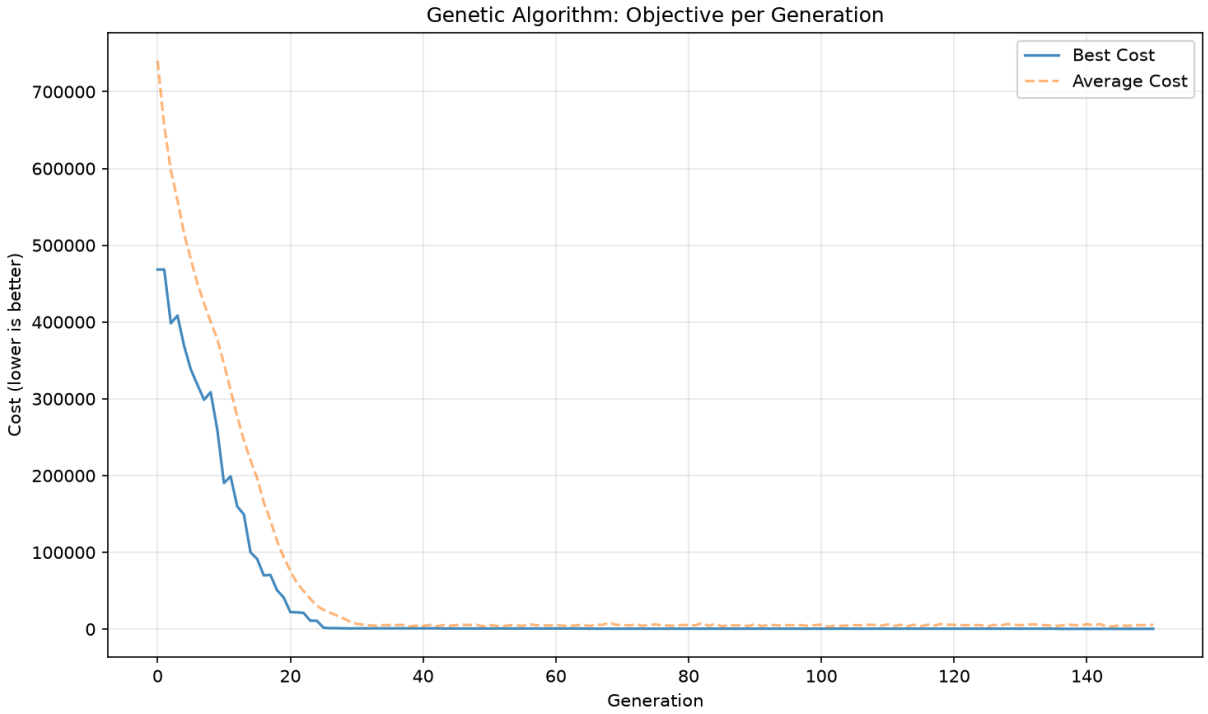


Populasi = 200
Jumlah generasi = 150
value akhir = 1279 | cost = 81 | durasi = 195.3s

Run 2



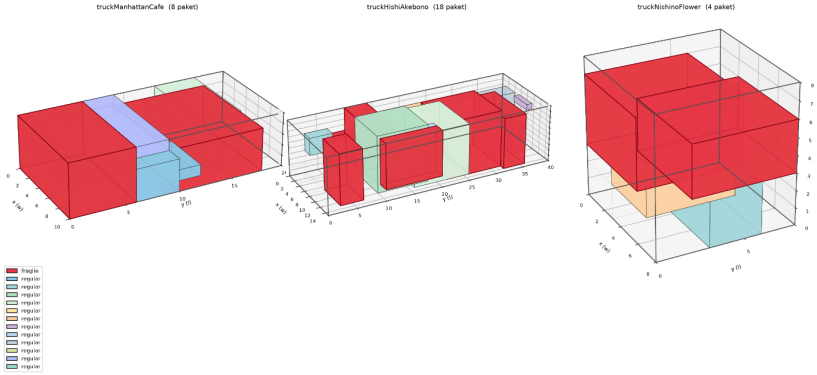




Populasi = 200
Jumlah generasi = 150
value akhir = 1326 | cost = 394 | durasi = 166.5s

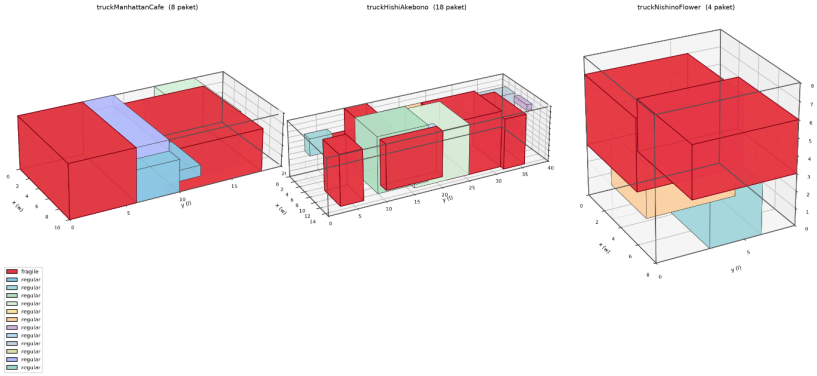
Run 3

Genetic Algorithm - Run 15 - Initial

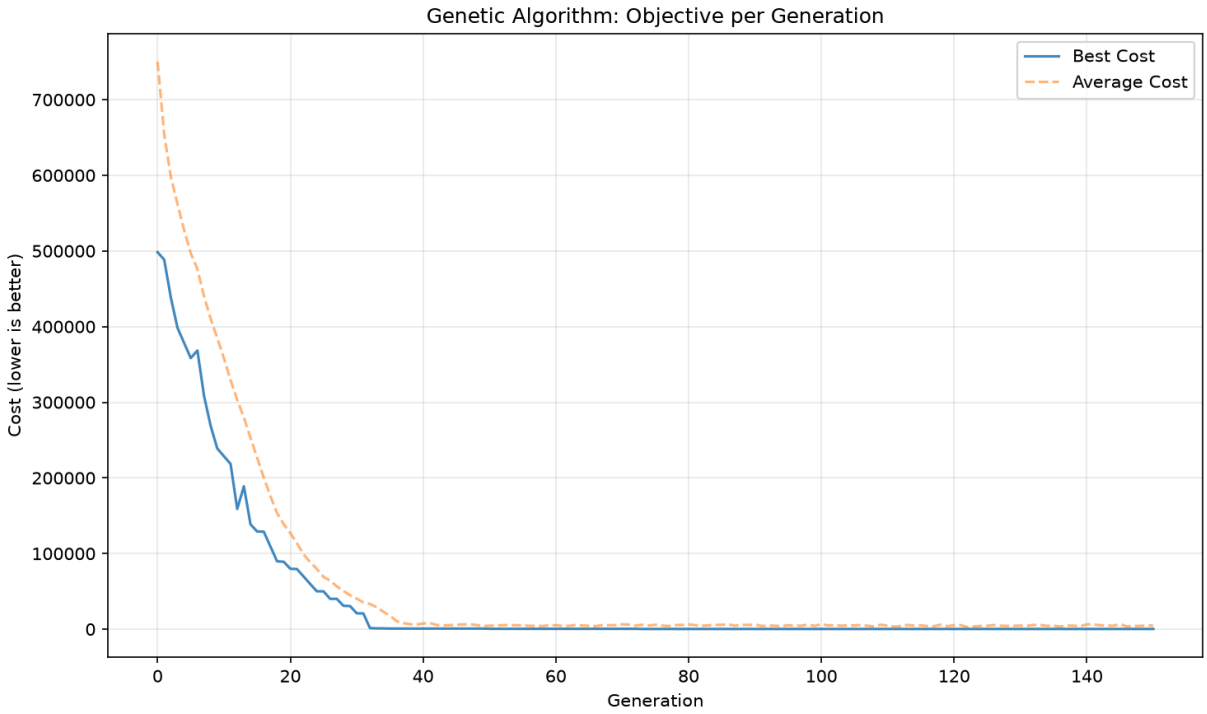



```
TOTAL VALUE di truk : 3625

[truckManhattanCafe] value=415  berat=467/500
  constraint: MELANGGAR -> clipping, melayang
[truckHishiAkebono] value=984  berat=227/500
  constraint: MELANGGAR -> clipping, melayang
[truckNishinoFlower] value=238  berat=66/500
  constraint: MELANGGAR -> clipping, melayang

DI DALAM PETI KEMAS (id: x,y,z | w x l x h (orientasi))
--- truckManhattanCafe (8 paket) ---
  18: 0,0,0 | 10x5x1 [yzx] w=50 <-- clipping dgn 2, clipping dgn 7, melayang
   2: 0,0,0 | 10x5x1 [yzx] w=50 *fragile <-- clipping dgn 18, clipping dgn 0, clipping dgn 2, clipping dgn 24
   6: 0,5,0 | 10x3x2 [yzx] w=42 *fragile <-- clipping dgn 2, clipping dgn 7, clipping dgn 24, clipping dgn 14
   7: 3,3,2 | 4x4x3 [yzx] w=64  <-- clipping dgn 18, clipping dgn 2, clipping dgn 6
  24: 5,3,1 | 5x9x1 [yzx] w=97  <-- clipping dgn 2, clipping dgn 0, clipping dgn 14, melayang
  17: 0,7,1 | 10x3x3 [xzy] w=73  <-  clipping dgn 14, clipping dgn 15, melayang
  14: 0,9,0 | 10x9x4 [xyz] w=76 *fragile <-- clipping dgn 0, clipping dgn 24, clipping dgn 12, clipping dgn 25
  15: 0,13,0 | 8x4x5 [yzx] w=25  <-  clipping dgn 17, clipping dgn 14
--- truckHishiAkebono (18 paket) ---
  12: 0,0,4 | 5x5x5 [xyz] w=75  <-  clipping dgn 0
  22: 15,10,2 | 2x10x8 [zxy] w=28 *fragile  <-- melayang
  26: 2,10,2 | 4x5x1 [zyx] w=61  <-- clipping dgn 9, clipping dgn 11, clipping dgn 8, melayang
  19: 7,10,0 | 4x5x1 [zyx] w=26  <-- melayang
  27: 8,14,2 | 4x4x4 [zyx] w=58  <-- clipping dgn 26, clipping dgn 3, clipping dgn 29, melayang
  11: 4,12,0 | 10x4x4 [zyx] w=96 *fragile  <-- clipping dgn 20
   3: 3,10,0 | 10x10x5 [yxz] w=150  <-- clipping dgn 9, clipping dgn 26, clipping dgn 3, melayang
  25: 5,10,0 | 10x10x9 [yxz] w=196  <-  clipping dgn 9, clipping dgn 26, clipping dgn 27, clipping dgn 19, clipping dgn 29
   1: 2,2,5 | 2x4x2 [zxy] w=68  <-- melayang
  29: 2,16,1 | 8x10x8 [yxz] w=55  <-  clipping dgn 9, clipping dgn 26, clipping dgn 27, clipping dgn 3, melayang
  20: 2,16,0 | 8x2x5 [yxz] w=33 *fragile  <-  clipping dgn 0
   9: 4,18,2 | 5x10x8 [yxz] w=43  <-  clipping dgn 26, clipping dgn 6, clipping dgn 23, melayang
   8: 1,18,2 | 2x10x8 [zyx] w=81  <-  clipping dgn 28, clipping dgn 29, melayang
   4: 2,2,8 | 1x3x6 [yxz] w=71  <-  clipping dgn 6
  23: 15,18,2 | 2x10x8 [zxy] w=28 *fragile  <-- melayang
  28: 2,30,7 | 4x5x1 [zyx] w=61  <-  clipping dgn 9, clipping dgn 11, clipping dgn 8, melayang
  16: 6,26,0 | 8x10x8 [zyx] w=43  <-  clipping dgn 26, clipping dgn 3, clipping dgn 29, clipping dgn 25
  10: 4,28,0 | 8x10x10 [zyx] w=43  <-  clipping dgn 27, clipping dgn 19, clipping dgn 3, clipping dgn 29, clipping dgn 16, clipping dgn 22
  18: 7,30,0 | 4x1x1 [zyx] w=26  <-- melayang
  27: 8,34,2 | 4x4x4 [zyx] w=58  <-- clipping dgn 26, clipping dgn 3, clipping dgn 29, melayang
  11: 4,32,0 | 10x4x4 [zyx] w=96 *fragile  <-- clipping dgn 20
   3: 3,38,0 | 10x10x5 [yxz] w=150  <-- clipping dgn 9, clipping dgn 26, clipping dgn 3, melayang
   5: 3,36,0 | 10x10x9 [yxz] w=196  <-  clipping dgn 9, clipping dgn 26, clipping dgn 27, clipping dgn 19, clipping dgn 29
   1: 2,2,5 | 2x4x2 [zxy] w=68  <-- melayang
  29: 2,36,1 | 8x10x8 [yxz] w=55  <-  clipping dgn 9, clipping dgn 26, clipping dgn 27, clipping dgn 3, melayang
  20: 2,36,0 | 8x2x5 [yxz] w=33 *fragile  <-  clipping dgn 0
   9: 4,38,2 | 5x10x8 [yxz] w=43  <-  clipping dgn 26, clipping dgn 6, clipping dgn 23, melayang
   8: 1,38,2 | 2x10x8 [zyx] w=81  <-  clipping dgn 28, clipping dgn 29, melayang
   6: 2,6,2 | 1x5x5 [xzy] w=95 *fragile  <-  clipping dgn 8, clipping dgn 36, melayang
  23: 4,12,6 | 5x2x6 [zxy] w=28 *fragile  <-  clipping dgn 26, clipping dgn 16, melayang
  13: 7,7,8 | 6x4x9 [zxy] w=66 *fragile
  --- truckNishinoFlower (4 paket) ---
  25: 3,3,0 | 5x3x5 [xyz] w=69  <-  clipping dgn 5
  17: 2,2,3 | 6x8x3 [xzy] w=87 *fragile  <-  clipping dgn 5, clipping dgn 21
   5: 3,8,2 | 4x5x3 [yxz] w=31  <-  clipping dgn 25, clipping dgn 17, clipping dgn 31, melayang
  21: 0,8,3 | 6x6x4 [zyx] w=43 *fragile  <-  clipping dgn 17, clipping dgn 5

DI LUAR / OVERAND (8 paket)
(kosong)
```

Genetic Algorithm - Run 15 - Initial




```
TOTAL VALUE di truk : 3625

[truckManhattanCafe] value=415  berat=467/500
  constraint: MELANGGAR -> clipping, melayang
[truckHishiAkebono] value=984  berat=227/500
  constraint: MELANGGAR -> clipping, melayang
[truckNishinoFlower] value=238  berat=66/500
  constraint: MELANGGAR -> clipping, melayang

DI DALAM PETI KEMAS (id: x,y,z | w x l x h (orientasi))
--- truckManhattanCafe (8 paket) ---
  18: 0,0,0 | 10x5x1 [yzx] w=50 <-- clipping dgn 2, clipping dgn 7, melayang
   2: 0,0,0 | 10x5x1 [yzx] w=50 *fragile <-- clipping dgn 18, clipping dgn 0, clipping dgn 2, clipping dgn 24
   6: 0,5,0 | 10x3x2 [yzx] w=42 *fragile <-- clipping dgn 2, clipping dgn 7, clipping dgn 24, clipping dgn 14
   7: 3,3,2 | 4x4x3 [yzx] w=64  <-- clipping dgn 18, clipping dgn 2, clipping dgn 6
  24: 5,3,1 | 5x9x1 [yzx] w=97  <-- clipping dgn 2, clipping dgn 0, clipping dgn 14, melayang
  17: 0,7,1 | 10x3x3 [xzy] w=73  <-  clipping dgn 14, clipping dgn 15, melayang
  14: 0,9,0 | 10x9x4 [xyz] w=76 *fragile <-- clipping dgn 0, clipping dgn 24, clipping dgn 12, clipping dgn 25
  15: 0,13,0 | 8x4x5 [yzx] w=25  <-  clipping dgn 17, clipping dgn 14
--- truckHishiAkebono (18 paket) ---
  12: 0,0,4 | 5x5x5 [xyz] w=75  <-  clipping dgn 0
  22: 15,10,2 | 2x10x8 [zxy] w=28 *fragile  <-- melayang
  26: 2,10,2 | 4x5x1 [zyx] w=61  <-- clipping dgn 9, clipping dgn 11, clipping dgn 8, melayang
   9: 4,12,6 | 5x10x8 [yxz] w=43  <-  clipping dgn 26, clipping dgn 3, clipping dgn 29
  19: 7,10,0 | 4x5x1 [zyx] w=26  <-- melayang
  27: 8,14,2 | 4x4x4 [zyx] w=58  <-- clipping dgn 26, clipping dgn 3, clipping dgn 29, melayang
  11: 4,12,0 | 10x4x4 [zyx] w=96 *fragile  <-- clipping dgn 20
  18: 10,12,0 | 3x8x1 [yzx] w=58  <-- clipping dgn 26, clipping dgn 3, melayang
   3: 3,10,0 | 10x10x5 [yxz] w=150  <-- clipping dgn 9, clipping dgn 26, clipping dgn 27, clipping dgn 19, clipping dgn 29
   1: 2,2,5 | 2x4x2 [zxy] w=68  <-- melayang
  29: 2,16,1 | 8x10x8 [yxz] w=55  <-  clipping dgn 9, clipping dgn 26, clipping dgn 27, clipping dgn 3, melayang
  20: 2,16,0 | 8x2x5 [yxz] w=33 *fragile  <-  clipping dgn 0
  10: 4,18,1 | 10x4x4 [yzx] w=91 *fragile  <-  clipping dgn 4, clipping dgn 23, melayang
   8: 1,18,2 | 2x10x8 [zyx] w=81  <-  clipping dgn 28, clipping dgn 29, melayang
   6: 2,6,2 | 1x5x5 [xzy] w=95 *fragile  <-  clipping dgn 8, clipping dgn 36, melayang
  23: 15,18,2 | 2x10x8 [zxy] w=28 *fragile  <-  clipping dgn 26, clipping dgn 16, melayang
  13: 7,7,8 | 6x4x9 [zxy] w=66 *fragile
  --- truckNishinoFlower (4 paket) ---
  25: 3,3,0 | 5x3x5 [xyz] w=69  <-  clipping dgn 5
  17: 2,2,3 | 6x8x3 [xzy] w=87 *fragile  <-  clipping dgn 5, clipping dgn 21
   5: 3,8,2 | 4x5x3 [yxz] w=31  <-  clipping dgn 25, clipping dgn 17, clipping dgn 31, melayang
  21: 0,8,3 | 6x6x4 [zyx] w=43 *fragile  <-  clipping dgn 17, clipping dgn 5

DI LUAR / OVERAND (8 paket)
(kosong)
```



Populasi = 200
Jumlah generasi = 150
value akhir = 1280 | cost = 300 | durasi = 218.2s

2.3.6.5.2. Analisis

Berdasarkan hasil yang sudah digenerasi, diketahui info sebagai berikut:
Run 1 menghasilkan value akhir sebanyak 1279, cost sebanyak 81, dan memiliki durasi 195.3 detik. Tidak ada paket yang melanggar constraint.
Run 2 menghasilkan value akhir sebanyak 1326, cost sebanyak 394, dan memiliki durasi 166.5 detik. Tidak ada paket yang melanggar constraint.
Run 3 menghasilkan value akhir sebanyak 1280, cost sebanyak 300, dan memiliki durasi 218.2 detik. Tidak ada paket yang melanggar constraint.

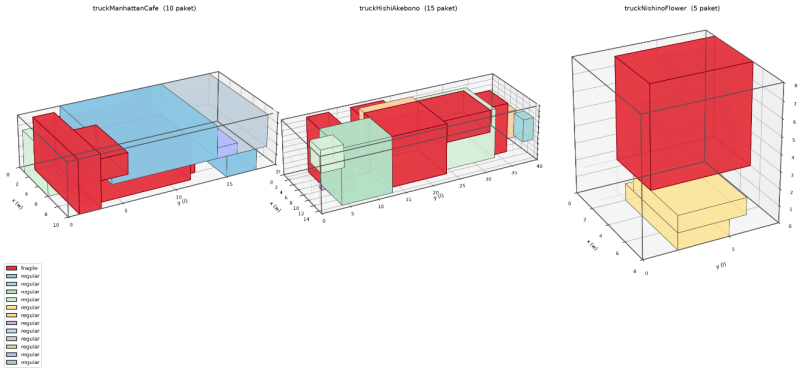
Cost yang dihasilkan lebih konsisten dibandingkan eksperimen 5. Value akhir konsisten. Durasi eksekusi lebih lama.

2.3.6.6. Jumlah Iterasi = 150 (sebagai kontrol) dan Jumlah Populasi = 300

2.3.6.6.1. Hasil

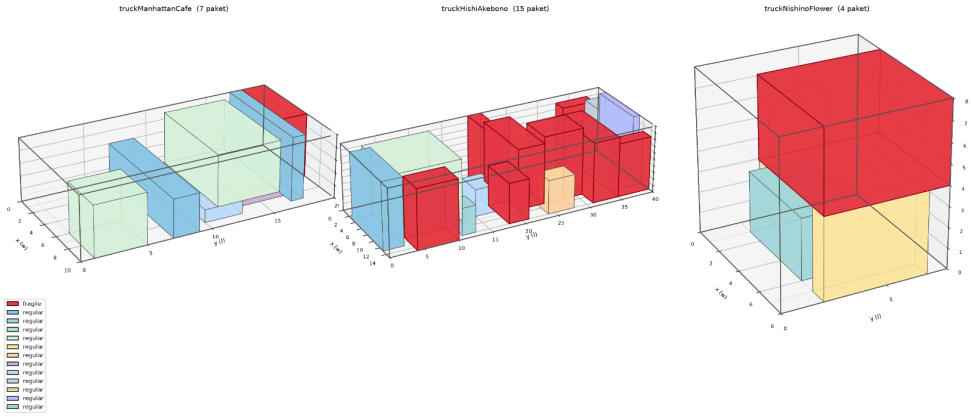
Run 1

Genetic Algorithm - Run 16 - Initial

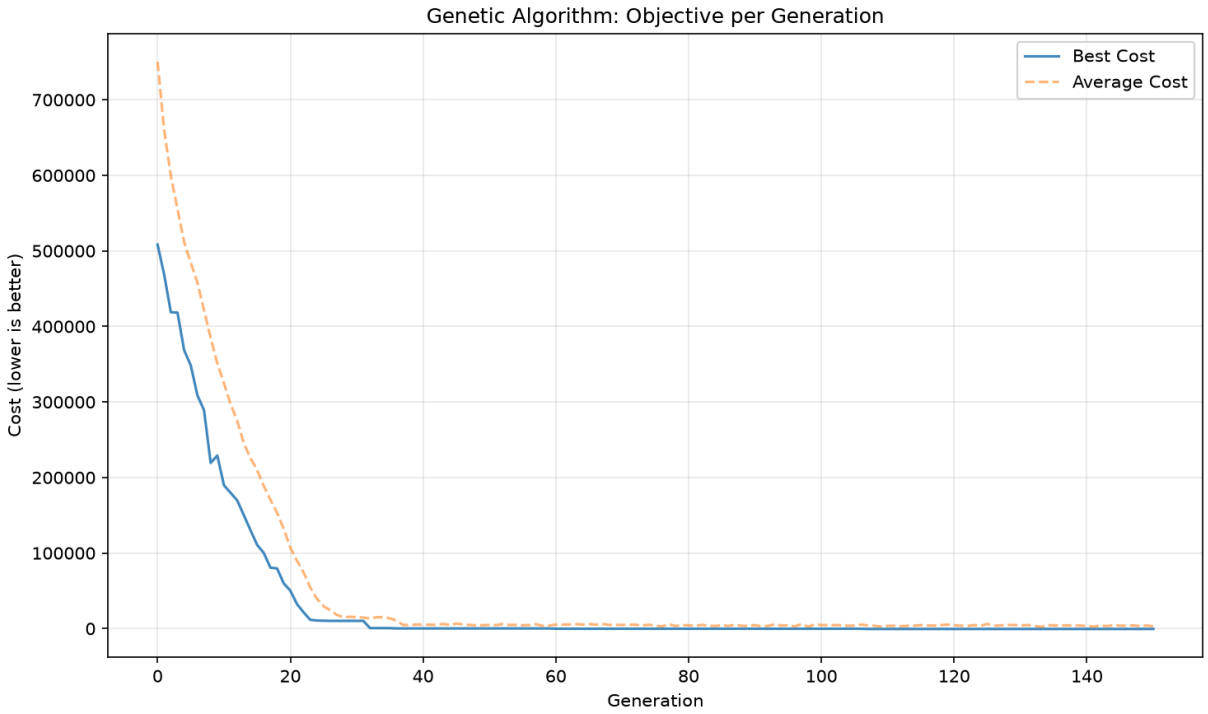



```
TOTAL VALUE dl truk : 3625

[truckManhattanCafe] value=571  berat=117/500
  constraint: MELANGGAR -> clipping, melayang
[truckHishiAkebono] value=746  berat=281/500
  constraint: MELANGGAR -> clipping, melayang, fragile di bawah
[truckNishinoFlower] value=1000  berat=500/500
  constraint: MELANGGAR -> clipping, melayang

DI DALAM PETI KOMBIS (id: x,y,z | w x l x h (orientasi))
  --- truckManhattanCafe (10 paket) ---
  29: 0,0,0 | 0x2x5 (yxz) v=35 *fragile <- clipping dgn 8, clipping dgn 14, clipping dgn 18
  8: 0,13,2 | 10x6x3 (yxz) v=81 <-- clipping dgn 8, clipping dgn 24, clipping dgn 18
  0: 0,4,2 | 10x10x3 (xzy) v=87 <-- clipping dgn 39, clipping dgn 8, clipping dgn 23, clipping dgn 14, clipping dgn 10, clipping dgn 15, melayang
  4: 1,1,0 | 9x2x1 (yzx) v=91 *fragile <- clipping dgn 23, clipping dgn 14, clipping dgn 15
  1: 1,3,3 | 9x5x4 (xzy) v=98 *fragile <- clipping dgn 0, clipping dgn 6, clipping dgn 10, clipping dgn 25, melayang
  24: 1,10,3 | 9x5x1 (xzy) v=97 *fragile <- clipping dgn 8
  14: 0,2,0 | 0x10x4 (yxz) v=36 *fragile <- clipping dgn 39, clipping dgn 8, clipping dgn 6, clipping dgn 23, clipping dgn 10, clipping dgn 15
  28: 0,7,2 | 10x9x1 (yzx) v=98 <-- clipping dgn 28, clipping dgn 8, clipping dgn 14, clipping dgn 25
  35: 1,0,3 | 8x8x4 (xyz) v=28 <-- clipping dgn 0, clipping dgn 8, clipping dgn 23, clipping dgn 14, clipping dgn 18
  --- truckHishiAkebono (15 paket) ---
  26: 6,4,0 | 4x4x10 (yxz) v=43 <-- clipping dgn 27, clipping dgn 18, clipping dgn 14
  27: 5,1,0 | 4x4x4 (zxy) v=24 <-- clipping dgn 18, melayang
  11: 2,23,3 | 0x10x4 (xzy) v=90 *fragile <-- clipping dgn 29, clipping dgn 37, clipping dgn 3, clipping dgn 22, melayang
  12: 4,22,0 | 6x4x8 (zxy) v=68 *fragile <-- clipping dgn 5, clipping dgn 3
  2: 4,10,0 | 6x4x8 (zxy) v=68
  29: 0,14,1 | 8x10x8 (xzy) v=85 <-- clipping dgn 11, clipping dgn 2, clipping dgn 9, clipping dgn 22, melayang
  5: 0,24,3 | 2x4x1 (yzx) v=21 <-- clipping dgn 13, melayang
  7: 1,12,4 | 10x2x8 (yzx) v=37 *fragile <-- clipping dgn 29, clipping dgn 9, clipping dgn 22
  3: 1,17,4 | 10x10x3 (yzx) v=97 <-- clipping dgn 29, clipping dgn 3, clipping dgn 22, melayang
  30: 12,1,1 | 0x10x4 (xzy) v=90 *fragile <-- clipping dgn 9
  36: 1,4,0 | 9x4x10 (yxz) v=91 *fragile <-- clipping dgn 26, clipping dgn 27
  1: 12,37,4 | 3x2x4 (xyz) v=68 <-- melayang
  17: 1,0,4 | 8x8x4 (xyz) v=92 *fragile <-- clipping dgn 11, clipping dgn 3, di atas fragile 22
  37: 1,10,4 | 8x8x4 (xyz) v=92 *fragile <-- clipping dgn 11, clipping dgn 17, clipping dgn 22
  22: 1,35,1 | 0x10x3 (xzy) v=20 *fragile <-- clipping dgn 31, clipping dgn 29, clipping dgn 2, clipping dgn 9, clipping dgn 3, melayang
  --- truckNishinoFlower (5 paket) ---
  20: 3,2,1 | 5x7x1 (zxy) v=61
  31: 2,0,1 | 5x7x1 (zxy) v=61
  9: 1,7,3 | 4x4x6 (zyx) v=44 *fragile <- clipping dgn 25, clipping dgn 7
  25: 0,9,2 | 3x3x5 (zxy) v=69 <-- clipping dgn 21, clipping dgn 7, melayang
  7: 1,3,3 | 4x4x1 (xzy) v=64 <-- clipping dgn 21, clipping dgn 25

DI LUAR / GUDANG (0 paket)
(kosong)
```

Genetic Algorithm - Run 16 - Final

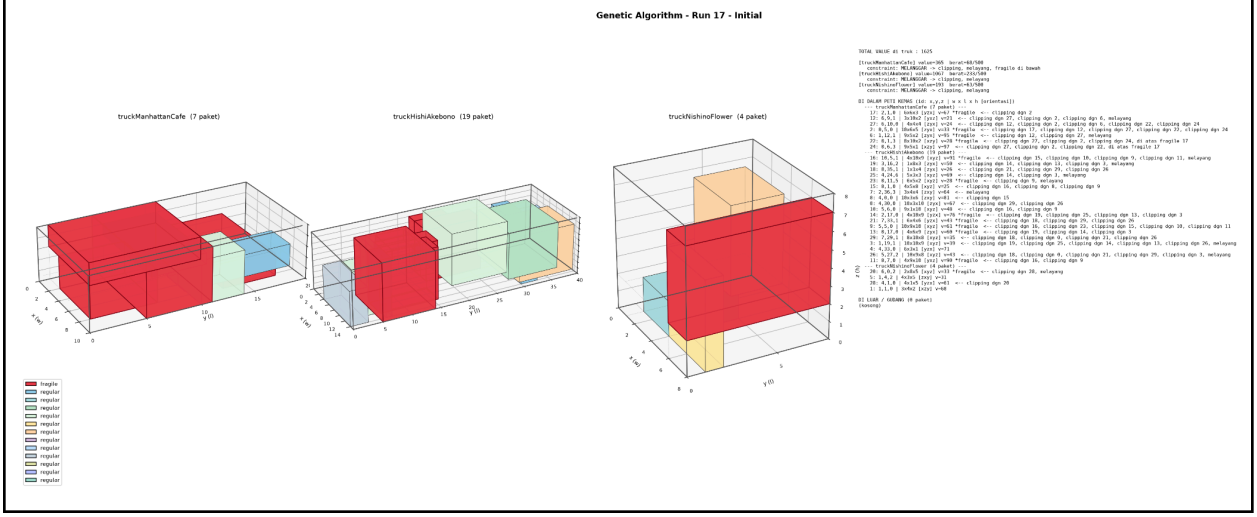


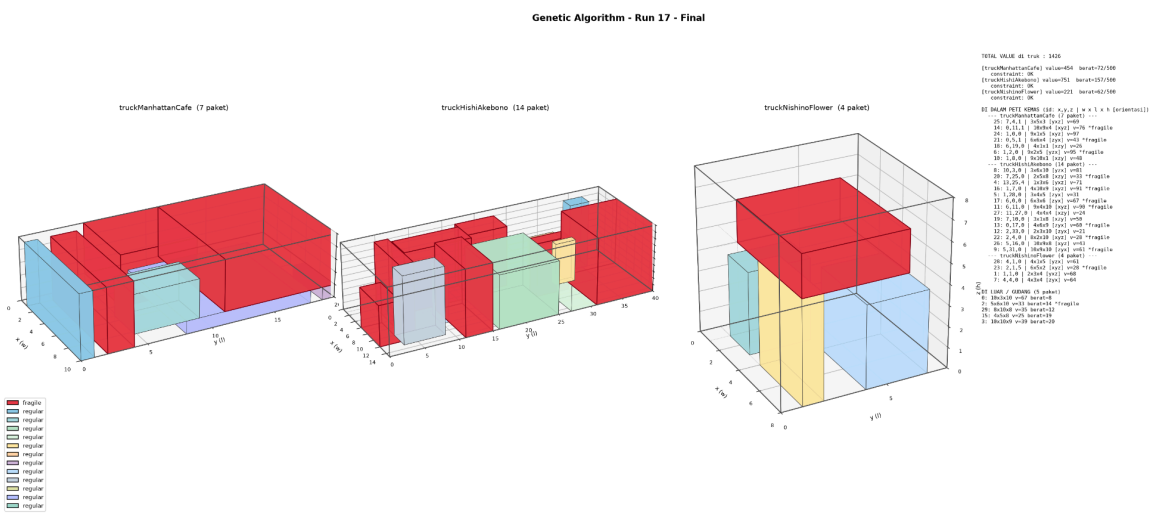

```
TOTAL VALUE dl truk : 1410

[truckManhattanCafe] value=271  berat=78/500
  constraint: OK
[truckHishiAkebono] value=895  berat=295/500
  constraint: OK
[truckNishinoFlower] value=244  berat=46/500
  constraint: OK

DI DALAM PETI KOMBIS (id: x,y,z | w x l x h (orientasi))
  --- truckManhattanCafe (7 paket) ---
  23: 0,18,0 | 0x2x5 (xzy) v=26 *fragile
  27: 0,1,0 | 4x4x4 (zyx) v=24
  15: 1,11,1 | 8x5x4 (yxz) v=25
  10: 7,13,0 | 1x4x1 (yzx) v=26
  24: 0,17,0 | 9x1x5 (xyz) v=97
  26: 0,7,0 | 9x3x3 (xzy) v=97
  12: 0,2,0 | 10x3x8 (xzy) v=21
  --- truckHishiAkebono (15 paket) ---
  1: 11,11,0 | 3x2x4 (xyz) v=68
  22: 0,19,0 | 8x2x10 (xyz) v=28 *fragile
  3: 0,3,0 | 10x10x9 (xzy) v=20
  6: 0,31,0 | 2x5x9 (xyz) v=95 *fragile
  7: 6,15,0 | 4x4x8 (zyx) v=63
  37: 1,0,4 | 8x8x4 (xyz) v=92 *fragile
  11: 1,21,0 | 10x4x9 (zyx) v=90 *fragile
  8: 3,0,0 | 10x3x10 (zyx) v=67
  0: 3,36,0 | 0x3x10 (xzy) v=81
  19: 2,39,0 | 9x1x10 (xyz) v=48
  13: 11,4,0 | 4x6x5 (zyx) v=60 *fragile
  16: 4,31,0 | 10x4x9 (yxz) v=91 *fragile
  17: 1,0,4 | 8x8x4 (xyz) v=92 *fragile
  5: 11,34,0 | 5x4x5 (zyx) v=53
  20: 12,35,0 | 2x5x8 (zyx) v=53 *fragile
  --- truckNishinoFlower (4 paket) ---
  4: 0,3,0 | 1x3x6 (yxz) v=71
  25: 1,2,0 | 5x3x1 (xyz) v=69
  21: 2,2,4 | 6x6x4 (zxy) v=62 *fragile
  28: 7,2,0 | 1x5x4 (zyx) v=61

DI LUAR / GUDANG (4 paket)
  26: 10x7x8 v=61 berat=6
  9: 10x9x10 v=61 berat=19 *fragile
  14: 10x9x6 v=76 berat=8 *fragile
  29: 8x10x8 v=35 berat=32
```

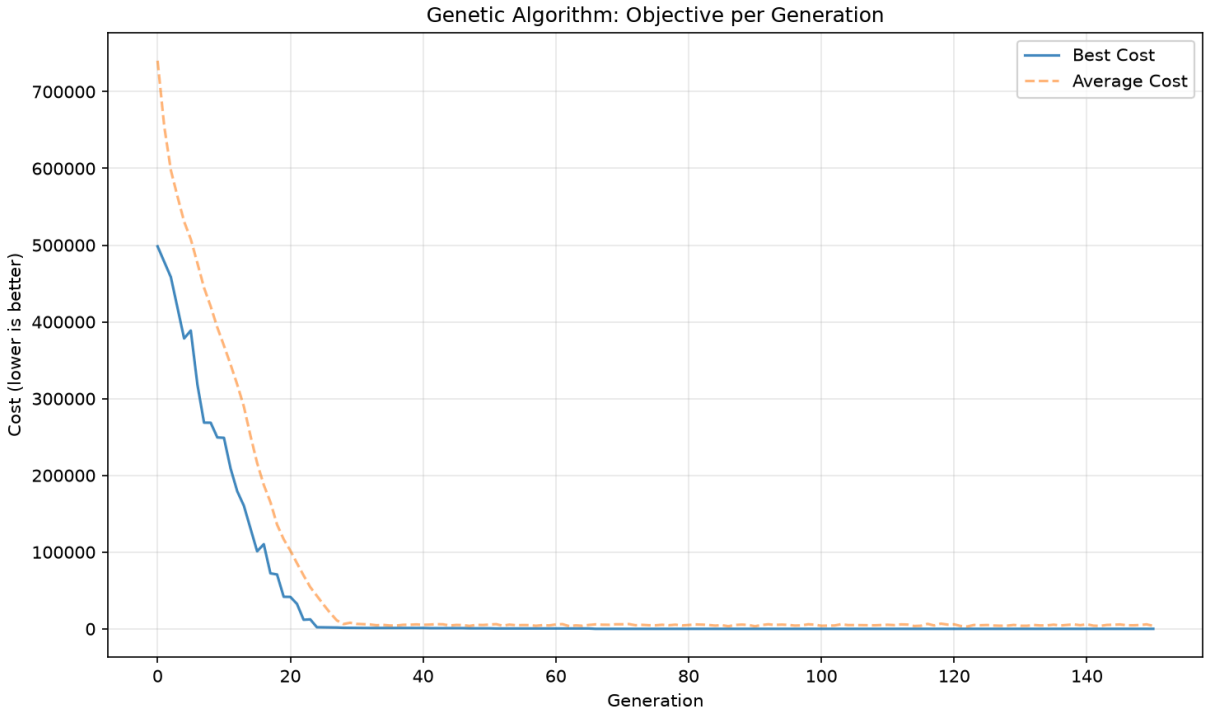


Populasi = 300
Jumlah generasi = 150
value akhir = 1410 | cost = -510 | durasi = 281.2s

Run 2



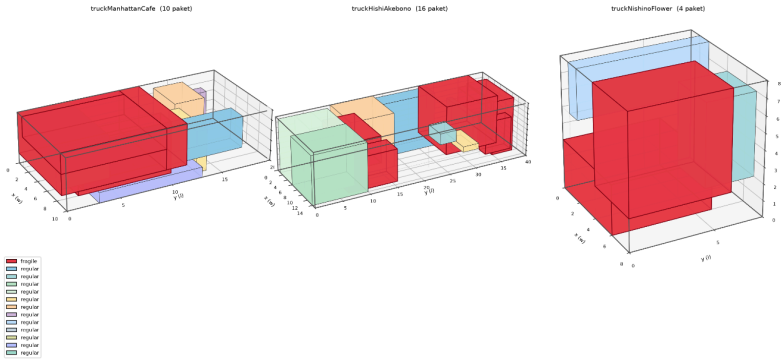




Populasi = 300
Jumlah generasi = 150
value akhir = 1426 | cost = 34 | durasi = 292.5s

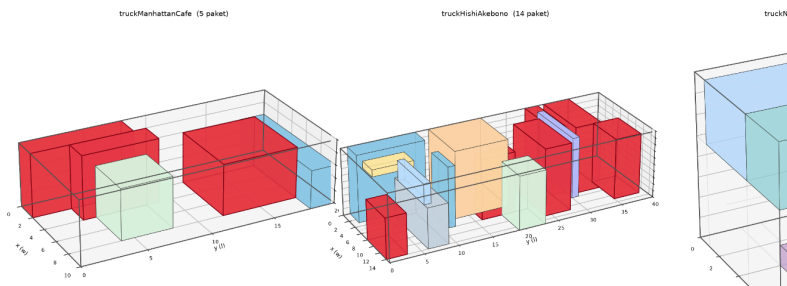
Run 3

Genetic Algorithm - Run 18 - Initial

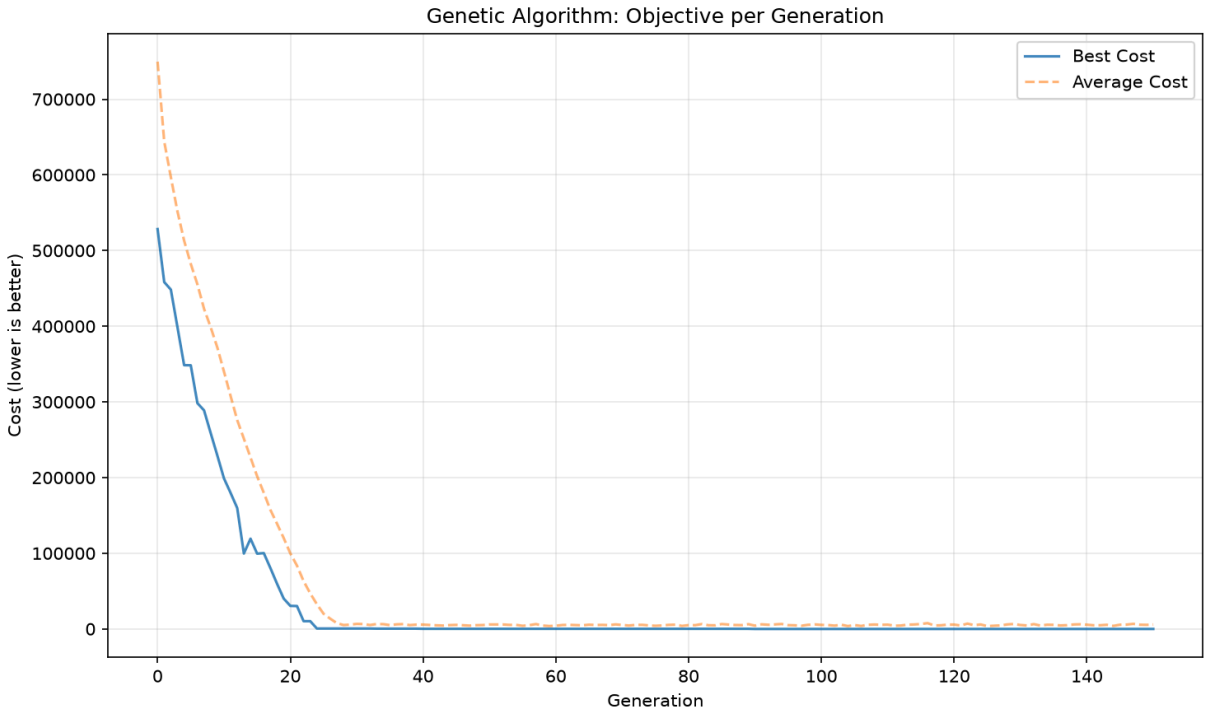



```
TOTAL VALUE di truk : 1025

[truckManhattanCafe] value=7992  berat=122/500
  constraint: MELANGGAR -> clipping, melayang
[truckHishiAkebono] value=895  berat=204/500
  constraint: MELANGGAR -> clipping, melayang
[truckNishinoFlower] value=138  berat=0/500
  constraint: MELANGGAR -> clipping, melayang

DI DALAM PETI KEMAS (id: x,y,z | w x l x h (orientasi)
--- truckManhattanCafe (10 paket) ---
  5: 3,13,0 | 4x3x5 [xyz] v=31  <- clipping dgn 20
 10: 1,3,0 | 3x3x1 [xyz] v=48  <- clipping dgn 6
 14: 0,6,1 | 9x3x4 [yxr] v=7& *fragile  <- clipping dgn 6, clipping dgn 13, clipping dgn 12, clipping dgn 15, clipping dgn 22
 15: 0,0,0 | 2x8x5 [xyz] v=33 *fragile  <- clipping dgn 13, clipping dgn 12, clipping dgn 15, clipping dgn 22, clipping dgn 2
 11: 8,2,3 | 3x10x4 [xyz] v=58 *fragile  <- clipping dgn 14, clipping dgn 6, clipping dgn 12, clipping dgn 15, clipping dgn 22
 18: 1,17,0 | 1x1x4 [zyx] v=36
 12: 8,7,2 | 2x10x3 [xyz] v=21  <- clipping dgn 1f, clipping dgn 11, clipping dgn 20, clipping dgn 22, melayang
 19: 8,0,1 | 8x5x4 [zyx] v=7  <- clipping dgn 15, clipping dgn 6, clipping dgn 11, clipping dgn 22
 17: 0,11,0 | 3x3x1 [zyx] v=61  <- clipping dgn 15, clipping dgn 6
 22: 1,0,3 | 8x10x2 [zyx] v=28 *fragile  <- clipping dgn 14, clipping dgn 6, clipping dgn 11, clipping dgn 12, clipping dgn 15, melayang
--- truckHishiAkebono (16 paket) ---
 23: 2,0,0 | 2x8x5 [xyz] v=33  <- clipping dgn 2
 0: 8,17,1 | 10x10x9 [xzy] v=61 *fragile  <- clipping dgn 27, clipping dgn 19, clipping dgn 10, clipping dgn 0, clipping dgn 20, clipping dgn 4, melayang
 27: 3,14,0 | 4x4x4 [zyx] v=24  <- clipping dgn 9, clipping dgn 19, clipping dgn 20
 13: 5,7,0 | 4x9x6 [yzx] v=60 *fragile  <- clipping dgn 26, clipping dgn 9, clipping dgn 2
 24: 3,15,5 | 9x1x5 [xyz] v=97  <- clipping dgn 9, melayang
 26: 3,0,0 | 8x10x3 [xyz] v=42  <- clipping dgn 27, clipping dgn 27, clipping dgn 20, clipping dgn 23
 8: 1,1,0 | 4x3x10 [zxy] v=81  <- clipping dgn 26, clipping dgn 1
 25: 0,0,0 | 8x10x9 [yzx] v=42  <- clipping dgn 13, clipping dgn 8, clipping dgn 2, clipping dgn 7
 6: 1,10,0 | 8x10x10 [yxz] v=67  <- clipping dgn 9
 20: 0,15,1 | 8x2x5 [yxz] v=13 *fragile  <- clipping dgn 9, clipping dgn 27, clipping dgn 26, melayang
 7: 0,0,0 | 8x10x10 [yxz] v=67  <- clipping dgn 27, clipping dgn 8, clipping dgn 26, clipping dgn 2, clipping dgn 7
 4: 8,20,1 | 9x7x1 [yzx] v=71  <- clipping dgn 9, melayang
 1: 11,23,0 | 4x3x2 [yzx] v=68  <- melayang
 21: 11,15,1 | 2x5x4 [zyx] v=78 *fragile  <- clipping dgn 10, melayang
 3: 8,1,2 | 10x5x5 [zyx] v=32 *fragile  <- clipping dgn 29, clipping dgn 13, clipping dgn 26, clipping dgn 2, melayang
 2: 8,3,2 | 2x6x4 [zyx] v=52  <- clipping dgn 26, clipping dgn 0, melayang
--- truckNishinoFlower (4 paket) ---
 19: 1,0,5 | 1x9x3 [zyx] v=50  <- melayang
 16: 0,0,0 | 9x4x10 [yzx] v=91 *fragile  <- clipping dgn 25, clipping dgn 14
 21: 4,0,2 | 8x9x1 [zxy] v=87  <- clipping dgn 11, melayang
 21: 4,0,2 | 4x6x6 [xyz] v=41 *fragile  <- clipping dgn 17, clipping dgn 25, melayang

DI LUAR / GUDANG (0 paket)
(kosong)
```

Genetic Algorithm - Run 18 - Final



```
TOTAL VALUE di truk : 1373

[truckManhattanCafe] value=149  berat=60/500
  constraint: OK
[truckHishiAkebono] value=905  berat=155/500
  constraint: OK
[truckNishinoFlower] value=319  berat=82/500
  constraint: OK

DI DALAM PETI KEMAS (id: x,y,z | w x l x h (orientasi)
--- truckManhattanCafe (5 paket) ---
 17: 0,17,0 | 4x4x4 [zyx] v=61 *fragile
 28: 0,0,0 | 2x8x5 [xyz] v=33 *fragile
 27: 4,4,0 | 4x4x4 [xyz] v=24
 12: 0,10,0 | 10x2x3 [xzy] v=21
 23: 2,3,0 | 2x8x5 [yxz] v=28 *fragile
--- truckHishiAkebono (14 paket) ---
 11: 0,31,0 | 9x4x10 [xyz] v=90 *fragile
 24: 0,24,0 | 3x5x1 [zyx] v=97
 6: 5,16,0 | 3x3x10 [yxz] v=95 *fragile
 24: 5,11,0 | 5x1x8 [zyx] v=97
 16: 4,24,0 | 9x4x10 [yzx] v=91 *fragile
 17: 0,0,0 | 4x1x6 [yzx] v=67 *fragile
 19: 0,30,0 | 10x1x9 [zyx] v=60
 13: 7,35,0 | 6x4x9 [zxy] v=60 *fragile
 22: 0,28,0 | 8x2x10 [xyz] v=28 *fragile
 1: 0,0,0 | 8x10x10 [yxz] v=67
 25: 6,13,0 | 8x8x10 [yzx] v=25
 8: 0,1,0 | 3x10x10 [yxz] v=67
 15: 10,19,0 | 5x4x8 [yxz] v=25
 19: 5,6,G | 8x1x3 [zyx] v=50
--- truckNishinoFlower (6 paket) ---
 20: 4,3,0 | 4x1x5 [yzx] v=61
 5: 5,7,0 | 4x5x5 [zyx] v=31
 1: 5,3,3 | 4x3x5 [zyx] v=31
 25: 5,0,5 | 3x5x3 [zyx] v=69
 7: 1,0,5 | 4x4x3 [xzy] v=66
 18: 4,2,0 | 4x1x1 [xzy] v=26

DI LUAR / GUDANG (5 paket)
 14: 10x5x8 v=78 berat=8 *fragile
 26: 10x5x8 v=78 berat=8 *fragile
 26: 10x5x8 v=61 berat=6
 2: 5x6x10 v=31 berat=14 *fragile
 9: 10x9x10 v=61 berat=19 *fragile
```



Populasi = 300
Jumlah generasi = 150
value akhir = 1373 | cost = -33 | durasi = 286.6s

2.3.6.6.2. Analisis

Berdasarkan hasil yang sudah digenerasi, diketahui info sebagai berikut:

4) Run 1 menghasilkan value akhir sebanyak 1410, cost sebanyak -510, dan memiliki durasi 281.2 detik. Tidak ada paket yang melanggar constraint.
5) Run 2 menghasilkan value akhir sebanyak 1426, cost sebanyak 34, dan memiliki durasi 292.5 detik. Tidak ada paket yang melanggar constraint.
6) Run 3 menghasilkan value akhir sebanyak 1373, cost sebanyak -33, dan memiliki durasi 286.6 detik. Tidak ada paket yang melanggar constraint.

Genetic algorithm memberikan hasil yang konsisten untuk value akhir. Cost sedikit variatif. Durasi untuk eksekusi program cukup lama.

2.3.6.7. Analisis Keseluruhan Genetic Algorithm

Hasil eksperimen pada 100, 200, dan 300 iterasi menunjukkan bahwa penambahan iterasi tidak selalu menghasilkan perbaikan cost yang signifikan. Pada 100 iterasi, GA sudah menemukan solusi dengan cost -303. Ketika iterasi ditambah menjadi 200 dan 300, value cenderung tetap di kisaran 1200-an, sedangkan cost masih berfluktuasi, bahkan mencapai 10141 pada Run 3 dengan 200 iterasi. Hal ini menunjukkan adanya kondisi dimana tambahan iterasi tidak lagi memberikan peningkatan solusi yang signifikan dan hanya menambah waktu komputasi.

Berbeda dengan jumlah iterasi, peningkatan ukuran populasi menunjukkan pengaruh yang lebih jelas terhadap kualitas solusi. Cost terbaik menurun dari 403 pada populasi 100, menjadi 81 pada populasi 200, dan -510 pada populasi 300. Pada populasi 300 juga diperoleh value tertinggi, yaitu 1410.

Peningkatan ini terjadi karena populasi yang lebih besar memberikan keragaman genetik yang lebih tinggi, sehingga peluang menghasilkan kombinasi penempatan paket yang lebih baik melalui crossover juga meningkat. Namun, populasi yang lebih besar membutuhkan waktu komputasi lebih lama, dengan durasi meningkat dari sekitar 80 detik pada populasi 100 menjadi sekitar 280 detik pada populasi 300. Dengan demikian, terdapat trade off antara kualitas solusi dan waktu komputasi.

3. Kesimpulan dan Saran

3.1. Kesimpulan

Berdasarkan implementasi dan analisis eksperimen terhadap algoritma local search di atas dapat ditarik beberapa kesimpulan, yaitu sebagai berikut.

1) Kedekatan dengan Global Optima dan Kualitas Solusi

Secara umum, Genetic Algorithm (GA) dan Simulated Annealing (SA) mampu mendekati global optima jauh lebih baik dibandingkan varian Hill-Climbing (HC). GA unggul karena mekanisme populasi dan crossover memungkinkannya mengeksplorasi banyak kombinasi state secara paralel, sehingga berhasil menemukan solusi dengan cost negatif (nilai value sangat tinggi, mencapai 1410). SA berada di posisi kedua. Kemampuannya menerima state yang lebih buruk di awal pencarian (saat suhu TT tinggi) membuatnya tidak mudah terjebak di local optimum. Sebaliknya, HC (terutama Steepest Ascent) memiliki performa terburuk karena sifatnya yang terlalu greedy. HC sering terjebak pada local optimum yang memiliki banyak pelanggaran (clipping/melayang) dan tidak berani melewati lembah untuk memperbaikinya. Sideways Move dan Random Restart sedikit lebih baik, tetapi masih kalah dari SA dan GA.

2) Konsistensi Hasil Eksperimen

Algoritma yang paling konsisten yaitu Stochastic HC dan GA. Stochastic HC selalu menghasilkan solusi konsisten di setiap run karena eksplorasinya yang panjang (5000 iterasi). GA sangat konsisten dalam hal value akhir karena populasi yang besar menjaga keragaman genetik sehingga hasil tidak melonjak drastis.

Sedangkan, algoritma yang paling tidak konsisten yaitu Steepest Ascent dan Random Restart. Hasil akhir kedua algoritma ini sangat bergantung pada initial state acak yang terbentuk. Pada Steepest Ascent, satu run bisa menghasilkan cost 3.549, namun run lain bisa meledak hingga 80.471.

3) Perbandingan Durasi Pencarian

Terdapat trade-off yang jelas antara durasi dan kualitas solusi. Urutan algoritma dari yang tercepat hingga terlambat adalah:

HC Steepest (0.5s) < HC Sideways (1.1s) < Stochastic (2.3s) < SA (26.7s) < Random Restart (40s) < GA (280s).

HC sangat cepat karena langsung berhenti saat mencapai puncak lokal. GA menjadi yang paling lambat karena harus mengevaluasi seluruh individu dalam populasi di setiap generasi.

4) Pengaruh Parameter pada Genetic Algorithm (GA)

Semakin besar populasi (100 → 200 → 300), semakin baik kualitas solusi yang dihasilkan (cost turun drastis dari 403 menjadi -510). Populasi besar menyediakan keragaman yang lebih tinggi untuk proses crossover, namun konsekuensinya waktu eksekusi meningkat drastis (dari ~80 detik menjadi ~280 detik).

Sementara itu, menambah jumlah iterasi dari 100 menjadi 300 tidak selalu menjamin perbaikan cost yang signifikan. GA dengan populasi 100 sudah mencapai konvergensi di iterasi awal, sehingga tambahan iterasi hanya membuang waktu komputasi.

3.2. Saran

Berdasarkan hasil eksperimen, terdapat beberapa saran untuk pengembangan sistem selanjutnya:

1) **Hybrid Algorithm:** Menggabungkan GA dengan HC atau SA. GA digunakan untuk mencari area solusi yang baik, kemudian HC atau SA digunakan untuk melakukan pencarian lokal agar solusi dapat diperbaiki lebih lanjut.
2) **Penyesuaian Parameter SA:** Parameter seperti suhu awal (T) dan cooldownRate dapat dioptimalkan. Pendinginan yang terlalu cepat dapat membuat SA lebih mudah terjebak pada local optimum, sedangkan pendinginan yang terlalu lambat meningkatkan waktu komputasi.
3) **Optimasi Komputasi GA:** Karena GA membutuhkan waktu komputasi paling besar, prosesnya dapat dipercepat dengan multiprocessing untuk evaluasi populasi atau caching pada fungsi calculateCost agar perhitungan state yang sama tidak dilakukan berulang.
4) **Repair Mechanism:** Untuk menangani solusi yang masih memiliki pelanggaran seperti clipping atau paket melayang, dapat ditambahkan repair mechanism setelah algoritma selesai. Mekanisme ini dapat memindahkan paket yang melanggar ke posisi valid atau ke warehouse sehingga solusi akhir menjadi feasible.

4. Pembagian Tugas

Nama - NIM	Tugas
Eliana Natalie Widjojo - 13524116	Membuat objective dan helper function, membuat algoritma GA, debugging, menulis laporan
Safira Berlianti - 13524128	Membuat states, algoritma HC, debugging, menulis laporan
Varistha Devi - 13524135	Membuat main dan display, membuat algoritma SA, debugging, menulis laporan

5. Referensi

[1] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: Beyond Classical Search: Classical vs Local Search [Accessed: Jul. 27, 2026].

[2] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Jul. 27, 2026].

[3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Jul. 27, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: Beyond Classical Search: Simulated Annealing [Accessed: Jul. 27, 2026].

[5] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: Beyond Classical Search: Genetic Algorithm [Accessed: Jul. 27, 2026].

6. Lampiran Form Penggunaan Generative AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:
Nama/NIM: Elianan Natalie W./13524116, Safira Berlianti/13524128, Varistha Devi/13524135
Fakultas/Sekolah: STEI-K
Kode Mata Kuliah: IF3170
Nama Mata Kuliah: Inteligensi Artifisial
Judul Tugas/Proposal: Tugas Besar 1
Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	Pemeriksaan tata bahasa supaya lebih baku
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Pembuatan laporan bagian saran
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Membantu debugging dan menyusun kode untuk visualisasi program
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI,	Tidak	

atau sejenisnya.		
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober

Eliana Natalie Widjojo
13524116

Safira Berlianti
13524128

Varistha Devi
13524135